

K0033 普通高等教育招生考试

专升本 计算机基础与程序设计（C 语言程序设计）（精编讲义）

目录

第 1 章 程序设计与 C 语言	1
1.1 计算机与程序、程序设计语言	2
1.2 C 语言的出现和发展过程	3
1.3 简单的 C 语言程序	4
1.4 运行 C 程序的步骤与方法	9
第 2 章 数据的存储与运算	14
2.1 数据在计算机中是怎样存储的	15
2.2 整型数据的运算与分析	17
2.3 实型数据的运算与分析	20
2.4 字符型数据的运算	23
2.5 符号常量	27
2.6 算术运算符和算术表达式	29
2.7 C 运算符和 C 表达式	32
2.8 提高部分	34
第 3 章 最简单的 C 程序设计——顺序程序设计	39
3.1 算法是程序的灵魂	40
3.2 程序的三种基本结构	45
3.3 C 语句综述	46
3.4 赋值表达式和赋值语句	48
3.5 数据输入输出的概念	51
3.6 字符数据的输入输出	52
3.7 简单的格式输入与输出	55
3.8 顺序结构程序设计举例	60
3.9 提高部分	62
第 4 章 选择结构程序设计	68
4.1 条件判断	69
4.2 用 if 语句实现选择结构	74

4.3 利用 switch 语句实现多分支选择结构	81
4.4 程序综合举例	85
4.5 提高部分	88
第 5 章 循环结构程序设计	91
5.1 程序中需要使用循环结构	92
5.2 while 和 do while 语句实现循环	93
5.3 for 语句实现循环	98
5.4 循环嵌套	103
5.5 提前结束循环	104
5.6 几种循环比较	108
5.7 循环程序综合举例	109
5.8 提高部分	113
第 6 章 利用数组处理批量数据	118
6.1 为什么要用数组	119
6.2 怎样定义和引用一维数组	120
6.3 怎样定义和引用二维数组	126
6.4 字符数组	131
6.5 提高部分	139
第 7 章 用函数实现模块化程序设计	145
7.1 函数是什么	145
7.2 函数的定义和调用	149
7.3 函数的嵌套调用和递归调用	155
7.4 数组作为函数参数	163
7.5 变量的作用域和生存期	169
7.6 内部函数和外部函数	178
7.7 提高部分	181
第 8 章 善于使用指针	184
8.1 什么是指针	185
8.2 指针变量	187
8.3 通过指针引用数组	194
8.3.1 数组元素的指针	194

8.4 通过指针引用字符串	202
8.5 提高部分	210
第 9 章使用结构体类型处理组合数据---用户自定义数据类型	215
9.1 定义和使用结构体变量	216
9.2 结构体数组	223
9.3 结构体指针	227
9.4 用结构体变量和结构体变量的指针作函数参数	232
9.5 用指针处理链表	236
9.6 提高部分	242
第 10 章 利用文件保存数据	247
10.1 C 文件的有关概念	248
10.2 文件的打开与关闭	252
10.3 文件的顺序读写	254
10.4 文件的随机读写	263
10.5 提高部分	267

对应网课（每年按照最新考纲更新）

【优课必过】网校

<http://zikao.p1tao.com/mall/detail?id=26359621>

开课加叶学长微信prince-zip咨询

课程: [44-K0033] 专升本计算机基础与程序设计

课程精讲班

共99讲 总时长2695分钟

普通高等教育招生考试
专升本 计算机基础与程序设计
(数据结构)

米秋香老师
计算机基础与程序设计-数据结构

00:02 / 49:24

线路1 超清 倍速

扫一扫看课



正版网课内部价开通
加叶学长微信

第 1 章 程序设计与 C 语言

本章重难点分析

本章主要介绍计算机与程序，程序设计语言的的基本概念。C 语言的出现背景及发展过程；简单 C 程序的举例介绍以及运行 C 程序的注意事项和基本步骤。

本章内容注意以下几点：

简单 C 程序的基本概念和组成	重点
-----------------	----

C 程序的步骤与运行	重点
------------	----

1.1 计算机与程序、程序设计语言

计算机不是天生自动工作的

计算机由人们事先编好的程序控制

人和计算机通过程序设计语言沟通

➤ 计算机产生初期使用机器语言

一个型号机器语言的指令的集合——机器语言

机器语言是由 0 和 1 组成的指令

机器语言是紧密依赖于计算机的硬件的

机器语言难学、难记、难写、难修改、难维护

在不同计算机之间互不通用

➤ 20 世纪 50 年代出现了高级语言

比较接近于人的自然语言（英文）和数学语言。

高级语言直观易学，易理解，易修改，易维护，易推广，通用性强。

用高级语言编写的程序，必须先翻译成机器语言程序，此翻译工作由编译系统实现。

1.2 C 语言的出现和发展过程

- C 语言是国际上广泛流行的计算机高级语言
- 最初的 C 语言只是为编写 UNIX 操作系统提供一种工作语言而设计的
- 随着 UNIX 的日益广泛使用，C 语言也迅速得到推广
- C 语言和 UNIX 在发展过程中相辅相成
- 1978 年以后，C 语言先后移植到大、中、小、微型计算机上，成为世界上应用最广泛的程序设计高级语言
- 1983 年，美国国家标准化协会（ANSI）根据 C 语言问世以来各种版本对 C 语言的发展和扩充，制定了新的标准草案，即 83 ANSI C
- ANSI C 比原来的标准 C 有了很大的发展
- 1989 年，ANSI 又公布了一个新的 C 语言标准—ANSI X3.159-1989（简称 C89）
- 1999 年，ISO 又对 C 语言标准进行修订，在基本保留原来的 C 语言特征的基础上，增加了一些面向对象的特征，命名为 ISO/IEC 9899:1999，简称为 C99
- 本书按照 **C99 标准** 的规定介绍程序设计
- C 语言功能强大、使用灵活，既可用于编写应用软件，又能用于编写系统软件
- 学习和使用 C 语言的人越来越多，成了学习和使用人数最多的一种计算机语言
- 熟练掌握 C 语言成为计算机开发人员的一项基本功

讲解归纳与举例

1.既具有低级语言的特点，又具有高级语言的特点的语言是（ ）。

A JAVA 语言

B C 语言

C 汇编语言

D 机器语言

【正确答案:B】

【答案解析】:C 语言的特点

1.3 简单的 C 语言程序

例 1.1 要求在屏幕上输出以下一行信息。

```
#include <stdio.h>
```

```
Void main ( )
```

```
{  
    printf ( "This is a C program.\n" );  
}
```

函数的名字，表示主函数

C程序必须有一个 main 函数

```
#include <stdio.h>
```

```
void main ( )
```

```
{  
    printf ( "This is a C program.\n" );  
}
```

主函数类型

```
#include <stdio.h>
```

```
void main ( )
```

```
{  
    printf ( "This is a C program.\n" );  
}
```

函数体

```
#include <stdio.h>
```

```
void main ( )
```

```
{  
    printf ( "This is a C program.\n" );  
}
```

输出函数

输出语句

```
#include <stdio.h>
```

```
void main ( )
```

```
{  
    printf ( "This is a C program.\n" );  
}
```

换行符

This is a C program.

```
#include <stdio.h>
void main ( )
{
    printf ( "This is a C program.\n" );
}
```

用到函数库中的输入输出函数时

表示语句结束

This is a C program.

例 1.2 求两个整数之和

```
#include <stdio.h>
void main ( )
{
    int a, b, sum; //定义整型变量a, b, sum
    a = 123; b = 456; //对变量a, b赋值
    sum = a + b; //将a与b的和赋给sum
    printf ( "sum is %d\n", sum );
}
```

```
#include <stdio.h>
void main ( )
{
    int a, b, sum;
    a = 123; b = 456;
    sum = a + b;
    printf ( "sum is %d\n", sum );
}
```

用sum的值替代

希望输出的字符 sum is 579

// 注释部分

- ◆ 注释可以用汉字或英文字符表示
- ◆ 注释只是给人看的，对编译和运行不起作用
- ◆ 注释可以出现在一行中的最右侧，也可以单独成为一行

例 1.3 求两个整数中的较大者。

```
#include <stdio.h>
void main ( )
{
    int max (int x, int y) ;
    int a, b, c;
    scanf ( "%d, %d", &a, &b) ;
    c = max (a, b) ;
    printf ("max=%d\n", c) ;
}

int max (int x,
        int y)
{
    int z;
    if (x > y) z =
        x;
    else z = y;
    return (z) ;
}
```

被调函数

主函数

```
#include <stdio.h>
Void main ( )
{
    int max (int x, int y) ;
    int a, b, c;
    scanf ( "%d, %d", &a, &b) ;
    c = max (a, b) ;
    printf ("max=%d\n", c) ;
}
```

将x和y中较大者值返回给主函数

```
#include <stdio.h>
void main ( )
{
    int max (int x, int y) ;
    int a, b, c;
    scanf ( "%d, %d", &a, &b) ;
    c = max (a, b) ;
    printf ("max=%d\n", c) ;
}

int max (int x,
        int y)
{
    int z;
    if (x > y) z =
        x;
    else z = y;
    return (z) ;
}
```

因max函数的定义在main函数之后，
需声明

```
#include <stdio.h>
void main ( )
{
    int max (int x, int y) ;
    int a, b, c;
    scanf ( "%d, %d", &a, &b) ;
    c = max (a, b) ;
    printf ("max=%d\n", c) ;
}

int max (int x,
        int y)
{
    int z;
    if (x > y) z =
        x;
    else z = y;
    return (z) ;
}
```

8, 5 ✓
max=8

结论：

➤ C 程序是由函数构成的

C 源程序必须包含一个 main 函数

可以包含若干个其他函数

函数是 C 程序的基本单位

被调函数可以是库函数，也可以是用户编制设计的函数

程序全部工作都由各个函数分别完成

C 语言容易实现程序的模块化

➤ 一个函数由两个部分组成

◆ 函数首部

int max (int x, int y)

函数类型

函数名

参数类型

参数名

函数名后面必须跟一对圆括号

函数可以没有参数，如: void main ()

◆ 函数体

● 声明部分

可以没有声明部分

➤ 定义在本函数中所用到的变量

➤ 对本函数所调用函数进行声明

● 执行部分

➤ 由若干个语句组成

◆ 函数体

```
void dump ( )
```

```
{  
    }  
}
```

可以是空函数

- C 程序是由函数构成的
- 一个函数由两个部分组成
- 程序总是从 main 函数开始执行
- C 程序书写格式自由
- 每个语句和数据声明的最后必须有分号
- C 语言本身没有输入输出语句
- //可以对程序中的任何一行或数行做注释

讲解归纳与举例

1.C 语言规定，在一个源程序中，main 函数的位置（ ）。

- A 必须在最开始
- B 可以放在系统库函数之前
- C 必须在系统调用的库函数后面
- D 必须在最后

【正确答案:C】

【答案解析】：main 函数的理解

2.以下叙述中正确的是（ ）。

- A C 程序中注释部分可以出现在程序中任意合适的地方
- B 花括号 "{" 和 "}" 只能作为函数体的定界符
- C 构成 C 程序的基本单位是函数，所有函数名都可以由用户命名
- D 分号是 C 语句之间的分隔符，不是语句的一部分

【正确答案:A】

【答案解析】：C 程序的基本概念和组成

1.4 运行 C 程序的步骤与方法

1.4.1 运行 C 程序的步骤

所谓程序，就是一组计算机能识别和执行的指令。

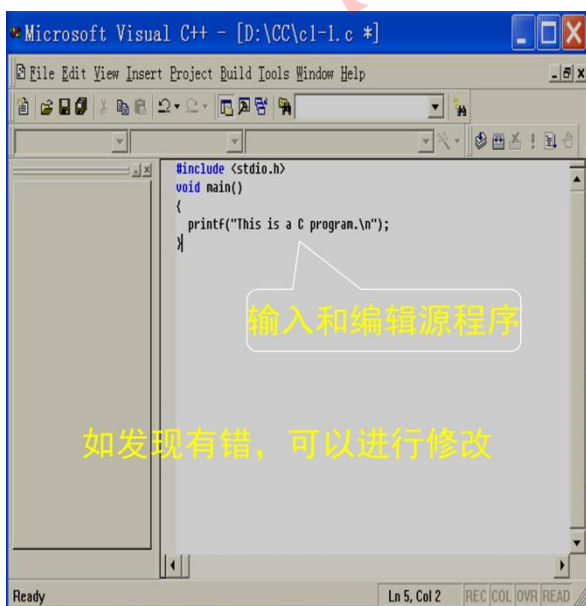
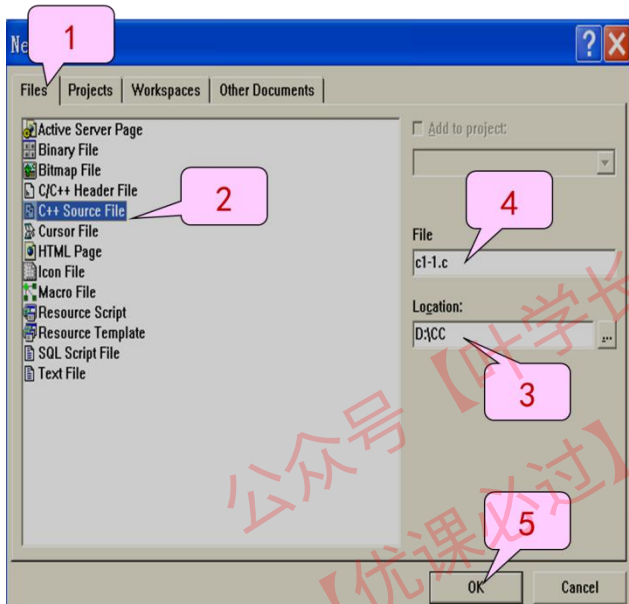
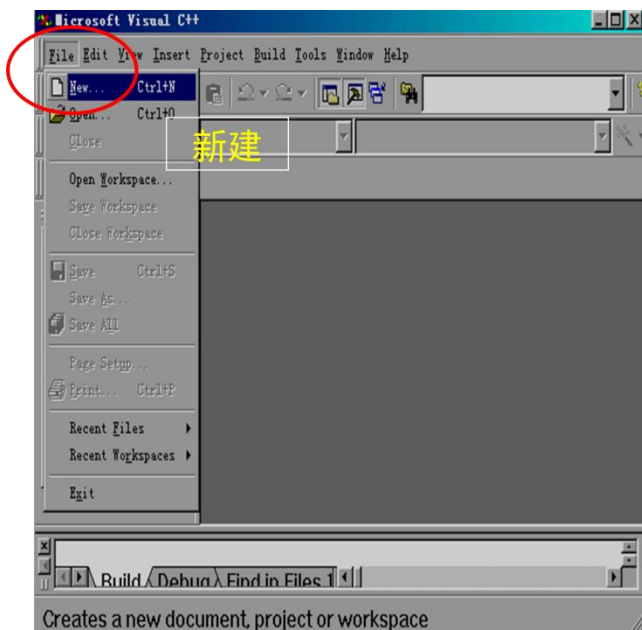
- 上机输入和编辑源程序（.c 文件）
- 对源程序进行编译（.obj 文件）
- 进行连接处理（.exe 文件）
- 运行可执行程序，得到运行结果

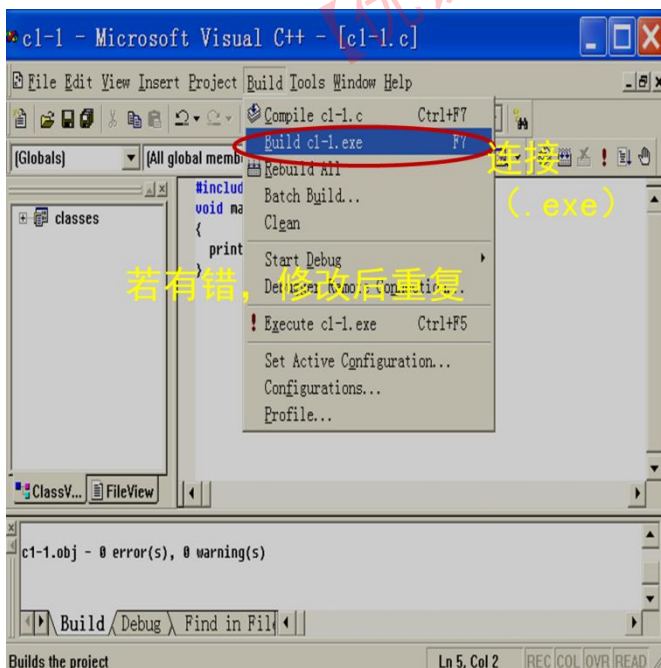
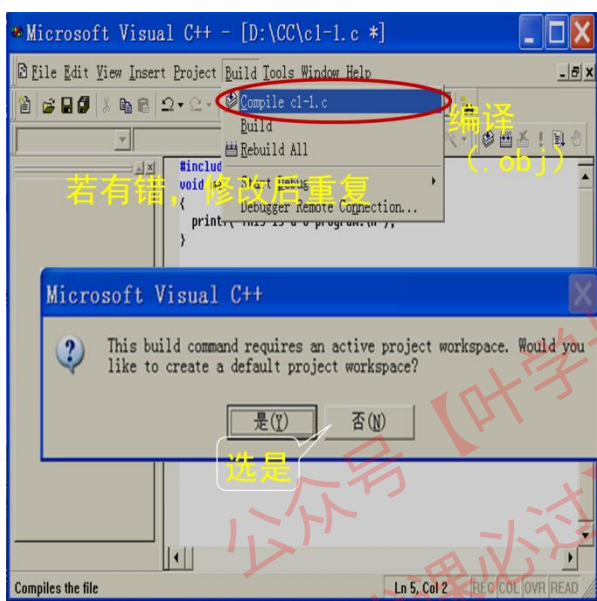
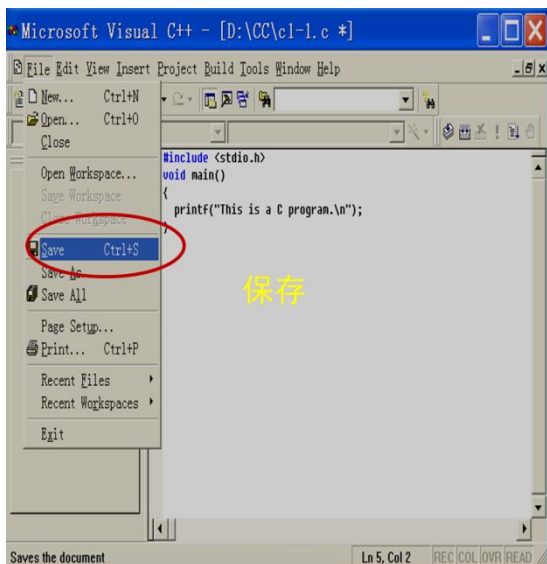
说明:以上过程参见教材中图 1.1（P7）

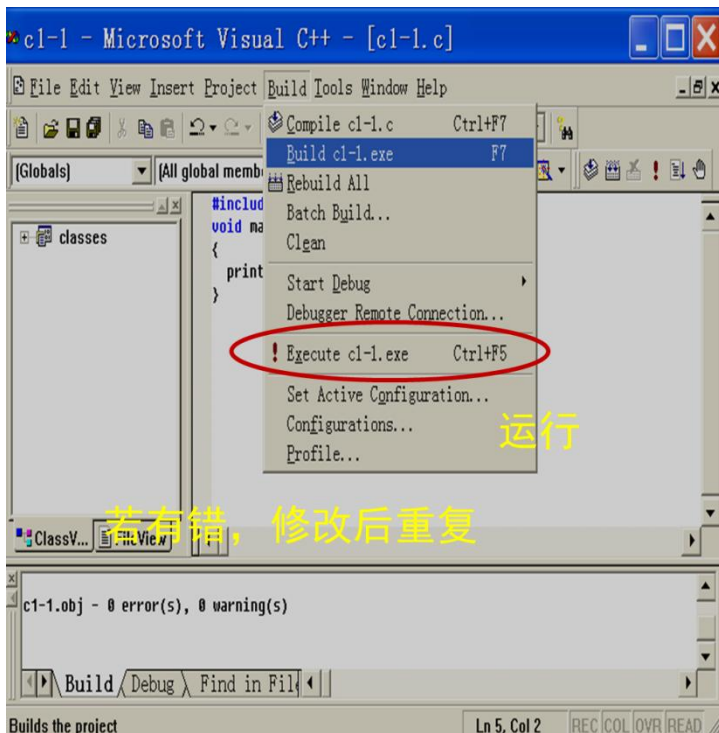
1.4.2 上机运行 C 程序的方法

- 可用不同的编译系统对 C 程序进行操作
- 本节介绍在 Visual C++ 6.0 中怎样编辑、编译、连接和运行 C 程序
- 安装 VC++ 6.0
- ◆ VC++ 6.0 有英文版和中文版
- ◆ 必须事先安装 VC++ 6.0 系统
- ◆ 安装后最好在桌面上设立快捷方式图标，以方便使用
- VC++6.0 集成环境



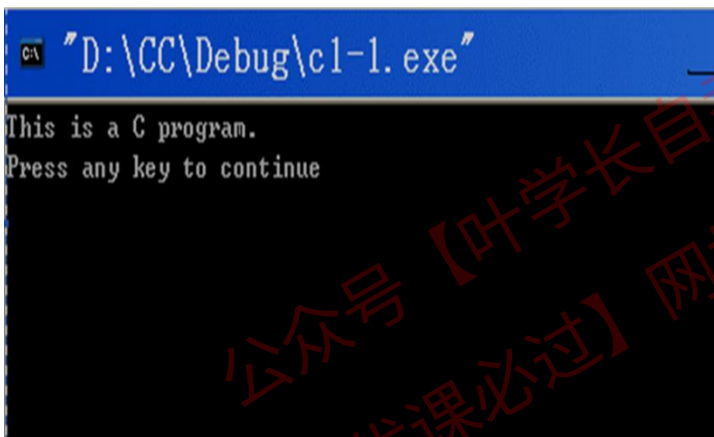






若有错，修改后重复

运行



如果不再进行其他的处理，应当选择文件→关闭工作区，结束对该程序的操作

讲解归纳与举例

1.C 文件通过了编译、连接之后，生成一个后缀名为_____的文件。

【正确答案:.exe】

【答案解析】：C 程序步骤与运行

2.C 文件通过了编译之后，生成一个后缀名为_____的文件。

【正确答案:.obj】

【答案解析】：C 程序步骤与运行

3.C 语言所编写的源程序，需要保存在一个后缀名为 _____ 的文件中。

【正确答案:c】

【答案解析】：C 程序步骤与运行

本章小结

本章主要介绍计算机与程序，程序设计语言的的基本概念。C 语言的出现背景及发展过程；简单 C 程序的举例介绍以及运行 C 程序的注意事项和基本步骤。

公众号【叶学长自考资料网】
【优课必过】网校出品

第2章 数据的存储与运算

本章重难点分析

本章主要介绍 C 语言中的数据类型、常量和变量的表现形式及定义、字符型数据的运算、算术运算符及算术表达式、C 运算符和 C 表达式、运算符的优先级等。

本章内容注意以下几点：

基本数据类型	重点
算术运算符及算术表达式	重点
C 运算符和 C 表达式	重点
运算符的优先级	难点

公众号【叶学长自考资料网】
【优课必过】网校出品

2.1 数据在计算机中是怎样存储的

2.1.1 数据在计算机中是以二进制形式存储的

计算机内部的信息都是用**二进制**表示

计算机存储器是用半导体集成电路构成的

有两种稳定的工作状态：“导通”与“截止”，

即电脉冲的“有”与“无

分别用“**1**”和“**0**”表示

若相邻的 8 个二极管元件中第 1, 3, 5, 7 个元件处于“导通”状态，第 2, 4, 6, 8 个元件处于“截止”状态，用“10101010”表示。

导通 截止 导通 截止 导通 截止 导通 截止

1 0 1 0 1 0 1 0

十进制数	二进制数
6	110
7	111
8	1000
9	1001
10	1010

$$1010 \Leftrightarrow 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$$

2.1.2 位、字节和地址

- **位 (bit)**，又称“**比特**”。每一个二极管元件称为一个“二进制位”，是存储信息的最小单位。它的值是“1”或“0”。
- **字节 (byte)**，又称“**拜特**”。一般将 8 个“二进制位”组织成一组，称为“字节”。这是人们最常用的存储单位。
- **地址**。计算机的存储器包含许多存储单元，操作系统把所有存储单元以字节为单位编号。

2001

2001	3
2002	4
2003	5
2004	7
2005	8
2006	9
	⋮

2.1.3 不同类型数据的存储方式

➤ 整数的存储方式

一个十进制整数，先转换为二进制形式

如整数 10，以二进制形式表示是 1010。

用一字节存放

0	0	0	0	1	0	1	0
---	---	---	---	---	---	---	---

➤ 实数的存储形式

对于实数，一律采用指数形式存储

123.456 标准化指数形式为 0.123456×10^3

+	. 123456	+	3
---	----------	---	---

数符	数值部分	指数符号	指数
----	------	------	----

➤ 字符的存储方式

对于字符，按 ASCII 代码存放

A 的 ASCII 代码是 65 (1000001)

0	1	0	0	0	0	0	1
---	---	---	---	---	---	---	---

2.2 整型数据的运算与分析

2.2.1 整型数据运算程序举例和分析

例 2.1 鸡兔同笼。在一个笼子里同时养着一些鸡和兔子，你想了解有多少只鸡和多少只兔，主人对你说：我只告诉你鸡和兔的总头数是 16，总脚数是 40，你能不能自己计算有多少只鸡和多少只兔？

解题思路：

设 x 代表鸡的数量， y 代表兔的数量，总头数为 h ，总脚数为 f

列出下面的方程式：

$$x + y = h \quad (1)$$

$$2x + 4y = f \quad (2)$$

解方程得到：

$$y = \frac{f - 2h}{2} \quad x = h - y$$

```
#include <stdio.h>
```

```
void main ( )
```

```
{ int h, f, x, y;    定义整型变量h, f, x, y
  h=16;  f=40;    y= (f-2* h)/2;  计算兔的个数
  x=h-y;          计算鸡的个数
  printf ( "%d%d\n", x, y);      输出鸡和兔子的数量
}
```

输出语句：

```
printf ( "cock=%d, rabbit=%d\n", x, y);
```

定义变量的一般形式是：

类型名 变量名；

例如：int h, f, x, y;

变量都必须在使用前定义，指定其类型

赋值：h=37; f=88;

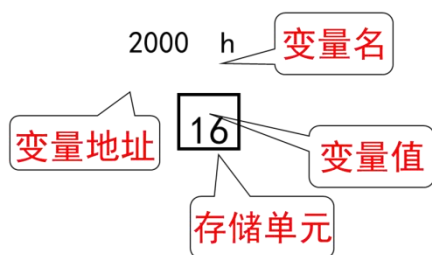
2.2.2 整型常量与整型变量

➤ 常量和变量

常量：是指在程序运行过程中其值不能改变的量

变量：是指在程序运行过程中其值可以改变的量

要注意区别变量名和变量值



➤ 变量名的取名规则：

变量名第一个字符必须是字母或下划线，其后字符必须是字母、数字或下划线。

合法：sum, average, _total, Class, day, month, Student_name, tan

不合法：Zhang-sun, Student' s, 263.com, \$123, #33, 3D64

1. 变量名第一个字符必须是字母或下划线，其后字符必须是字母、数字或下划线。
2. 大小写字母代表不同的字符，一般地，程序中的变量名用小写字母表示。
3. 变量名的长度不是无限的。
4. 变量名尽量简单易记、见名知意。
5. 在同一程序或函数中，不同变量不能取相同名。

➤ 变量必须“先定义，后使用”

1. 根据定义变量时指定的类型，编译系统为变量分配相应的存储单元。
2. 凡未被事先定义的，系统不把它认作变量名，这就能保证程序中变量名使用得正确。
3. 指定了每一变量属于一个类型，就便于在编译时据此检查在程序中要求对该变量进行的运算是否合法。

➤ 整型常量的表示形式

除了常用的十进制形式外，还允许使用八进制形式和十六进制形式表示的数。

➤ 整型变量的种类

基本整型，类型名为 int

长整型，类型名为 long int

短整型，类型名为 short int

.....

讲解归纳与举例

1.以下（ ）是正确的变量名。

- A 5f B if
C f.5 D _f5

【正确答案：D】

【答案解析】：P25 变量命名规则。

2.以下不能定义为用户标识符的是（ ）。

- A s1 B if2
C _3com_ D int

【正确答案：D】

【答案解析】：P25 变量命名规则。

Int 是 C 语言中的关键字。不能作为变量名。

2.3 实型数据的运算与分析

2.3.1 实型数据的运算举例

例 2.2 分期付款的计算。张先生为购房，向银行贷款，贷款额为 D 元，每月准备还 P 元，月利率为 R ，求需要多少个月才能还清。

$$m = \frac{\log(p) - \log(p - d \times r)}{\log(1 + r)}$$

m 是还清贷款所需月数

$d=324500$, $p=3245$, $r=0.8\%$

(1) d 和 p 是整数， r 是一个小数，因此程序中要分别定义整型变量和实型变量。

(2) 公式中用到对数 $\log$ ，这个对数是以 10 为底的。对数 $\log$ 的函数 $\log_{10}$ (附录 E)。

```
#include <stdio.h>
#include <math.h>
void main ( )
{ int d, p;    定义整型变量d, p
  float r, m;  定义实型变量r, m
  d=324500;   p=3245;   r=0.008;  赋值
  m= (log10 (p)-log10 (p-d*r))/log10 (1+r);  求m的值
  printf ( "month=%f\n", m);
  printf ( "total=%f\n", m*p);
}
```

用到数学函数时

实数用%f格式输出

```
#include <stdio.h>
#include <math.h>
void main ( )
{ int d=324500, p=3245;  定义变量时赋初值
  double r=0.008, m;
  m= (log10 (p)-log10 (p-d*r))/log10 (1+r);
  printf ( "month=%f\n", m);
  printf ( "total=%f\n", m*p);
}
```

2.3.2 实型常量的表示形式

➤ 实数在计算机语言中常称为浮点数

十进制小数形式

如：0.123，123.23，0.0

指数形式

注意：字母 e (E)之前必须有数字，e (E)后面的指数必须为整数。

如：123e3 或 123E3

2.3.3 实型变量

➤ 实型变量的分类

单精度实型变量（float 型，4 字节）

双精度实型变量（double 型，8 字节）

长双精度实型变量（long double 型，8/16 字节）

提示：sizeof（类型名）或者

sizeof（变量名）

➤ 实型数据的舍入误差

实型变量是由有限的存储单元组成的能提供的有效数字总是有限的。在有效位以外的数字将被舍去会产生一些误差。

例2.3 实型数据的舍入误差

```
#include <stdio.h>
```

```
void main（）
```

```
{ float a;
```

```
  a = 1234.1415926;
```

```
  printf（"a=%f\n", a);
```

```
}
```

a是单精度浮点类型。6-7位有效位

a=1234.141602

➤ 把实数按双精度数处理

把程序中的实常量处理成双精度型，分配 8 个字节（15-16 位有效位），以提高精度。一般对精度有要求的定义为 double 类型。

最好：double a;

a = 1234.1415926

讲解归纳与举例

1. 下列常量中，为不合法的实型常量表示的是（ ）。

A 0032 B 0.0

C 0.3242E8 D E3

【正确答案：D】

【答案解析】：P30 用指数形式表示，字母 e (E) 之前必须有数字，e (E) 后面的指数必须是整数。D 不合法。

2.4 字符型数据的运算

2.4.1 字符数据运算的简单例子

例 2.4 逐个输出英文字母 C, H, I, N, A。然后按反序输出，即 A, N, I, H, C。

```
#include <stdio.h>

void main ( )
{ char a= 'C' , b= 'H' ,
  c= 'I' , d= 'N' , e= 'A' ;
  printf ( "%c%c%c%c%c\n" , a, b, c, d, e);
  printf ( "%c%c%c%c%c\n" , e, d, c, b, a);
}
```

2.4.2 字符常量和字符变量

➤ 字符常量

字符常量是用单撇号括起来的一个字符

英文字母可以作为字符常量

键盘上的字符都可以作为字符常量

小写字母和大写字母是不同的字符常量

➤ 转义字符

转义字符必须以反斜杠“\”开头

“\”后只能有一个字符（或代表字符的 8 进制或 16 进制数）

\n 换行

\t 跳到下一个输出区

\b 消除前一个已输出的字符

\r 将当前的输出位置返回在本行开头

\f 将当前的输出位置移到下页的开头

\0 常用于字符串中，作为串结束标志

\\ 代表一个反斜杠字符“\”

\' 代表一个单撇号字符

\" 代表一个双撇号字符

\ddd 1~3 位 8 进制数所代表的字符

\xhh 1~2 位 16 进制数所代表的字符

➤ 字符变量

用来存放字符常量，只能放一个字符。

字符变量的定义形式：

char 字符变量列表；

char c1;

c1 = 'a' ;

➤ 字符数据与整型数据在一定条件下通用

char c = 'a' ; 与 **char c=97;** 等价

例 2.5 将两个整数分别赋给两个字符变量，再将字符数据按字符和整数形式输出。

```
#include <stdio.h>
```

```
void main ( )
```

```
{ char c1=97, c2=98; char c1='a', c2='b';
```

```
printf ( "%c %c\n", c1, c2); a b
```

```
printf ( "%d %d\n", c1, c2); 97 98
```

```
}
```

0	1	1	0	0	0	0	1
---	---	---	---	---	---	---	---

按%c输出: a

按%d输出: 97

```
printf ( "%c %c\n", 97+1, ' a' +1);
```

输出什么? **b b**

```
printf ( "%d %d\n", 97+1, ' a' +1);
```

输出什么? **98 98**

例2.6 将小写字母转换为大写字母。

```
#include <stdio.h>
```

```
void main ( )
```

```
{ char c1='a', c2='b';
```

```
  c1=c1-32;
```

```
  c2=c2-32;
```

```
  printf ("%c, %c\n", c1, c2);
```

A, B

```
}
```

a—97 A—65

b—98 B—66

c—99 C—67

……规律是？

2.4.3 字符串常量

➤ 字符串常量是一对双撇号括起来的字符序列

" How do you do."

" CHINA "

" a "

都是合法的字符串

' a ' 是字符常量

" a " 是字符串常量

二者的含义是不同的

```
char c;  
c= ' a ' ; 正确
```

```
char c;  
c= " a " ; 错误
```

存放字符' a ' 的存储单元 a

存放字符串" a " 的存储单元 a \0

讲解归纳与举例

1.已定义c为字符型变量，则下列语句中正确的是（ ）。

A c= ' 97' ; B c=""97"";

C c=97; D c="a";

【正确答案：C】

【答案解析】：P33 双引号是表示字符串。

小写字母 a 的 ASCII 码值是 97。

2.若有以下程序运行后的输出结果是 。

```
main ( )  
{   char  a;  
    a=' H' -' A' +' 0' ;  
    printf ("%c ", a);   }
```

【正确答案： 7】

【答案解析】： ' H' -' A' ，用字母 H 和 A 的
ASCII 码值参与运算。大写字母 A 的 ASCII 码值是 65。

3.判断题：字符常量是用单撇号括起来的一个字符，这个字符可以是任意的。

【正确答案： 错 】

【答案解析】：字符常量

4.判断题：转义字符是一种常量。

【正确答案： 对 】

【答案解析】：字符常量

2.5 符号常量

2.5.1 为什么要用符号常量

例 2.7 求以 r 为半径的圆周长 c 和圆面积 s ，再求以 r 为半径的圆球体积 v 。

$$c = 2\pi r \quad c=2*3.1415926*r;$$

$$s = \pi r^2 \quad s=3.1415926*r*r;$$

$$v = \frac{4}{3}\pi r^3 \quad v=4/3*3.1415926*r*r*r;$$

```
#define PI 3.1415926
```

```
#include <stdio.h>
```

```
#include <math.h>
```

```
void main ( )
```

```
{double r=3.67, c, s, v;
```

```
    c=2 * PI *r;
```

```
    s= PI * pow (r, 2);
```

```
    v= 4.0/3.0 * PI *pow (r, 3);
```

```
    printf ( "c=%f\ns=%f\nv=%f\n", c, s, v);
```

```
}
```

符号常量

2.5.2 符号常量的性质和使用方法

- `#define` 不是 C 语句，行末没有分号
- `#define` 是一个“预编译命令”
- 符号常量一般用大写，以示与变量区别
- 好处：

含义清楚

在需要改变一个常量时能做到“一改全改”

用符号常量能保护所代表的数据不被破坏

讲解归纳与举例

1.定义一个符号常量 pi 用来表示数学符号的方法是（ ）。

A define PI 3.14

B #define PI 3.14

C define PI=3.14

D #define PI=3.14

【正确答案：B】

【答案解析】：常量的定义。

公众号【叶学长自考资料网】
【优课必过】网校出品

2.6 算术运算符和算术表达式

2.6.1 算术运算符

1.基本的算术运算符：

+ ： 加法运算符

- ： 减法运算符

* ： 乘法运算符

/ ： 除法运算符

% ： 求余运算符（要求两侧均为整数。如 $19\%4$ ，结果为 3 ）

➤ 两个整数相除的结果为整数

◆ 如 $5/3$ 的结果值为 1，舍去小数部分

◆ 如果除数或被除数中有一个为负值，舍入方向不固定。例如， $-5/3$ ，有的系统中得到的结果为-1，在有的系统中则得到结果为-2

◆ VC++采取“向零取整”的方法

如 $5/3=1$ ， $-5/3=-1$ ，取整后向零靠拢

➤ 如果参加+、-、*、/ 运算的两个数中有一个数为 float 或 double 型，则结果都是 double 型，因为系统将所有 float 型数据都先转换为 double 型，然后进行运算。这是为了提高运算精度。

2. 自增、自减运算符：

➤ 作用是使变量的值增 1 或减 1

++i, --i: 在使用 i 之前，先使 i 的值加（减）1

i++, i--: 在使用 i 之后，使 i 的值加（减）1

2.6.2 算术表达式

➤ 用算术运算符和括号将运算对象（也称操作数）连接起来的、符合 C 语法规则的式子，称为 C 算术表达式。

➤ 运算对象包括常量、变量、函数等。

➤ 各类数值型数据间的混合运算

char 和 short 型转换为 int 型

float 型一律转换为 double 型

整型（包括 int, short, long）数据与 double 型数据进行运算，先将整型转换为 double 型

字节少的数据转换成字节多的类型。

➤ 各类数值型数据间的混合运算

$10 + 'a' + i * f - d / e$

表达式最终结果为什么类型？

i 为整型 f 为 float 型 d 为 double 型 e 为 long 型
--

自动转换！

➤ 强制类型转换

在表达式中也可以利用“强制类型转换”运算符将数据转换成所需的类型

$(double)a$ （将 a 转换成 double 类型）

$(int)(x+y)$ （将 x+y 的值转换成 int 型）

$(float)(5\%3)$ （将 5%3 的值转换成 float 型）

强制类型转换的一般形式为：

$(类型名)(表达式)$

例 2.8 强制类型转换。

```
#include <stdio.h>
```

```
void main ( )
```

```
{ float f=3.6;    int i;
```

```
    i= (int)f;
```

```
    printf ("f=%f, i=%d\n", f, i);
```

```
}
```

$f=3.600000, i=3$

➤ 有两种类型转换

系统自动进行的类型转换

强制类型转换

$(类型名)(表达式)$

讲解归纳与举例

1. 设有 `int x=11`；则表达式 `(x++*1/3)` 的值是 ()。

- A 3 B 4
C 11 D 12

【正确答案：A】

【答案解析】：x 先用值 11 参与运算，
运算之后 x 再自增 1。

2. 若 a 是 int 型变量，则执行表达式 `a=24/3%3` 后 a 的值为 。

【正确答案：2】

【答案解析】：算术运算符的应用。

3. 下面程序的输出结果是 ()。

```
#include <stdio.h>
main ( )
{   int   x=5, y=3;
    printf ( "%d\n", y=x/y); }
```

- A 0 B 1
C 3 D 不确定的值

【正确答案：B】

【答案解析】：算术运算符的应用。

4. 执行下列程序的结果是 ()。

```
#include <stdio.h>
void main ( )
{float   x=3, y;
    y=++x * x++;
    printf ("%f\n", y); }
```

- A 9.000000 B 16.000000
C 1.000000 D 程序有错误

【正确答案：B】

【答案解析】：++x, x 变成了 4。Y=4*4=16，然后 x++，x 的值是 5。

2.7 C 运算符和 C 表达式

2.7.1 C 运算符

- (1) 算术运算符 (+ - * / %)
- (2) 关系运算符 (> < == >= <= !=)
- (3) 逻辑运算符 (! && ||)
- (4) 位运算符 (<< >> ~ | ^ &)
- (5) 赋值运算符 (= 及其扩展赋值运算符)
- (6) 条件运算符 (? :)
- (7) 逗号运算符 (,)
- (8) 指针运算符 (* &)
- (9) 求字节数运算符 (sizeof)
- (10) 强制类型转换运算符 ((类型))
- (11) 成员运算符 (. ->)
- (12) 下标运算符 ([])
- (13) 其他 (如函数调用运算符 ())

2.7.2 C 表达式

算术表达式。如 $2+6.7*3.5+\sin(0.5)$

关系表达式。如 $x>0$, $y<z+6$

逻辑表达式。如 $x>0 \&\& y>0$ (表示 $x>0$ 与 $y>0$ 同时成立, &&是逻辑运算符, 代表“与”)

赋值表达式。如 $a=5.6$

逗号表达式。如 $a=3, y=4, z=8$ 用逗号连接若干个表达式, 顺序执行这些表达式, 整个逗号表达式的值是最后一个表达式的值 (今为 8)

讲解归纳与举例

1. 设 x, y 和 z 均为 `int` 型变量, 则执行语句 $x = (y = (z = 10) + 5) - 5$; 后, x, y 和 z 的值是 ()。

A $x=10, y=15, z=10$

B $x=10, y=10, z=10$

C $x=10, y=10, z=15$

D $x=10, y=5, z=10$

【正确答案：A】

【答案解析】：运算符的应用。

2.执行下面程序中的输出语句后，输出的值是（ ）。

```
#include <stdio.h>
```

```
main ( )
```

```
{int a;
```

```
printf ("%d\n", (a=3*5, a*4, a+5)); }
```

A 65 B 20

C 15 D 10

【正确答案：B】

【答案解析】：逗号运算符，取最后一个表达式的值，注意相互之间的影响。

3.若 int a=2, b=3; float x=3.5, y=2.5; 则表达式

(float) (a+b)/2+ (int) x% (int)y 的值是。

【正确答案：3.500000】

【答案解析】：强制类型转换运算符。

4.判断题：表达式 $1/4+2.75$ 的值是 3。

【正确答案：错】

【答案解析】基本算术运算符。

2.8 提高部分

2.8.1 求补码的方法

- 计算机不论对正数和负数都按“补码”形式存放到存储单元
- 对于正数来说，补码就是该数的“原码”（该数的二进制形式）
- 负数的补码不是它的原码

-1的二进制表示，不考虑符号（原码）

0	0	0	0	0	0	0	1
---	---	---	---	---	---	---	---

诸位取反（反码）

1	1	1	1	1	1	1	0
---	---	---	---	---	---	---	---

反码加1（补码）

1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---

- 求一个负数的补码的方法是：

- 1.取该数（不考虑数的符号）的二进制形式，它就是原码
- 2.对该原码逐位“取反”（逐位把0变1，把1变0），得到其“反码”
- 3.将得到的反码加1

求-10的补码的方法是？

0	0	0	0	1	0	1	0
---	---	---	---	---	---	---	---

1	1	1	1	0	1	0	1
---	---	---	---	---	---	---	---

1	1	1	1	0	1	1	0
---	---	---	---	---	---	---	---

2.8.2 整型常量的表示形式

- 在C语言中，整常数可用3种形式表示：

10进制整数，如123，-456

8进制整数，特点是逢8进1。在程序中凡以0开头的数都认作8进制数。

16进制整数。特点是逢16进1。用0~9、a~f分别代替0~15。在程序中凡以0x开头的数都认作16进

制数。

```
a=83;          /* 10进制数 */  
a=0123;        /* 8进制数 */  
a=0x53;        /* 16进制数 */
```

2.8.3 整型变量的类型

➤ 可以定义和使用以下 6 种整型变量：

有符号基本整型 [signed] int

无符号基本整型 unsigned int

有符号短整型 [signed] short [int]

无符号短整型 unsigned short [int]

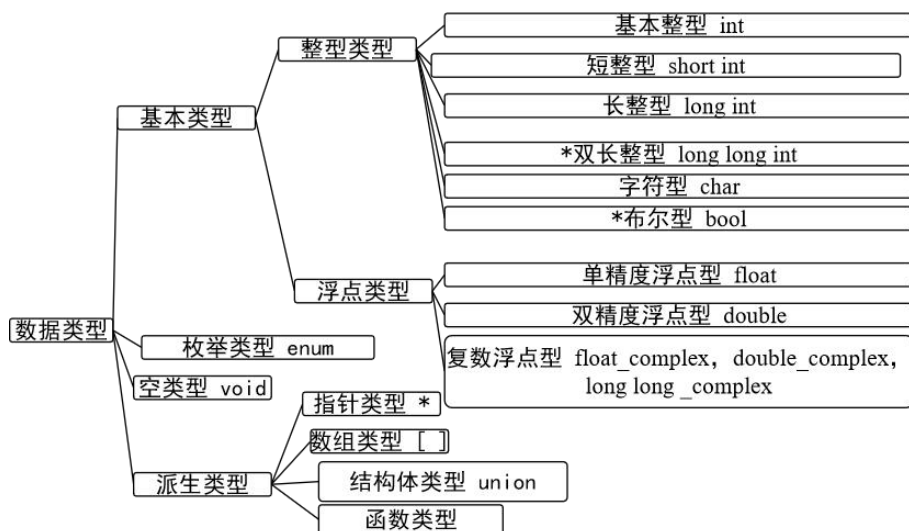
有符号长整型 [signed] long [int]

无符号长整型 unsigned long [int]

2.8.4 整型常量的类型

- 在计算机语言中，常量是有类型的，这也是计算机的特点。
- 数据是要存储的，不同类型的数据所分配的字节和存储方式是不同的。
- 整型变量有类型，因此整型常量也应该有类型，才能在赋值时匹配，根据常量的大小、数后面加的字母 l 或 L、u 或 U 判断其类型。

2.8.5 C 语言允许使用的数据类型



2.8.6 运算符的优先级与结合性

- 先按运算符的优先级别高低次序执行
- 如果优先级别相同，则按规定的“结合方向”处理

左结合性：运算对象先与左边的运算符结合，即自左至右，如 $x+y-z$

右结合性：运算对象先与右边的运算符结合，即自右向左，如 $a=b=c=d$

本章编程题

讲解归纳与举例

1. 计算存款利息。

计算存款利息。有 1000 元，想存一年。有 3 种方法可选：

- (1) 活期，年利率为 r_1 ；
- (2) 一年定期，年利率为 r_2 ；
- (3) 存两次半年定期，年利率为 r_3 。请分别计算出一年后按 3 种方法所得到的本息和。

```
#include <stdio.h>
```

```
int main ( )
```

```
{ float p0=1000, r1=0.0036,
```

```
  r2=0.0225, r3=0.0198, p1, p2, p3;
```

```
p1 = p0 * (1 + r1);
p2 = p0 * (1 + r2);
p3 = p0 * (1 + r3/2) * (1 + r3/2);
printf ("p1=%f\np2=%f\np3=%f\n", p1, p2, p3);
return 0; }
```

本章小结

(1) 变量命名规则:

只能由字母、数字、下划线构成，且不能数字开头

不能和关键字重名

区分大小写

(2) 符号常量 #define PRICE 30

(3) 基本数据类型 (用 sizeof(类型名)测试各类型的的字节数)

定义	变量类型	字节数	范围	scanf	printf
int	基本整型	2/4	-21 亿 ~ +21 亿	%d	%d
short	短整型	2	-32768 ~ +32767	%d	%d
long	长整型	4	-21 亿 ~ +21 亿	%ld	%ld
char	字符型	1	-128 ~ +127	%c	%c
float	单精度浮点型	4	6~7 位有效数字	%f	%f
double	双精度浮点型	8	15~16 位有效数字	%lf	%f

(4) 浮点型常量: .123、123.、123.0、0.0、123e3、.123e-3, 均默认为 double 类型。

(5) 字符型数据

用 ASCII 码表示

‘A’ ==65 ‘a’ ==97

‘0’ ==48 ‘ ’ ==32

```
printf (" %d %c\n", c, c);
```

➤ 义字符: \n \t \0 \\ \' \

➤ 大小写字母的转换, 大写字母+32=小写字母

➤ 字符串常量 “A” 和 ‘A’ 的区别

(6) 数据混合运算

10+ ‘a’ +i*6-d/e

char 和 short 默认转换成 int，float 默认转换为 double；

整数（int,short,long）数据与 double 数据进行运算，整数转换成 double。

字节少的转成字节多的数据类型。

（7）运算符

➤ 基本算术运算符

先 * / % 后 + -

/: 10/3 10/3.0

%: 求余，要求整数运算

➤ 强制类型转换 (int) x+y (int) (x+y)

➤ ++和--

➤ 逗号运算符

等于最后一个表达式的值，a = (3*5, 3*8) 相当于 a=3*8

➤ 运算符的优先级及结合性

公众号【叶学长自考资料网】
【优课必过】网校出品

第3章 最简单的C程序设计——顺序程序设计

本章重难点分析

要编写C程序，需要两个方面的知识：

一是根据所解问题的要求，设计出解题的具体步骤，这一步骤称为设计算法

二是用C语言写出程序，以便计算机能正确地执行

本章主要介绍算法的基本知识，最简单最基本的C语句，进而介绍简单的C语言程序，为后续编写复杂的程序打下基础。

本章内容注意以下几点：

算法的基本概念及表示

程序的三种基本结构

赋值语句及赋值表达式 **重点**

输入输出语句 **重点**

公众号【叶学长自考资料网】
【优课必过】网校出品

3.1 算法是程序的灵魂

3.1.1 什么是算法

➤ 一个程序包括以下两个方面的内容：

对数据的描述。在程序中要指定数据的类型和数据的组织形式，即数据结构

对操作的描述。即操作步骤，也就是**算法**

➤ 数据是操作的对象

➤ 操作的目的是对数据进行加工处理，以得到期望的结果

➤ 著名计算机科学家沃思（Niklaus Wirth）提出一个公式：

算法 + 数据结构 = 程序

➤ 一个程序除了算法和数据结构这主要要素外，还应当采用结构化程序设计方法进行程序设计，并且用某一种计算机语言表示。

➤ 算法、数据结构、程序设计方法和语言工具是一个程序设计人员应具备的知识。

算法是灵魂，数据结构是加工对象，语言是工具，编程需要采用合适的方法。

➤ 算法可以分为两大类：数值运算算法和非数值运算算法

➤ 算法是解决“做什么”和“怎么做”的问题

➤ 程序中的操作语句，是算法的体现

➤ 不了解算法就谈不上程序设计

3.1.2 怎样表示算法

1.用自然语言表示算法

自然语言就是人们日常使用的语言，可以是汉语、英语或其他语言

用自然语言表示通俗易懂，但文字冗长，容易出现歧义性

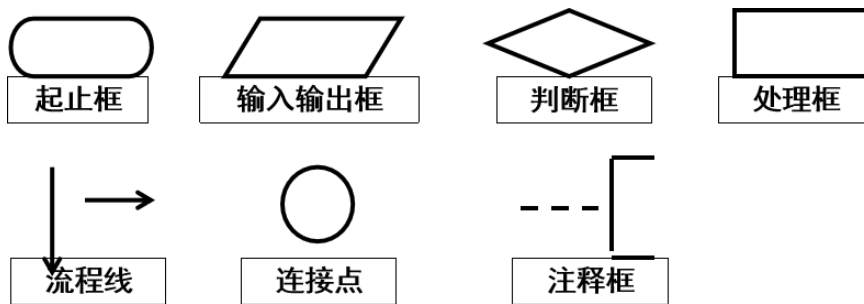
用自然语言描述包含分支和循环的算法，不很方便

除了很简单的问题外，一般不用自然语言

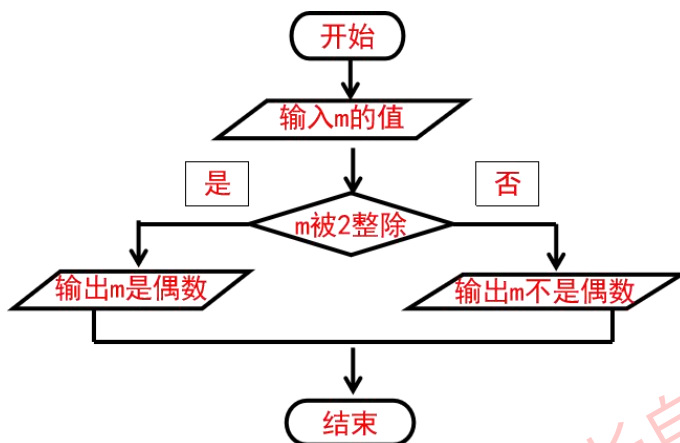
2.用流程图表示算法

流程图是用一些图框来表示各种操作

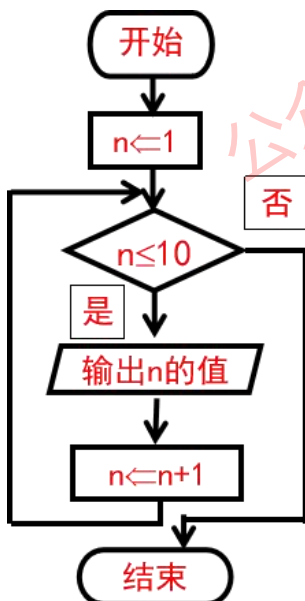
用图形表示算法，直观形象，易于理解



➤ 判断一个数是否偶数的算法



➤ 输出 1~10 的算法



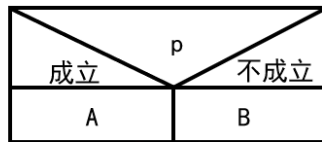
3.用 N-S 流程图表示算法

➤ N-S 流程图用以下的流程图符号



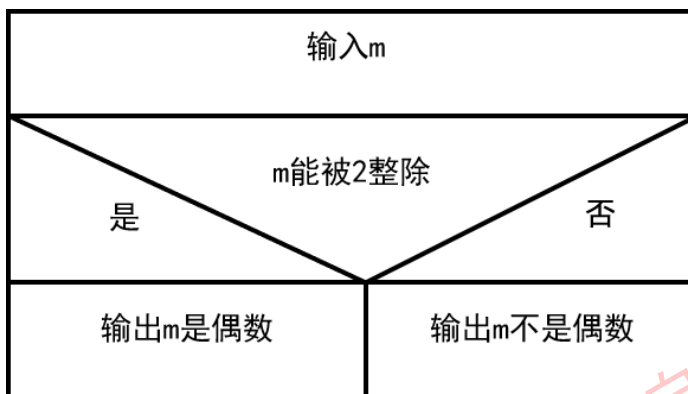
顺序结构

循环结构

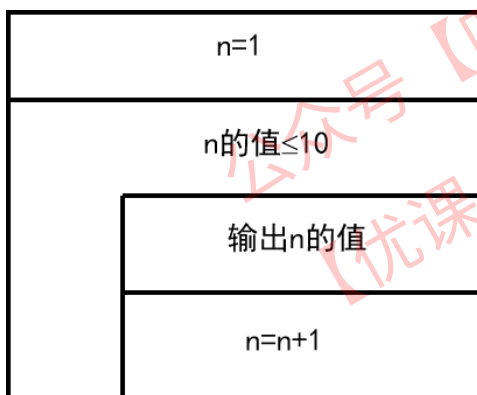


选择结构

- 判断一个数是否偶数的算法



- 输出 1~10 的算法



4.用伪代码表示算法

- 伪代码是用介于自然语言和计算机语言之间的文字和符号来描述算法。
- 用伪代码写算法并无固定的、严格的语法规则，可以用英文，也可以中英文混用。

“输出 x 的绝对值”的算法

```
if x is positive then
    print x
else
    print -x
```

若x为正 if x为正
 输出x print x
否则 else
 输出-x print -x

讲解归纳与举例

1.什么数据结构？算法？程序？

【参考答案：对数据的描述，在程序中指定数据的类型以及数据的组织形式，即数据结构。对操作的描述，即操作步骤，就是算法。】

数据结构+算法=程序。算法是灵魂，数据结构是加工对象，语言是工具，编程需要采用合适的方法。】

【答案解析】：基本概念

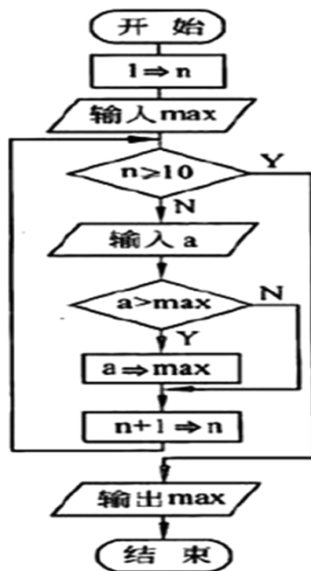
2.什么是伪代码？

【参考答案：伪代码是介于自然语言和计算机语言之间的文字和符号来描述算法。】

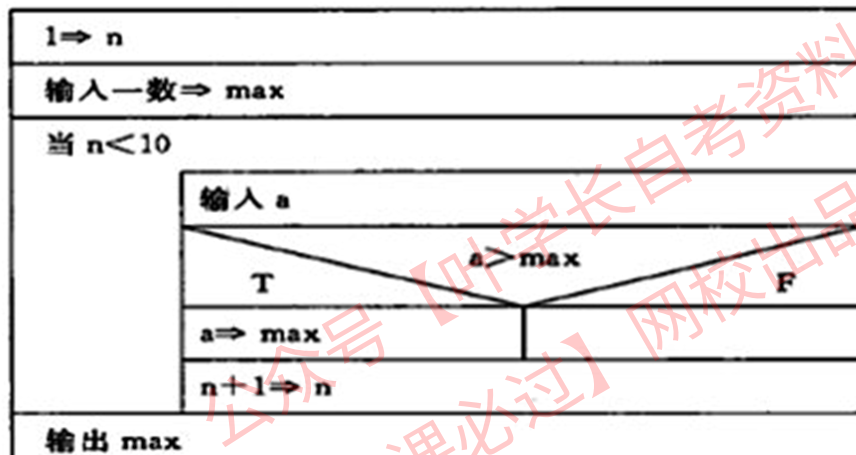
【答案解析】：基本概念

3.用流程图、NS 图、伪代码分别描述解决如下实际问题的算法。依次将 10 个数输入，求其中最大的数输出。

【正确答案：流程图】



【正确答案：NS 图】



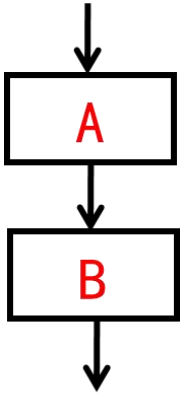
【正确答案：伪代码】

```

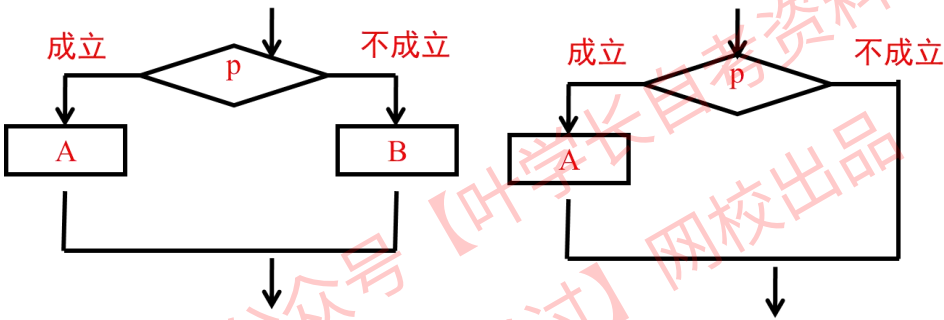
n=1
input max
while n < 10 do
    input a
    if a > max then max=a
    n=n+1
end do
print max
    
```

3.2 程序的三种基本结构

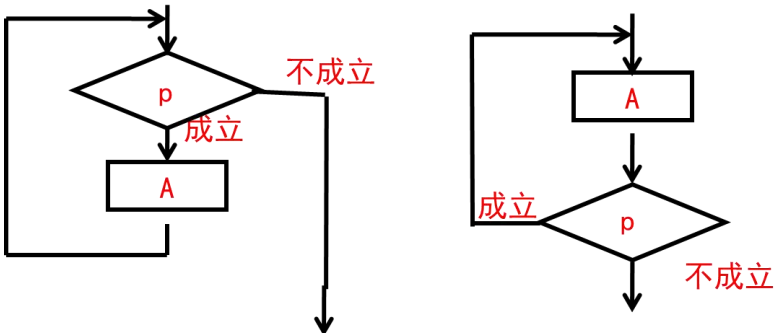
- 顺序结构。各操作步骤是顺序执行的
- 顺序结构是最简单的一种基本结构



- 选择结构又称判断结构或分支结构
- 根据是否满足给定的条件而从两组操作中选择一种操作

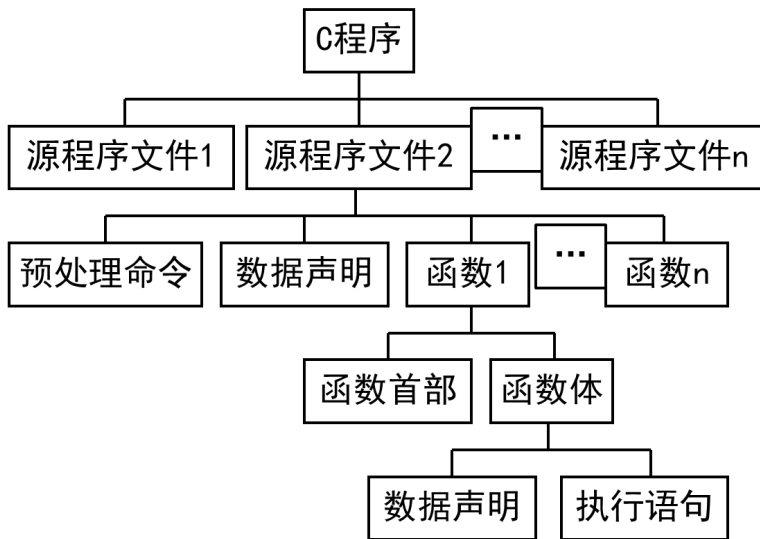


- 循环结构。它又称重复结构
- 在一定条件下反复执行某一部分的操作



- 一个良好的程序，无论多么复杂，都可以由这3种基本结构组成
- 用这三种基本结构构成的程序称为“结构化程序”

3.3 C 语句综述



➤ C 语句分为以下 5 类：

(1) 控制语句：if、switch、for、while、do...while、continue、break、return、goto 等

(2) 函数调用语句

```
printf ( "%d" , a ) ;
```

(3) 表达式语句

```
a=3;   i++;   i=a+b;
```

(4) 空语句

```
;
```

(5) 复合语句

```
{  
z=x+y;  
t=z/100;  
printf ( "%f" , t ) ;  
}
```

讲解归纳与举例

1. C 语句的五种类型？

【正确答案：C 语句分为以下 5 类：

- （1）控制语句：if、switch、for、while、do...while、continue、break、return、goto 等
- （2）函数调用语句
- （3）表达式语句
- （4）空语句
- （5）复合语句】

【答案解析】：基本概念

公众号【叶学长自考资料网】
【优课必过】网校出品

3.4 赋值表达式和赋值语句

3.4.1 赋值表达式

➤ 赋值运算符

“=” 是赋值运算符

作用是将一个数据赋给一个变量

也可以将一个表达式的值赋给一个变量

$a=3;$

$a=b+3;$

➤ 复合的赋值运算符

在赋值符“=”之前加上其他运算符，可以构成复合的运算符

$a+=3$ 等价于 $a=a+3$

$-=$, $*=$, $/=$, $\%=$ 同理。

➤ 赋值表达式

◆ 一般形式为：

变量赋值运算符表达式

◆ 对赋值表达式求解的过程：

求赋值运算符右侧的“表达式”的值

赋给赋值运算符左侧的变量

➤ 赋值表达式

赋值表达式“ $a=3*5$ ”的值为15，对表达式求解后，变量a的值和表达式的值都是15

$a=(b=5)$ ”和“ $a=b=5$ ”等价

$a=b$ ”和“ $b=a$ ”含义不同

分析

$a=(b=4)+(c=6)$ //正确

$(a=3*5)=3*4$ //正确

$a=3*5=3*4$ //错误

3.4.2 赋值过程中的类型转换

- 两侧类型一致时，直接赋值
- 两侧类型不一致，但都是数值型或字符型，自动将右侧的类型转换为左侧类型后赋值

```
int i;
```

```
i=3.56; //结果是使 i 的值为 3
```

```
float f;
```

```
f=23; //结果是使 f 的值为 23.00000
```

- 要防止数据溢出

```
Float f; double d=123.456789e100;
```

```
f=d; //数据溢出，错误
```

- 将一个占字节多的数据赋值给一个占字节数少的的变量，会发生截断，即将低字节送到该变量。

```
int i = 289;
```

```
char c ;
```

```
c = i;
```

最后 c 的值是 33，用%c 输出是！

3.4.3 赋值语句

- 赋值语句是由赋值表达式加上一个分号构成

```
a=3*m;
```

- 赋值表达式的末尾没有分号，而赋值语句有分号

```
if ( (a=b) >0) t=0;
```

- 一个表达式可以包含赋值表达式，但决不能包含赋值语句

```
if ( (a=b) ; >0) t=0; //错误
```

3.4.4 变量赋初值

- 程序中常需要对一些变量预先设置一个初值
- 设置初值既可以用赋值语句去实现，也可以在定义变量的同时使变量初始化，后者更为方便

```
int a, b, c=5;
```

相当于：

```
int a, b, c;
```

```
c=5;
```

讲解归纳与举例

1.有复合赋值语句 $x*=y+8$ ，与下面语句等价的是（ ）。

A $x=x*(y+8)$ B $x=x*y+8$

C $y=x*(y+8)$ D $y=y*(x+8)$

【正确答案：A】

【答案解析】：复合赋值表达式的理解

2.若 a 为 int 类型，且其值为 3，则执行完表达式 $a+=a-=a*a$ 后，a 的值是（ ）。

A -3 B 9

C -12 D 6

【正确答案：C】

【答案解析】：赋值运算符是自右向左结合。

3.5 数据输入输出的概念

- 几乎每一个 C 程序都包含输入输出
- 输入输出是程序中最基本的操作之一
- 所谓输入输出是以计算机主机为主体而言的
- 从计算机向输出设备输出数据称为输出，从输入设备向计算机输入数据称为输入
- C 语言本身不提供输入输出语句

输入和输出操作是由 C 函数库中的函数来实现的。

printf、scanf、putchar、getchar、puts、gets 不是关键字，而是库函数的名字。

- 在使用系统库函数时，要在程序中使用预编译命令“**#include**”。

#include <stdio.h> //标准方式

或者

#include "stdio.h"

3.6 字符数据的输入输出

3.6.1 用 putchar 函数输出一个字符

- 从计算机向显示器输出一个字符
- putchar 函数的一般形式为：

putchar (c)

例 3.1 先后输出几个字符。

```
#include <stdio.h>
```

```
void main ( )
```

```
{
```

```
    char a, b, c;
```

```
    a='B', b='O', c='Y';
```

```
    putchar (a) ;
```

```
    putchar (b) ;
```

```
    putchar (c) ;
```

```
    putchar ('\n') ;
```

BOY

```
}
```

```
#include <stdio.h>
```

```
void main ( )
```

```
{
```

改为int a=66, b=79, c=89;

```
    char a, b, c;
```

```
    a='B', b='O', c='Y';
```

```
    putchar (a) ;
```

```
    putchar (b) ;
```

```
    putchar (c) ;
```

```
    putchar ('\n') ;
```

BOY

Putchar ('\101') (输出字符 A)

putchar (' \' ') (输出单撇号字符')

3.6.2 用 getchar 函数输入一个字符

- 向计算机输入一个字符
- getchar 函数的一般形式为：

getchar ()

例 3.2 输入多个字符。

```
#include <stdio.h>
```

```
void main ( )
```

```
{ char a, b, c;
```

```
    a=getchar ( ) ;    输入一个字符，送给变量a
```

```
    b=getchar ( ) ;
```

```
    c=getchar ( ) ;
```

```
    putchar (a) ;    putchar (b) ;    putchar (c) ;
```

```
    putchar ('\n') ;
```

BOY✓ B✓ B
 O✓ O

BOY

```
}
```

```
#include <stdio.h>
```

```
void main ( )
```

```
{ char a, b, c ;
```

```
    a=getchar ( ) ;
```

```
    b=getchar ( ) ;    putchar (getchar ( ) ) ;
```

```
    c=getchar ( ) ;
```

```
    putchar (a) ;    putchar (b) ;    putchar (c) ;
```

```
    putchar ('\n') ;
```

```
}
```

```
#include <stdio.h>
```

```
void main ( )
```

```
{
```

```
    putchar (getchar ( ) ) ;
```

```
    putchar (getchar ( ) ) ;
```

```
putchar (getchar ( ) ) ;  
    putchar ( '\n' ) ;  
}  
BOY✓  
BOY
```

讲解 归纳与举例

1.从键盘输入一个小写字母，转换为大写字母后输出。

要求有文字说明：

Please enter the character:

The Changed Result is:

【正确答案：见下面】

【答案解析】：字符数据的输入输出

```
#include <stdio.h>  
  
void main ( )  
{  
    char ch;  
  
    printf ( "Please enter the character: " ) ;  
    ch=getchar ( )      //scanf ( "%c", &ch ) ;  
  
    ch=ch-32;  
  
    printf ( "The Changed Result is: %c\n", ch ) ;  
}
```

3.7 简单的格式输入与输出

3.7.1 用简单的 printf 函数输出数据

- 在 C 程序中用来实现输出和输入的，主要是 **printf** 函数和 **scanf** 函数
- 这两个函数是格式输入输出函数
- 用这两个函数时，必须指定格式

1.printf 函数的一般格式

printf (格式控制，输出表列)

格式控制：格式声明和普通字符

例如：

```
printf ( "i=%d, c=%c\n" , i, c ) ;
```

格式声明

例如：

```
printf ( "i=%d, c=%c\n" , i, c ) ;
```

普通字符

1.printf 函数的一般格式

printf (格式控制，输出表列)

格式控制：格式声明和普通字符

例如：

```
printf ( "i=%d, c=%c\n" , i, c ) ;
```

输出列表：可以是常量、变量或表达式

2. 基本的格式字符

- ◆ d 格式符。按十进制整型数据的实际长度输出
- ◆ i 格式符。作用与 d 格式符相同
- ◆ c 格式符。用来输出一个字符

```
char ch= 'a' ;
```

```
printf ( "%c" , ch ) ;
```

◆ s 格式符。用来输出一个字符串

```
printf ( "%s" , "CHINA" ) ;
```

例 3.3 用 printf 函数输出字符数据。

```
#include <stdio.h>

void main ( )

{ char c='a';

  int i=97;

  printf ("c=%c, c=%d\n", c, c) ;

  printf ("i=%c, i=%d\n", i, i) ;

}
```

c=a, c=97

i=a, i=97

◆ f 格式符。用来输出实数，以小数形式输出

例 3.4 输出实数时的有效位数。

```
#include <stdio.h>

void main ( )

{ float  a, b;

  a=111111.111; b=222222.222;

  printf ("%f\n", a+b) ;

}
```

333333.328125

例 3.5 输出双精度数时的有效位数。

```
#include <stdio.h>

void main ( )

{ double  a, b;

  a=11111111.11111111;

  b=22222222.22222222;

  printf ("%f\n", a+b) ;
```



```
}
```

33333333.33

◆ f 格式符。用来输出实数，以小数形式输出

float 型数据只能保证 6~7 位有效数字

double 型数据能保证 15~16 位有效数字

计算机输出的数字不都是绝对精确有效的

◆ e 格式符。指定以指数形式输出实数

VC++给出小数位数为 6 位

指数部分占 5 列

小数点前必须有而且只有 1 位非零数字

```
printf ( "%e" , 123.456 ) ;
```

输出：1.234560 e+002

3.7.2 用简单的 scanf 函数输入数据

1. scanf函数的一般形式

```
scanf ( 格式控制, 地址表列)
```

含义同printf函数,

1. scanf函数的一般形式

```
scanf ( 格式控制, 地址表列)
```

可以是变量的地址 (&取地址符号), 或字符串的首地址

例 3.6 用 scanf 函数输入数据。

```
#include <stdio.h>
```

```
void main ( )
```

```
{ int a, b, c;
```

```
scanf ( "%d%d%d" , &a, &b, &c ) ;
```

```
printf ( "a=%d, b=%d, c=%d\n", a, b, c ) ;
```

}

3 4 5 ✓
a=3, b=4, c=5

2. scanf 函数中的格式声明

➤ 与 printf 函数中的格式声明相似

➤ 以 % 开始，以一个格式字符结束，中间可以插入附加的字符

scanf ("a=%db=%dc=%d", &a, &b, &c);

➤ 说明

scanf ("%d, %d", a, b); 错

scanf ("%d, %d", &a, &b); 对

对于

scanf ("a=%d, b=%d, c=%d", &a, &b, &c);

12 24 36 ✓ 错

a=12, b=24, c=36 ✓ 对

a=12 b=24 c=36 ✓ 错

➤ 说明

对于 scanf ("%c%c%c", &c1, &c2, &c3);

abc ✓ 对

a b c ✓ 错

对于 scanf ("%d%c%f", &a, &b, &c);

若输入以下数据结果如何？

1234a123o.26 ✓

讲解归纳与举例

1. 若变量已正确说明为 float 类型，要通过语句 scanf ("%f%f%f", &a, &b, &c); 给 a 赋于 10.0, b 赋于 22.0, c 赋于 33.0, 不正确的输入形式是 ()。

A 10<回车> 22<回车> 33<回车>

B 10.0, 22.0, 33.0<回车>

C 10.0<回车> 22.0 33.0<回车>

D 10 22<回车> 33<回车>

【正确答案：B】

【答案解析】：scanf ("%f%f%f", &a, &b, &c) 输入

语句的格式里面没有逗号，因此 B 不对。

2.有定义语句：int x, y; 若 x 值 11，变量 y 得到数值 12，下面四组输入要通过 scanf ("%d, %d", &x, &y)；语句使变量 x 得到数形式中，错误的是（ ）。

A 11 12<回车>

B 11, 12<回车>

C 11, 12<回车>

D 11, <回车> 12<回车>

【正确答案：A】

【答案解析】：输入输出语句基本格式。

3.8 顺序结构程序设计举例

例 3.7 输入三角形的三边长，求三角形面积。

解题思路：假设给定的三个边符合构成三角形的条件。找到求三角形面积的公式：

其中 $s = (a+b+c) / 2$

$$\text{area} = \sqrt{s(s-a)(s-b)(s-c)}$$

```
#include <stdio.h>
```

```
#include <math.h>
```

```
void main ( )
```

```
{ double a, b, c, s, area;
```

```
scanf ("%lf, %lf, %lf", &a, &b, &c); s = (a+b+c) / 2.0;
```

```
area=sqrt (s* (s-a) * (s-b) * (s-c) );
```

```
printf ("a=%f\nb=%f\nc=%f\n", a, b, c);
```

```
printf ("area=%f\n", area);
```

```
}
```

```
9.89, 12.65, 8.76↵  
a=9.890000  
b=12.65000  
c=8.760000  
area=43.165683
```

例 3.8 从键盘输入一个大写字母，要求改用小写字母输出。

➤ 解题思路：

关键是找到大、小写字母间的内在联系

小写字母的 ASCII 码=大写字母的 ASCII 码+32

```
#include <stdio.h>
```

```
void main ( )
```

```
{
```

```
char c1, c2;
```

```
c1=getchar ();
```

```
printf ("%c, %d\n", c1, c1);
```

```
c2=c1+32;
```

```
printf ("%c, %d\n", c2, c2);
```

```
}
```

```
A↵  
A, 65  
a, 97
```

输入 3 个大写字母，输出相应的 3 个小写字母。

```
#include <stdio.h>

void main ( )

{ printf ( "%c" , getchar ( ) +32 ) ;

  printf ( "%c" , getchar ( ) +32 ) ;

  printf ("%c\n", getchar ( ) +32 ) ;

}
```

```
#include <stdio.h>

void main ( )

{ putchar ( getchar ( ) +32 ) ;

  putchar ( getchar ( ) +32 ) ;

  putchar ( getchar ( ) +32 ) ;

  putchar ( '\n' ) ;

}
```

例 3.9 求 $ax^2+bx+c=0$ 方程的根。a、b、c 由键盘输入。设 $b^2-4ac > 0$ 。

```
#include <stdio.h>

#include <math.h>

void main ( )

{double a, b, c, disc, x1, x2, p, q;

  scanf ( "a=%lf, b=%lf, c=%lf", &a, &b, &c ) ;

  disc=b*b-4*a*c;

  p=-b/ ( 2*a ) ;

  q=sqrt ( disc ) / ( 2*a ) ;

  x1=p+q;    x2=p-q;

  printf ( "x1=%5.2f\nx2=%5.2f\n", x1, x2 ) ; }
```

a=1, b=3, c=2
X1=-1. 00
X2=-2. 00

3.9 提高部分

3.9.1 关于无符号数据与有符号数据之间的赋值

➤ 将有符号整数赋值给长度相同的无符号整型变量：

按字节原样赋值（连原有的符号位也作为数值一起传送）

例3.10 有符号整数赋值给无符号整型变量，数据会失真。

```
#include <stdio.h>
```

```
void main ( )
```

```
{ unsigned a;
```

```
  int b;
```

```
  b=-1;
```

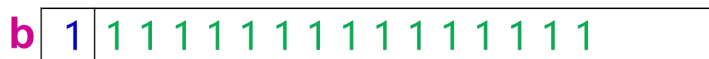
```
  a=b;
```

```
  printf ("%u\n", a) ;
```

65535

```
}
```

输出无符号整数



➤ 将无符号整数赋值给长度相同的有符号整型变量：

不要超出有符号整型变量的数值范围，否则会出错。

例 3.11 无符号整数赋值给有符号整型变量，注意数值范围。

```
#include <stdio.h>
```

```
void main ( )
```

```
{ unsigned a;
```

```
  int b;
```

```
  a=65535;
```

```
  b=a;
```

```
  printf ("%d\n", b) ;
```

```
}
```



-1

补码形式

3.9.2 较复杂的输入输出格式控制

1. 输出数据时的格式控制

- ◆ `%md`。用来指定输出数据的宽度，`m` 是指定的宽度，如果数据实际的位数小于 `m`，则左端补以空格，若大于 `m`，按实际位数输出。

```
printf ("%4d, %4d", 123, 12345);
```

□123, 12345

- ◆ `%ld`。对于 `int` 型数据占 2 字节的系统，在输出长整型数据时要在格式字符 `d` 前面加一个英文字母 `l`

```
long a=135790;
```

```
printf ("%ld", a);
```

```
printf ("%8ld", a);
```

135790

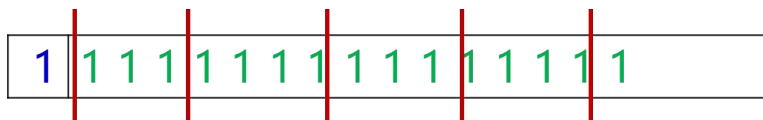
□□135790

- ◆ `%o`。以八进制整数形式输出。由于是将内存单元中的各位的值（0 或 1）按八进制形式输出，因此输出的数值不带符号，即将符号位也一起作为八进制数的一部分输出

```
int a=-1;
```

```
printf ("%d, %o", a, a);
```

-1, 177777



- ◆ `%x`。以十六进制数形式输出整数。同样不会出现负的十六进制数

```
int a=-1;
```

```
printf ("%d, %x", a, a);
```

-1, ffff



- ◆ `%u`。用来输出无符号型数据，以十进制整数形式输出
- ◆ 一个有符号整数也可用 `%u` 格式输出；反之，一个无符号型数据也可用 `%d` 格式输出
- ◆ 按相互赋值的规则处理
- ◆ 无符号型数据也可用 `%o` 或 `%x` 格式输出

例 3.12 无符号数据的输出。

```
#include <stdio.h>

void main ( )
{
    unsigned a=65535;

    int b=-2;

    printf ("a=%d, %o, %x, %u\n", a, a, a, a) ;

    printf ("b=%d, %o, %x, %u\n", b, b, b, b) ;

}
```

在TC2.0下（占2字节）的运行结果为：
a=-1, 177777, ffff, 65535
b=-2, 177776, fffe, 65534
在VC++6.0下（占4字节）的运行结果为：
a=65535, 177777, ffff, 65535
b=-2, 3777777776, ffffffff, 4294967294

➤ %mc。用来指定输出字符数据的宽度

```
char c= 'a' ;

printf ( "%3c" , c) ;
```

a

➤ %ms。指定输出的字符串占m列

若串长大于m，则将串全部输出

若串长小于m，则左补空格

➤ %-ms。如果串长小于m，则在m列范围内，字符串向左靠，右补空格。

例 3.13 字符串的输出。

```
#include <stdio.h>

void main ( )
{
    printf ("%3s, %7.2s, %.4s, %-5.3s\n",
        "CHINA", "CHINA", "CHINA", "CHINA") ;

}
```

CHINA, CHINA, CHINA, CHINA

➤ %m.nf。指定输出的实数共占m列，其中有n位小数

如果数值长度小于m，则左端补空格

➤ %-m.nf与%m.nf基本相同，只是使输出的数值向左端靠，右端补空格

例 3.14 输出实数时指定输出两位小数。

```
#include <stdio.h>

void main ( )

{ float f=123.456;

    printf ("%f    %10fn" , f, f) ;

    printf ( " %10.2f    %.2fn" , f, f) ;

    printf ( " %-10.2fn", f) ;

}
```

123.456001_ _123.456001

_ _ _ _123.46 _ _123.46

123.46_ _ _ _

例 3.15 求 3 个圆的周长，输出结果时上下按小数点对齐，取两位小数。

```
#include <stdio.h>

#define PI 3.1415926

void main ( )

{ double  r1=1.53, r2=21.83, r3=123.71,  s1, s2, s3;

    s1=2.0*PI*r1;    s2=2.0*PI*r2;    s3=2.0*PI*r3;

    printf ("r1=%10.2fnr2=%10.2fnr3=%10.2fn " ,  r1, r2, r3) ;

}
```

r1=_ _ _ _ _1.53

r2=_ _ _ _ _21.83

r3=_ _ _ _ 123.71

- %m.ne 和 %-m.ne。m、n 和 “-” 字符的含义与前相同。此处 n 指拟输出的数据的小数部分（又称尾数）的小数位数
- %g。用来输出实数，它根据数值的大小，自动选 f 格式或 e 格式（选择输出时占宽度较小的一种），且不输出无意义的零

2.输入数据格式控制

- ◆ 与 printf 函数中的格式声明相似

- ◆ 以%开始，以一个格式字符结束，中间可以插入附加的字符
- ◆ 可以指定输入数据所占的列数，系统自动按它截取所需数据
- ◆ 输入数据时不能规定精度

```
scanf ( “%3d%3d” , &a, &b) ;
```

输入：123456

系统自动将 123 赋给 a，将 456 赋给 b。

讲解归纳与举例

1. 以下程序运行后的输出结果是（ ）。

```
main ( )  
{ int a; char c=10;  
  
float f=100.0; double x;  
  
a=f/=c*=(x=6.5);  
  
printf ( “%d %d %3.1f%3.1f ” , a, c, f, x) ; }
```

A 1 65 1 6.5

B 1 65 1.5 6.5

C 1 65 1.0 6.5

D 2 65 1.5 6.5

【正确答案：B】

【答案解析】：P93 页 %3.1f 表示数据位数是 3，小数位占 1 位。

2. 设 int a=1234; 执行语句 printf (“%2d” , a) ; 后的输出结果是（ ）。

A 12

B 34

C 1234

D 出错

【正确答案：C】

【答案解析】：宽度说明是非负的十进制整数，它规定输出占位的最小宽度。但输出大于宽度时，按实际值的输出，小的宽度不会引起输出值的截断。

3.9.3 简单易用的 C++的输入输出

➤ C 的输入输出格式繁多。C++对此作了改进，用简单易用的 cin 进行输入，用 cout 进行输出。

```
printf ( “a=%f\n” , a ) ;
```

在 C++中:

```
cout<< “a=” <<a<< “\n” ;
```

```
scanf ( “%d %f” , &a, &b ) ;
```

在 C++:

```
cin>>a>>b;
```

本章小结

- (1) 算法、数据结构、程序的基本概念
- (2) 算法的表示方法：自然语言、流程图、伪代码、NS 图等
- (3) 三种程序设计结构：顺序、选择、循环
- (4) C 语言中的语句、表达式、赋值语句
- (5) 输入输出语句

格式控制：“ ”,如 “a=%d

格式声明：%和格式字符，如%d,%c

格式字符：d, c, f, e, g

附加格式字符：%3d, %7.2f, %-7.2f

- (6) 学会编写简单的程序

第4章 选择结构程序设计

本章重难点分析

选择结构的作用是：检查人们指定的条件是否满足，决定在事先给定的两种操作中选定执行其中一种操作。要实现选择结构，关键是判定所给的条件是否满足。本章主要介绍条件判断、if 和 switch 语句实现选择结构程序设计，条件运算符。

本章内容注意以下几点：

条件判断

if 语句实现选择结构程序设计 重点&难点

switch 语句实现选择结构程序设计 重点&难点

条件运算符

本章重难点分析

4.1 条件判断

4.2 用 if 语句实现选择结构

4.3 利用 switch 语句实现多分支选择结构

4.4 程序综合举例

4.5 提高部分

4.1 条件判断

4.1.1 条件判断的含义

在现实生活中，需要进行判断和选择的情况是很多的

- 如果你在家，我去拜访你
- 如果考试不及格，要补考
- 如果遇到红灯，要停车等待
- 周末我们去郊游
- 70 岁以上的老年人，入公园免票

在现实生活中，需要进行判断和选择的情况是很多的

条件判断的结果是一个逻辑值：“是”或“否”

在计算机语言中用“真”和“假”来表示“是”或“否”

用选择结构检查所指定的条件是否满足，并根据判断的结果决定执行哪种操作。

如：

```
if (x > 0)
```

```
    printf(“%d”, x);
```

```
else
```

```
    printf(“%d”, -x);
```

当 x 为正时，输出 x 的值，否则输出 -x。

4.1.2 关系运算符和关系表达式

“条件”在程序中用一个表达式表示

$x > 0$

$\text{age} \geq 70$

$a + b > c$

$b * b - 4 * a * c > 0$

$'a' < 'v'$

>, >=, <, <=, ==, != 关系运算符

整个表达式称为关系表达式

1.关系运算符及其优先次序

C 语言提供 6 种关系运算符：

① $<$ (小于) ② $<=$ (小于或等于)

③ $>$ (大于) ④ $>=$ (大于或等于)

优先级相同 (高)

⑤ $==$ (等于) ⑥ $!=$ (不等于)

优先级相同 (低)

关系、算术、赋值运算符的优先级

算术运算符 (高)

关系运算符

赋值运算符 (低)



$c>a+b$ 等效于 $c>(a+b)$

$a>b==c$ 等效于 $(a>b)==c$

$a==b<c$ 等效于 $a==(b<c)$

$a=b>c$ 等效于 $a=(b>c)$

2.关系表达式

用关系运算符将两个数值或数值表达式连接起来的式子

关系表达式的值是一个逻辑值，即“真”或“假”

在 C 的逻辑运算中，以“1”代表“真”，以“0”代表“假”

2.关系表达式

当 $a=3$, $b=2$, $c=1$ 时

$a>b$ 的值为“真”，表达式值为 1

$(a>b)==c$ 的值为“真”，表达式值为 1

$b+c<a$ 的值为“假”，表达式值为 0

4.1.3 逻辑运算符和逻辑表达式

有时判断的条件是一个复合的条件：

- 是中国公民，且在 18 岁以上
- 5 门课都及格
- 70 岁以上的老人或 10 岁以下儿童

1.逻辑运算符及其优先次序

- 3 种逻辑运算符：

&&（逻辑与） ||（逻辑或） !（逻辑非）

- &&和||是双目（元）运算符
- !是一目（元）运算符

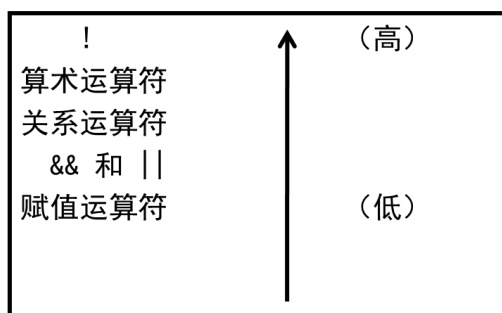
逻辑运算的真值表

a	b	! a	! b	a && b	a b
真	真	假	假	真	真
真	假	假	真	假	真
假	真	真	假	假	真
假	假	真	真	假	假

逻辑运算符的优先次序

! → && → || （!为三者中最高）

与其他运算符的优先次序



$(a > b) \&\& (x > y)$ 可写成 $a > b \&\& x > y$

$(a == b) || (x == y)$ 可写成 $a == b || x == y$

$(!a) || (a > b)$ 可写成 $!a || a > b$

2.逻辑表达式

用逻辑运算符将关系表达式或其他逻辑量连接起来的式子

➤ 逻辑表达式的值应该是逻辑量“真”或“假”

➤ 编译系统在表示逻辑运算结果时

以数值 1 代表“真”，以 0 代表“假”

➤ 但在判断一个量是否为“真”时

以 0 代表“假”，以非 0 代表“真”

➤ 注意：将一个非零的数值认作为“真”

(1) 若 $a=4$ ，则 $!a$ 的值为 0

(2) 若 $a=4$ ， $b=5$ ，则 $a \&\& b$ 的值为 1

(3) a 和 b 值分别为 4 和 5，则 $a||b$ 的值为 1

(4) a 和 b 值分别为 4 和 5，则 $!a||b$ 的值为 1

(5) $4 \&\& 0 || 2$ 的值为 1

修改后的逻辑运算真值表

a	b	$!a$	$!b$	$a \&\& b$	$a b$
非0	非0	0	0	1	1
非0	0	0	1	0	1
0	非0	1	0	0	1
0	0	1	1	0	0

➤ 判别某一年是否闰年，用逻辑表达式表示

➤ 闰年的条件是符合下面二者之一：

①能被 4 整除，但不能被 100 整除，如 2008

②能被 400 整除，如 2000

$(year \% 4 == 0 \&\& year \% 100 != 0)$

$|| year \% 400 == 0$

如果表达式值为 1，则闰年；否则为非闰年

讲解归纳与举例

1.已知 char ch;判断 ch 为大写字母的表达式为（ ）。

A.ch>= 'A' && ch<= 'Z'

B.ch>= 'A' ||ch<= 'Z'

C.ch>= 'A' and ch<= 'Z'

D.ch>= 'A' or ch<= 'Z'

【正确答案：A】

【答案解析】：条件判断

2.设 a, b, c, d, m, n 均为 int 型变量, 且 a=5, b=6, c=7, d=8, m=2, n=2, 则逻辑表达式 (m=a>b)&&(n=c>d) 运算后, n 的值为（ ）。

A.0 B.1

C.2 D.3

【正确答案：C】

【答案解析】：&&是阻塞式运算符，当左边(m=a>b)为假时，右边的(n=c>d)不执行。

3.设 int a=1, b=2, c=3, d;

执行 d=! (a+b+c) 后, d 的结果是

【正确答案：0】

【答案解析】：P106。a+b+c=6, 6 非 0 即为逻辑真,! (a+b+c) 为逻辑假 (0)。

4.以下程序输出的结果是_____。

main()

```
{    int a=5,b=4,c=3,d;
```

```
    d=(a>b>c);
```

```
    printf("%d\n",d);
```

```
}
```

【正确答案：0】

【答案解析】：a>b>c, 先执行 a>b 结果为逻辑真 1, 然后 1>c 即 1>3 为逻辑假 0。

4.2 用 if 语句实现选择结构

4.2.1 用 if 语句实现选择结构举例

例 4.1 输入两个学生 a 和 b 的成绩，输出其中高的成绩。

➤ 解题思路：

设定变量 max，将高的成绩放在其中。

如果 a 大于或等于 b，把 a 的值赋给 max

如果 b 大于 a，把 b 的值赋给 max

输出 max。

```
#include <stdio.h>
```

```
void main()
```

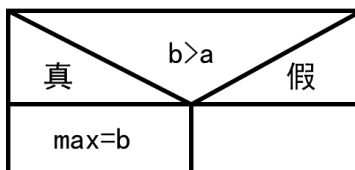
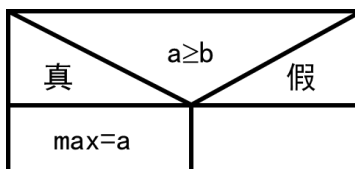
```
{
    float a,b,max;

    printf("please enter a and b:");
    scanf("%f,%f",&a,&b);

    if(a>=b) max=a;

    if(b>a) max=b;

    printf("max=%6.2f\n",max);
}
```



please enter a and b: 67.5, 95.5 ✓

max= 95.50

```
#include <stdio.h>
```

```
void main()
```

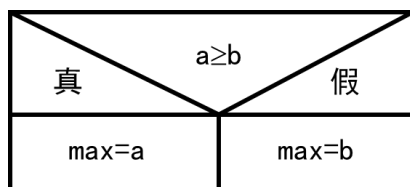
```
{
    float a,b,max;

    printf("please enter a and b:");
    scanf("%f,%f",&a,&b);

    if(a>=b) max=a;

    else max=b;

    printf("max=%6.2f\n",max);
}
```



please enter a and b: 67.5, 95.5 ✓

max= 95.50

例 4.2 输入 3 个成绩 a, b, c, 要求按由高到低的顺序输出。

➤ 解题思路：可以先用伪代码写出算法：

if a<b, a 和 b 对换 (a 是 a、b 中的大者)

if a<c, a 和 c 对换 (a 是三者中最大者)

if b<c, b 和 c 对换 (b 是三者中次大者)

输出 a, b, c 的值

```
#include <stdio.h>
```

```
void main()
```

```
{ float a,b,c,t;
```

```
    printf("please enter a,b,c:");
```

```
    scanf("%f,%f,%f",&a,&b,&c);
```

```
    if(a<b)
```

```
    {   t=a;  a=b;  b=t;   }
```

```
    if(a<c)
```

```
    {   t=a;  a=c;  c=t;   }
```

```
    if(b<c)
```

```
    {   t=b;  b=c;  c=t;   }
```

```
    printf("%6.2f,%6.2f,%6.2f\n",a,b,c);
```

```
}
```

please enter a,b,c:73.5,82.5,99 ✓
99.00, 82.50, 73.50

实现a和b的互换

4.2.2 if 语句的一般形式

1. if (表达式) 语句

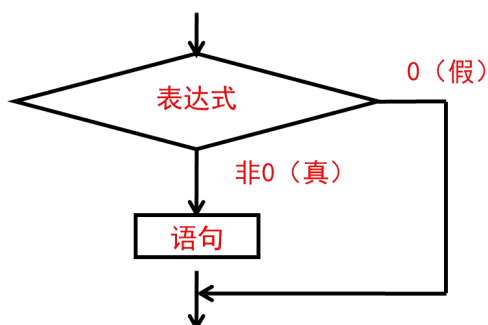
```
if(x>y) printf( "%d" ,x);
```

语句

非 0 (真)

表达式

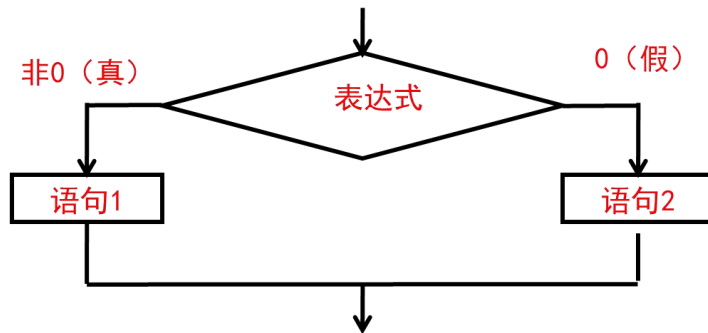
0 (假)



2. if (表达式) 语句1 else 语句2

```
if(x>y) printf( "%d" ,x);
```

```
else    printf( "%d" ,y);
```



4.2.3 if 语句的使用说明

1. if (表达式) 语句

2. if (表达式) 语句1 else 语句2

关系表达式
逻辑表达式

```
if(a==b && x==y)
```

```
    printf( "a=b,x=y");
```

1. if (表达式) 语句

2. if (表达式) 语句1 else 语句2

数值表达式

关系表达式
逻辑表达式

```
if(3) printf("O.K.");
```

```
if(x>0) printf("%f",x);
```

```
else printf("%f",-x);
```

1. if (表达式) 语句
2. if (表达式) 语句1 else 语句2

可以是：
简单的语句
复合语句
另一个if语句等

例 4.3 给出三角形的三个边长，求三角形的面积。

➤ 解题思路：

构成三角形的必要条件是两边之和大于第三边，因此在计算三角形面积之前应当进行该条件的判断。

```
#include <stdio.h>
#include <math.h>
void main ( )
{ double a,b,c,s,area;
  printf("please enter a,b,c:");
  scanf("%lf,%lf,%lf",&a,&b,&c);
  if (a+b>c && b+c>a && c+a>b)
  { s=0.5*(a+b+c);
    area=sqrt(s*(s-a)*(s-b)*(s-c));
    printf("area=%6.2fn",area);
  }
  else printf("It is not a trilateral.\n");
}
```

please enter a,b,c:2,3,6✓
It is not a trilateral.

please enter a,b,c:2,3,4✓
area= 2.90

复合语句

4.2.4 使用嵌套的 if 语句实现多层条件判断

- 有的选择结构中又包含一个或多个选择结构，这称为**选择结构的嵌套**
- if 语句中可以又包括另一个 if 语句，这就是 if 语句的嵌套
- 可以用 if 语句的嵌套实现嵌套的选择结构

➤ 一般形式：

if ()

if () 语句1
else 语句2

else

内嵌if

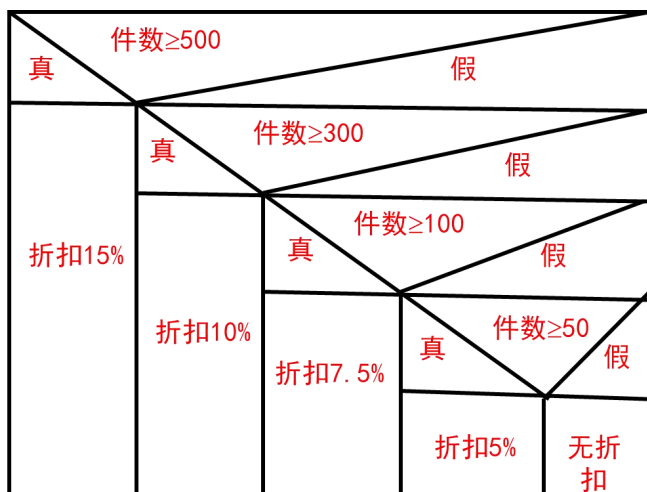
if () 语句3
else 语句4

例 4.4 为促销，对购买货物多的顾客有优惠：凡买 50 件以上的优惠 5%，买 100 件以上的优惠 7.5%，买 300 件以上的优惠 10%，买 500 件以上的优惠 15%。输入购买的数量和单价，输出应付货款。

➤ 解题思路：

关键是写出判断货物折扣的选择结构

应付货款=件数*单价*（1-优惠折扣）



Void main ()

```
{ int number;
```

```
double cost,price,total;
```

```
printf( "please enter number and price:");
```

```
scanf("%d,%lf",&number,&price);
```

```
if (number>=500) cost=0.15;
```

else总是与它上面最近的未配对的if配对

```
else if (number>=300) cost=0.10;
```

```
else if (number>=100) cost=0.075;
```

```
else if (number>=50) cost=0.05;
```

```
else cost=0;
```

enter number and price:512,821.5 ✓
 Total= 357516.80

```
total=number*price*(1-cost);  
printf(“Total=%10.2f\n”,total);  
}
```

else 总是与它上面最近的未配对的 if 配对

enter number and price:512,821.5 ✓

Total= 357516.80

讲解归纳与举例

1. 以下是 if 语句的基本形式：

if(表达式)语句

其中“表达式”（ ）。

- A. 必须是逻辑表达式
- B. 必须是关系表达式
- C. 必须是逻辑表达式或关系表达式
- D. 可以是任意合法的表达式

【正确答案：D】

【答案解析】：if 语句基本格式

2. 判断题：在 if 语句的三种形式中，如果要想在满足条件时执行一组多个语句，则必须把这一组语句用 {} 括起来组成一个复合语句。

【正确答案：对】

【答案解析】：if 语句基本格式。

3. 以下程序执行后输出结果是 。

```
main ( )  
{   int i=1,j=1,k=2;  
    if ( (j++ || k++) && i++)  
        printf ( "%d,%d,%d\n", i,j,k );    }
```

【正确答案：2,2,2】

【答案解析】：if 语句基本格式，k++ 没有执行。

4.编程题：输入一个整数，判断该数是奇数还是偶数。要求必须使用 if-else 结构实现。

【正确答案：见下一页】

【答案解析】：if 语句的应用。

```
#include <stdio.h>

int main() {
    int x;

    scanf("%d",&x);

    if(x%2==0)
        printf("偶数\n");
    else
        printf("奇数\n");
}
```


4.3 利用 switch 语句实现多分支选择结构

➤ switch 语句用来实现多分支选择结构

- 学生成绩分类

85 分以上为 'A' 等

70~84 分为 'B' 等

60~69 分为 'C' 等

.....

- 人口统计分类

按年龄分为老、中、青、少、儿童

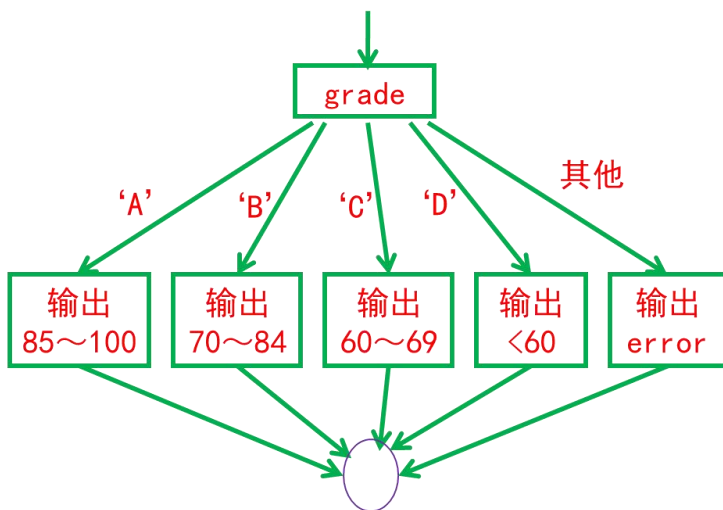
➤ switch语句的一般形式：

```
switch (表达式)
{
    case 常量表达式1 : 语句1
    case 常量表达式2 : 语句2
        :           :           : 不能相同
    case 常量表达式n : 语句n
    default          : 语句n+1
}
```

```
switch(grade)
{
    case 'A' : printf( "85~100\n" );break;
    case 'B' : printf( "70~84\n" );break;
    case 'C' : printf( "60~69\n" );break;
    case 'D' : printf( "<60\n" );break;
    default : printf( "error\n" );
}
```

若grade的值为 "A"

输出：85~100



说明：

- (1) 括号内的“表达式”，其值的类型应为整数类型(包括字符型)。
- (2) 执行 switch 语句时，先计算 switch 后面的“表达式”的值，然后将它与各 case 标号比较，如果与某一个 case 标号中的常量相同，流程就转到此 case 标号后面的语句。如果没有与 switch 表达式相匹配的 case 常量，流程转去执行 default 标号后面的语句。
- (3) 可以没有 default 标号，此时如果没有与 switch 表达式相匹配的 case 常量，则不执行任何语句。
- (4) 各个 case 标号出现次序不影响执行结果。
- (5) 每一个 case 常量必须互不相同；否则就会出现互相矛盾的现象。
- (6) case 标号只起标记的作用。在执行 switch 语句时，根据 switch 表达式的值找到匹配的入口标号，在执行完一个 case 标号后面的语句后，从此标号开始执行下去，不再进行判断。因此，一般情况下，在执行一个 case 子句后，应当用 break 语句使流程跳出 switch 结构。最后一个 case 子句（今为 default 子句）中可不加 break 语句。
- (7) 在 case 子句中虽然包含了一个以上执行语句，但可以不必用花括号括起来，会自动顺序执行本 case 标号后面所有的语句。当然加上花括号也可以。
- (8) 多个 case 标号可以共用一组执行语句。

讲解归纳与举例

1.判断题：case 语句后如没有 break，会继续向下执行。

【正确答案：对】

【答案解析】：switch 语句的理解。

2.执行以下程序段后，i 的值是 。

```
int i=10;

switch(i+1)

{   case 10: i++; break;

      case 11: ++i;

      case 12: ++i; break;

      default : i=i+1; }
```

【正确答案：12】

【答案解析】：case 11: ++i; 后面没有 break 语句。所以会无条件执行下一个 case 后面的语句，直到遇到 break 结束。

3.编程题：输入某学生的成绩，输出该学生的成绩和等级。（A 级：90-100，B 级：80-89，C 级：60-79，D 级：0-59）（考虑非法输入）。

【正确答案：见下面】

【答案解析】：分支语句的基本应用。

参考程序一：

```
if(score>=90 &&score<=100)

    printf("the level of %d is:A\n",score);

else if(score>=80 &&score<90)

    printf("the level of %d is:B\n",score);

else if(score>=70 &&score<80)

    printf("the level of %d is:C\n",score);

else if(score>=60 &&score<70)

    printf("the level of %d is:D\n",score);

else if(score>=0 &&score<60)

    printf("the level of %d is:E\n",score);

else

    printf("Error!\n");
```

参考程序二：

```
s=score/10;

if(score==100)s=9;
```

```
switch(s){  
    case 1:case 2:case 3:case 4:case 5:  
printf("the level of %d is:E\n",score);break;  
    case 6:  
printf("the level of %d is:D\n",score);break;  
    case 7:  
printf("the level of %d is:C\n",score);break;  
    case 8:  
printf("the level of %d is:B\n",score);break;  
    case 9:  
printf("the level of %d is:A\n",score);break;  
    default:  
printf("Error!\n");  
}
```

4.4 程序综合举例

例 4.5 写程序，判断某一年是否为闰年。

用变量 leap 代表是否闰年的信息

若闰年，令 leap=1；非闰年，leap=0

最后判断 leap 是否为 1（真），若是，则输出“闰年”信息

参见教材图 4.11

```
#include <stdio.h>
```

```
void main()
```

```
{int year,leap;
```

```
    printf("enter year:"); scanf("%d",&year);
```

```
    if (year%4==0)
```

```
        if(year%100==0)
```

```
            if(year%400==0) leap=1;
```

```
            else leap=0;
```

```
        else leap=1;
```

```
    else leap=0;
```

```
    if (leap) printf("%d is ",year);
```

```
    else printf("%d is not ",year);
```

```
    printf("a leap year.\n");
```

```
}
```

```
if((year%4==0 && year%100!=0)
```

```
    || (year%400==0))
```

```
    leap=1;
```

```
else leap=0;
```

例 4.6 运输公司对用户计算运费。运输距离越远，单位运费越低。

➤ 标准如下：

$s < 250$	没有折扣
-----------	------

$250 \leq s < 500$	2%折扣
--------------------	------

$500 \leq s < 1000$	5%折扣
---------------------	------

$1000 \leq s < 2000$	8%折扣
$2000 \leq s < 3000$	10%折扣
$3000 \leq s$	15%折扣

➤ 设每吨每千米货物的基本运费为 p ，货物重为 w ，距离为 s ，折扣为 d

➤ 总运费 f 的计算公式为 $f = p \times w \times s \times (1 - d)$

➤ 折扣的变化规律（参见教材图 4.12）：

折扣的“变化点”都是 250 的倍数

在横轴上加一种坐标 c ， c 的值为 $s/250$

c 代表 250 的倍数

当 $c < 1$ 时，表示 $s < 250$ ，无折扣

$1 \leq c < 2$ 时，表示 $250 \leq s < 500$ ，折扣 $d = 2\%$

$2 \leq c < 4$ 时， $d = 5\%$ ； $4 \leq c < 8$ 时， $d = 8\%$ ；

$8 \leq c < 12$ 时， $d = 10\%$ ； $c \geq 12$ 时， $d = 15\%$

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
    int c,s;
```

```
    double p,w,d,f;
```

```
    printf("请输入单价、重量和距离:");
```

```
    scanf("%lf%lf%d",&p,&w,&s);
```

```
    if(s>=3000)    c=12;
```

```
    else          c=s/250;
```

```
    switch(c)
```

```
    { case 0:    d=0; break;
```

```
      case 1:    d=2; break;
```

```
      case 2:
```

```
      case 3:    d=5; break;
```

```
      case 4:
```

```
case 5:
case 6:
case 7:    d=8; break;
case 8: case 9: case 10:
case 11:   d=10; break;
case 12:   d=15; break;
}
f=p * w * s * (1 - d / 100.0);

printf(“运费:%10.2f 元\n”,f);
}

请输入单价、重量和距离：15,145.6,346.9✓
运费：740550.72 元
```

4.5 提高部分

4.5.1 用条件表达式实现简单的选择结构

- 在 if 语句中，当被判别的表达式的值为“真”或“假” 时，都执行一个赋值语句且向同一个变量赋值。

- 如：if (a>b)

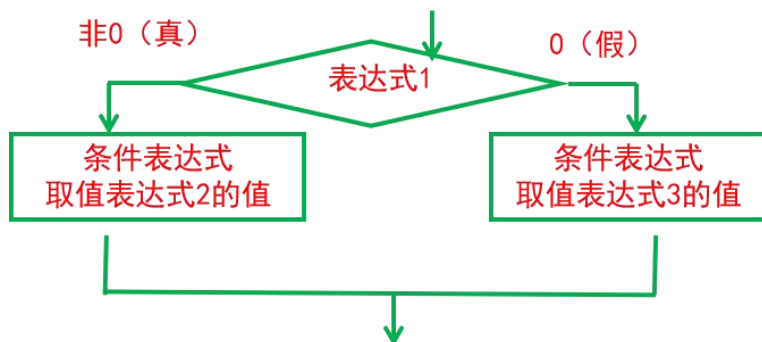
```
    max=a;  
  
else  
  
    max=b;
```

$\text{max} = (\text{a} > \text{b}) ? \text{a} : \text{b};$

- 条件运算符优先于赋值运算符
- 条件运算符的结合方向为“自右至左”
- 以下为合法的使用方法：

$\text{a} > \text{b} ? (\text{a} = 100) : (\text{b} = 100);$

$\text{a} > \text{b} ? \text{printf}(\text{"\%d"}, \text{a}) : \text{printf}(\text{"\%d"}, \text{b});$



条件表达式的一般形式为

表达式 1? 表达式 2: 表达式 3

4.5.2 在程序中使用条件表达式

例 4.7 输入一个字符，判别它是否大写字母，如果是，将它转换成小写字母；如果不是，不转换。然后输出最后得到的字符。


```
#include <stdio.h>

void main ( )
{ char ch;

  scanf("%c",&ch);

  ch=(ch>='A'&& ch<='Z')?(ch+32):ch;

  printf("%c\n",ch);
}
```

A ✓
a

讲解归纳与举例

1.以下程序运行后的输出结果是 。

```
Void main()
{ int p=30;

  printf("%d ",(p/3>0 ? p/10 : p%3));    }
```

【正确答案： 3】

【答案解析】： 条件表达式应用。

2.输入一整数，判别它是否是正数，如果是正数或者 0，原样输出；如果不是，取反再输出。

【正确答案： 见下一页】

【答案解析】： 条件表达式应用。

```
#include <stdio.h>

int main()
{
  int n;

  scanf("%d",&n);

  (n>=0) ? printf(""%d\n"",n): printf(""%d\n"",-n);

  return 0;
}
```

本章小结

- （1）掌握算术运算符、关系运算符、逻辑运算符以及算术表达式、关系表达式、逻辑表达式的概念和用法。
- （2）掌握 if 语句和 switch 语句的基本语法规则的应用。能使用分支语句解决实际问题。
- （3）条件运算符的基本应用。
- （4）学会编写选择结构程序。

第5章 循环结构程序设计

本章重难点分析

现实生活中许多问题是需要重复处理。这就需要用到循环结构来解决。本章主要介绍三种常用的循环结构、循环嵌套、提前结束循环、以及几种循环结构的比较。

本章内容注意以下几点：

三种循环结构 **重点&难点**

循环嵌套 **重点&难点**

提前结束循环 **重点**

三种循环结构的比较

5.1 程序中需要使用循环结构

- 现实生活中许多问题是需要重复处理

计算一个班 50 学生每人的平均成绩

工厂各车间的生产日报表

全国各省市的人口统计分析

各大学招生情况统计

全校教职工工资报表

- 绝大多数的应用程序都包含重复处理

- 循环结构又称为重复结构

- 循环结构和顺序结构、选择结构是结构化程序设计的**三种基本结构**，它们是各种复杂程序的基本构造单元

- 要构成一个有效的循环，应当指定两个条件：

- (1) 需要重复执行的操作，这称为**循环体**
- (2) **循环结束的条件**，即在什么情况下停止重复的操作

5.2 while 和 do while 语句实现循环

5.2.1 用 while 语句实现循环

例 5.1 求 $1+2+3+\cdots+100$ ，即 $\sum_{n=1}^{100} n$ 。

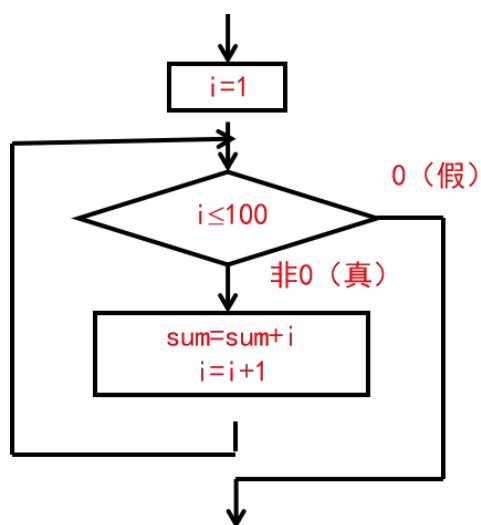
➤ 解题思路：

这是累加问题，需要先后将 100 个数相加

要重复 100 次加法运算，可用循环实现

后一个数是前一个数加 1 而得

加完上一个数 i 后，使 i 加 1 可得到下一个数



```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
    int i,sum=0;
```

```
    i=1;
```

```
    while (i<=100)
```

```
    { sum=sum+i;
```

```
      i++;
```

```
    }
```

```
    printf("%d\n",sum);
```

```
}
```

能改为 $i < 100$?

复合语句

while语句的一般形式如下：

while (表达式) 语句

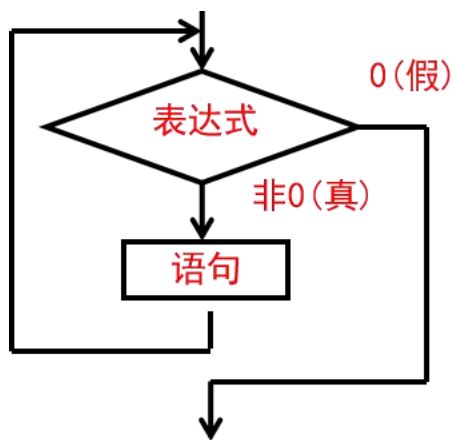
循环体

循环条件表达式：
“真”时执行循环体语句
“假”时不执行

while 循环的特点是：

先判断条件表达式

后执行循环体语句



5.2.2 用 do...while 语句实现循环

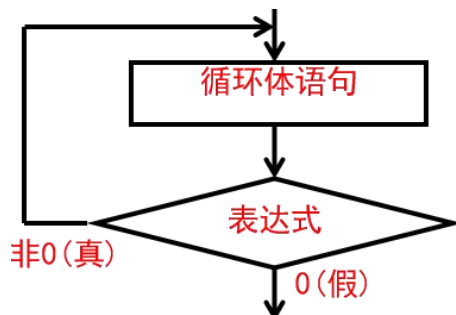
do---while 语句的特点：先无条件地执行循环体，然后判断循环条件是否成立

一般形式为：

do

循环体语句

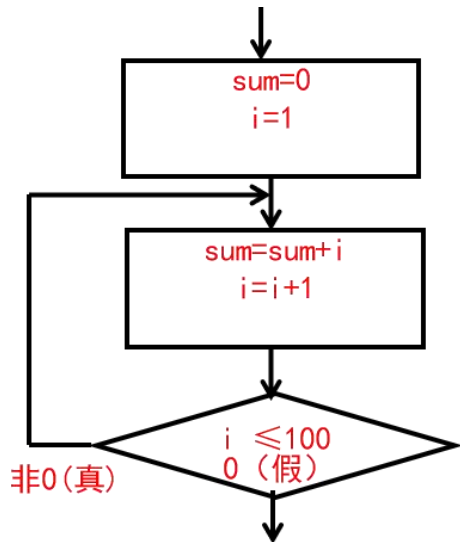
while (表达式);



例 5.2 用 do...while 循环求：

$$1+2+3+\cdots+100, \text{ 即 } \sum_{n=1}^{100} n$$

➤ 解题思路：



```
#include <stdio.h>
```

```
void main()
```

```
{ int i,sum=0;
```

```
    i=1;
```

```
    do
```

```
    {
```

```
        sum=sum+i;
```

```
        i++;
```

```
    }while(i<=100);
```

```
    printf( "%d\n" ,sum);
```

```
}
```

例 5.3 募集慈善基金 10000 元，有若干人捐款，每输入一个人的捐款数后，计算机就输出当时的捐款总和。当某一次输入捐款数后，总和达到或超过 10000 元时，即宣告结束，输出最后的累加值。

➤ 解题思路：

设计一个循环结构，在其中输入捐款数，求出累加值，然后检查此时的累加值是否达到或超过预定值，如果达到了，就结束循环操作。

```
#include <stdio.h>

void main()
{ float amount,sum=0;
  do
  { scanf("%f",&amount);
    sum=sum+amount;
  } while(sum<10000);
  printf("sum=%9.2f\n",sum);
}
```

1000 ✓
1850 ✓
1500 ✓
2600 ✓
2500 ✓
1200 ✓
sum= 10650.00

循环执行的条件

讲解归纳与举例

1.下列程序的输出结果是（ ）。

```
#include "stdio.h"

void main ( )
{ int i=6;
  while (i--)
  printf ("%d",--i) ;
  printf ("\n") ;}
```

- A.531 B.420
C.654321 D.死循环

【正确答案： B】

【答案解析】： 注意 i--及--i 的区别。当 i=0 时结束循环，因此输出 420。

2.有以下程序段

```
int x=3;

do
{ printf("%d",x-=2); }
while (!(--x));
```

其输出结果是（ ）。

【正确答案： 1， -2】

【答案解析】：第一次循环，输出 $x=2$ ， x 的值为 1。然后判断条件 $!(--x)$ ，则 $!(0)$ 为真，输出 $x=2$ ， x 的值为 -2。当 x 的值为 -2 时，判断条件 $!(--x)$ ，则 $!(-3)$ 为假。循环结束。

注意：非 0 都是表示逻辑真，-3 也是逻辑真，取反！变成逻辑假。

5.3 for 语句实现循环

5.3.1 for 语句的一般形式和执行过程

- for 语句不仅可以用于循环次数已经确定的情况，还可以用于循环次数不确定而只给出循环结束条件的情况。
- for 语句完全可以代替 while 语句
- for 语句的一般形式为
- `for (表达式 1; 表达式 2; 表达式 3)`

语句

设置初始条件，只执行一次。可以为零个、一个或多个变量设置初值执行。

for 语句的一般形式为

`for (表达式 1; 表达式 2; 表达式 3)`

语句

设置初始条件，只执行一次。可以为零个、一个或多个变量设置初值执行。

- for语句的一般形式为

`for (表达式1; 表达式2; 表达式3)`

语句

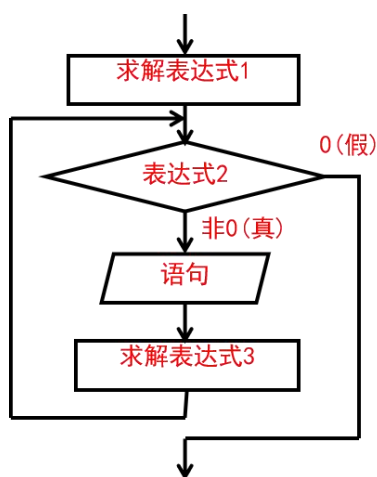
循环条件表达式，用来判定是否继续循环。在每次执行循环体前先执行此表达式，决定是否继续执行循环

- for语句的一般形式为

`for (表达式1; 表达式2; 表达式3)`

语句

作为循环的调整器（循环控制条件），例如使循环变量增值，它是在执行完循环体后才进行的。



➤ for 语句的执行过程：

- (1) 先求解表达式 1
- (2) 求解表达式 2，若其值为真，执行循环体，然后执行下面第 (3) 步。若为假，则结束循环，转到第 (5) 步
- (3) 求解表达式 3
- (4) 转回上面步骤 (2) 继续执行
- (5) 循环结束，执行 for 语句下面的一个语句

for(i=1;i<=100;i++)

sum=sum+i;

用for语句更简单、方便

等价于

i=1;

while(i<=100)

{

sum=sum+i;

i++;

}

5.3.2 for 循环程序举例

➤ 例 5.4 在象棋的棋盘第 1 个格子中放 1 粒麦子，第 2 个格子中放 2 粒麦子，第 3 个格子中放 4 粒麦子，以后按此比例每一格加一倍，一直放到第 64 格(象棋的棋盘是 $8 \times 8 = 64$ 格)，共放多少？

➤ 麦子总粒数= $1+2+2^2+2^3+\dots+2^{63}$

$1\text{m}^3 = 1.42 \times 10^8$ 粒

```
#include <stdio.h>
void main()
{ double p=1, t=1, v;  int i;
  for(i=1; i<64; i++)
  { p = p * 2;          p是当前一个格子中麦子粒数
    t = t + p;          t是当前麦子总粒数
  }                    v是总体积
  v = t / 1.42e8;
  printf("total=%e\n",t);
  printf("volume=%e\n",v);
}
```

total=1.844674e+019
volnme=1.299066e+011

```
#include <stdio.h>
void main()
{ double p=1, t=1, v;  int i;
  for(i=1; i<64; i++)
  { p = p * 2;          改为i=1;i<=64;i++会怎样?
    t = t + p;
  }
  v = t / 1.42e8;
  printf("total=%e\n",t);
  printf("volume=%e\n",v);
}
```

例 5.5 人口增长预测。据 2012 年末统计，我国人口为 13.54 亿人，如果人口的年增长率为 1%，请计算到哪一年中国总人口超过 15 亿。

➤ 解题思路：

假设原来人口为 p_0 ，则一年后的人口为： $p=p_0*(1+r)$ ， r 是年增长率每算出一年的人口后就检查是否达到或超过 15 亿。如果未达到或超过，再计算下一年的人口，直到某一年的人口达到或超过 15 亿为止

```
#include <stdio.h>

void main()
{
    double p=1.354e9,r=0.01;
    int year;
    for(year=2012; p<1.5e9; year++) 注意不是year
        p=p*(1+r);
    printf("year=%d,p=%e\n",year-1,p);
}
```

year=2023,p=1.510615e+009

```
#include <stdio.h>

void main() 计算1000年后的人口??
{
    double p=1.354e9, r=0.01;
    int year;
    for(year=2012; p<1.5e9; year++)  year<=3012
        p=p*(1+r);
    printf("year=%d,p=%e\n",y-1,p);
}
```

讲解归纳与举例

1. 以下程序的输出结果是 。

```
void main()
{
    int a,b;
    for(a=1,b=1;a<=100;a++)
    {
        if(b%3 ==1)
        {
            b+=3;break;
        }
        printf( "%d\n", b);
    }
}
```

【正确答案： 4】

【答案解析】： for 循环语句的理解。

2. 设有整型变量 x，下述语句（ ）不会出现死循环。

A. `for(;; x+=1);`

B. `for( ; (x=getchar())!= '\n' ; );`

C. `while(1) {x++;};`

D. `for(x=10; ;x--);`

【正确答案：B】

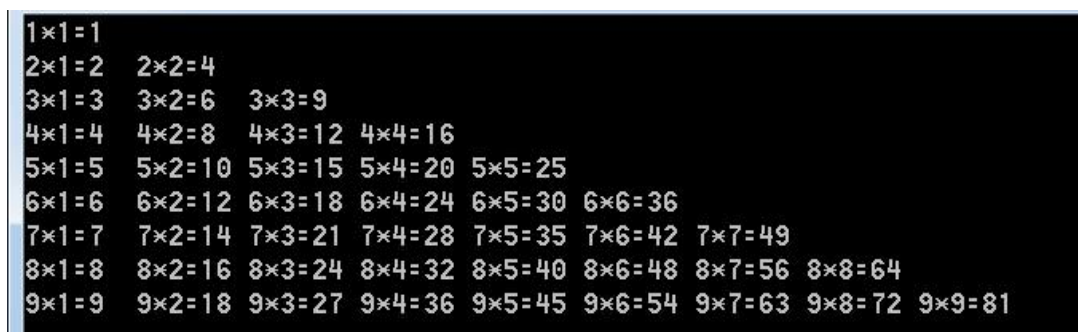
【答案解析】：A 和 D 选项没有循环控制条件，为死循环；C 选项循环条件 `while(1)`，死循环；B 选项当输入回车结束循环。

5.4 循环嵌套

- 一个循环体内又包含另一个完整的循环结构，称为**循环的嵌套**。
- 内嵌的循环中还可以嵌套循环，这就是多层循环。
- 3 种循环（while 循环、do...while 循环和 for 循环）可以互相嵌套。

讲解归纳与举例

1.编程实现如图所示乘法口诀表。



1×1=1								
2×1=2	2×2=4							
3×1=3	3×2=6	3×3=9						
4×1=4	4×2=8	4×3=12	4×4=16					
5×1=5	5×2=10	5×3=15	5×4=20	5×5=25				
6×1=6	6×2=12	6×3=18	6×4=24	6×5=30	6×6=36			
7×1=7	7×2=14	7×3=21	7×4=28	7×5=35	7×6=42	7×7=49		
8×1=8	8×2=16	8×3=24	8×4=32	8×5=40	8×6=48	8×7=56	8×8=64	
9×1=9	9×2=18	9×3=27	9×4=36	9×5=45	9×6=54	9×7=63	9×8=72	9×9=81

【正确答案：见下一页】

【答案解析】：循环结构的应用。

```
#include "stdio.h"
```

```
void main()
```

```
{    int i,j,result;
```

```
    for(i=1;i<10;i++)
```

```
    { for(j=1;j<=i;j++)
```

```
        { result=i*j;
```

```
            printf("%d*%d=%-3d",i,j, result); }
```

```
            printf("\n");
```

```
    }
```

```
}
```

换成 while 循环或者 do...while 循环试试？

5.5 提前结束循环

5.5.1 用 break 语句提前退出循环

在执行循环语句时，在正常情况下只要满足给定的循环条件，就应当一次一次地执行循环体，直到不满足给定的循环条件为止。但是有些情况下，需要提前结束循环。

例 5.6 统计各班级学生的平均成绩。已知各班人数不等，但都不超过 30 人。编一个程序能处理人数不等的各班学生的平均成绩。

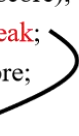
➤ 解题思路：

输入的成绩是负数时，表示本班数据已结束

接收到一个负分数时就提前结束循环

用 break 语句实现提前结束整个循环

```
#include <stdio.h>
void main()
{ float score,sum=0,ave; int i,n;
  for(i=1; i<31; i++)
  { scanf("%f",&score);
    if(score<0) break;
    sum=sum+score;
  }
  n=i-1;
  ave=sum/n;
  printf("n=%d,ave=%7.2f\n",n,ave);
}
```



100 ✖
80 ✖
70 ✖
-1 ✖
n=3, ave= 90.00

➤ break 语句的一般形式为：

```
break;
```

只能用于循环语句（三种循环语句都可以）
和 switch 语句之中，而不能单独使用

5.5.2 用 continue 语句提前结束本次循环

➤ continue 语句的一般形式为：

```
continue;
```

➤ 其作用为结束本次循环，即跳过循环体中下面尚未执行的语句，接着进行下一次是否执行循环的判断。

➤ continue 语句和 break 语句的区别：

continue 语句只结束本次循环，而不是终止整个循环的执行

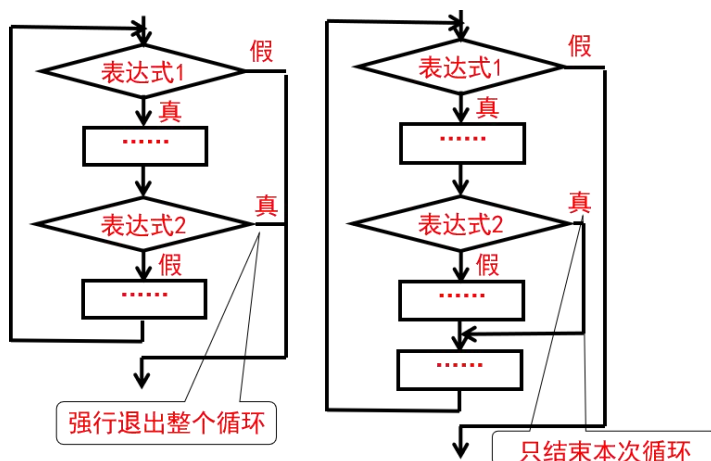
break 语句结束整个循环过程，不再判断执行循环的条件是否成立

while(表达式 1)

```
{  
    :  
    if(表达式 2)  
        break; //强行结束整个循环  
    :  
}
```

while(表达式 1)

```
{  
    :  
    if(表达式 2)  
        continue;  
    /*结束本次循环。即跳过循环体中下面尚未执行的语句，接着进行下一次是否执行循环的判断。*/  
    :  
}
```



例 5.7 输入一个班全体学生的成绩，把不及格的学生成绩输出，并求及格学生的平均成绩。

➤ 解题思路：

在进行循环中，检查学生的成绩，把其中不及格的成绩输出，然后跳过后面总成绩的累加和求平均成绩的语句。

用 continue 语句处理。

```
#include <stdio.h>
void main()
{float score,sum=0,ave; int i,n=0;
  for(i=1; i<6; i++)
  { printf("score:"); scanf("%f",&score);
    if(score<60)
    { printf("Fail:%7.2f\n",score);
      continue; }
    sum=sum+score; n=n+1; }
  ave=sum/n;
  printf("n=%d,ave=%7.2f\n",n,ave);
}
```

score:89✓
score:56✓
Fail:56
score:76✓
score:58✓
Fail:58
score:98✓
n=3 ave=87.67

讲解归纳与举例

1.下面程序段中 while 循环的循环次数是（ ）。

```
int i = 0;
while ( i < 10 )
{ if ( i < 1 )
    continue;
  if ( i == 5 )
    break;
  i++;}
```

- A.1 B.10
C.6 D.死循环、不能确定循环次数

【正确答案：D】

【答案解析】进入循环时，i 值为 0，所以 if 条件成立，执行 continue 语句。会跳过循环体内的其他语句，使得 i 值为 0 无法改变。因此，是死循环。

2.编程题：实现 $1*2*3*\dots*n$ 的操作，当阶乘值>5000 时结束操作。

【正确答案：见下一页】

【答案解析】循环的基本应用。

```
#include <stdio.h>

void main()
{
    int  n;
    int  mul;

    for(n=1,mul=1; ;n++ )
    {
        mul=mul*n;

        if(mul>5000)
            break;
    }

    printf("n=%d,mul=%d\n",n,mul);
}
```

5.6 几种循环比较

- (1) 一般情况下，3 种循环可以互相代替
- (2) 在 while 和 do---while 循环中，循环体应包含使循环趋于结束的语句。
- (3) 用 while 和 do---while 循环时，循环变量初始化的操作应在 while 和 do---while 语句之前完成。而 for 语句可以在表达式 1 中实现循环变量的初始化。
- (4) 都可以用 break 和 continue 语句。

5.7 循环程序综合举例

例 5.8 有一对兔子，出生后第 3 个月起每个月都生一对兔子。小兔子长到第 3 个月后每个月又生一对兔子。假设所有兔子都不死，问 40 个月的兔子总数为多少？

- 每个月的兔子总数依次为 1, 1, 2, 3, 5, 8, 13...是斐波那契（Fibonacci）数列。
- 特点：第 1、2 两个数为 1、1

从第 3 个数开始，该数是其前面两个数之和。

第几个月 小兔子对数 中兔子对数 老兔子对数 兔子总数

第几个月	小兔子对数	中兔子对数	老兔子对数	兔子总数
1	1	0	0	1
2	0	1	0	1
3	1	0	1	2
4	1	1	1	3
5	2	1	2	5
6	3	2	3	8
7	5	3	5	13
⋮	⋮	⋮	⋮	⋮

$$\begin{cases} F_1 = 1 & (n = 1) \\ F_2 = 1 & (n = 2) \\ F_n = F_{n-1} + F_{n-2} & (n \geq 3) \end{cases}$$

f1=1,f2=1
for i=1 to 20
输出f1,f2
f1=f1+f2 f2=f2+f1

```
#include <stdio.h>
void main()
{ long int f1,f2; int i;
  f1=1;f2=1;
  for(i=1; i<=20; i++)
  { printf("%12ld %12ld ",f1,f2);
    if(i%2==0)      控制一行输出的数据个数
      printf("\n");
    f1=f1+f2;
    f2=f2+f1;
  }
}
```

例 5.9 有一个整数 m ，判断它是否素数。

➤ 解题思路：

所谓素数（或称质数）是指除了 1 和它本身以外，不能被任何整数整除的数。

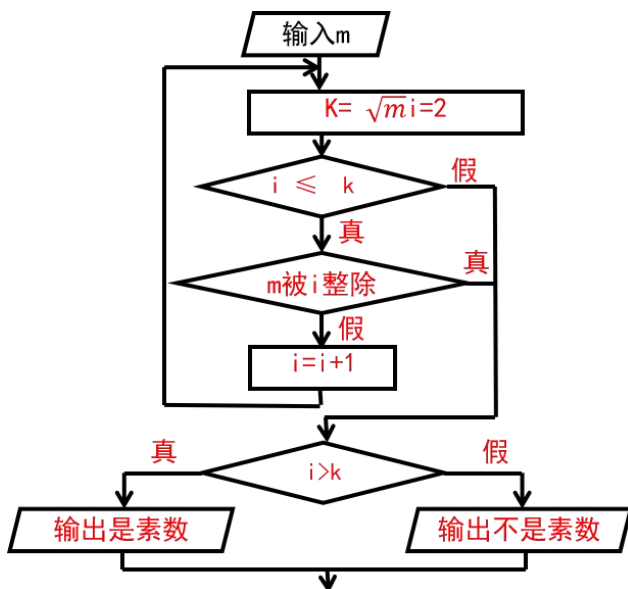
例如 17 是素数，因为它不能被 2 到 16 间任一整数整除。

因此判断一个整数 m 是否素数，只需把 m 被 2 到 $m-1$ 之间的每一个整数去除，如果都不能被整除，那么 m 就是一个素数。

例 5.9 有一个整数 m ，判断它是否素数。

➤ 简化思路：

另外判断方法还可以简化。 m 不必被 $2 \sim m-1$ 之间的每一个整数去除，只需被 $2 \sim \sqrt{m}$ 之间的每一个整数去除就可以了。如果 m 不能被 $2 \sim \sqrt{m}$ 间任一整数整除， m 必定是素数。例如判别 17 是是否为素数，只需使 17 被 $2 \sim 4$ 之间的每一个整数去除，由于都不能整除，可以判定 17 是素数。



```
#include <stdio.h>
#include <math.h>
void main()
{int m,i,k;
printf("number:");
scanf("%d",&m);
k=(int)sqrt(m);
for (i=2;i<=k;i++)
    if(m%i==0) break;
if(i>k) printf("%d is a prime number\n",m);
else printf("%d is not a prime number\n",m);
}
```

number: 17
17 is a prime number

例 5.10 译密码。为使电文保密，往往按一定规律将其转换成密码，收报人再按约定的规律将其译回原文。

- 非字母字符保持原状不变
- 输入一行字符，要求输出其相应的密码
- 解题思路：

(1) 从字母 A 到 V，只需将字母加上 4 就行

例如，‘A’ +4 就是 ‘E’

(2) 从字母 W 到 Z，将字母加 4，再减去 26

‘W’ +4-26 就是 ‘A’

- 解题思路：

输入一个字符给字符变量 c

判定它是否字母（包括大小写），若不是字母，不改变 c 的值；若是字母，则使变量 c 的值改变为其后第 4 个字母

如果新的 c 值范围不在字母范围内，应减去 26

```
#include <stdio.h>
```

```
Void main()
```

```
{ char c;
```

```
while((c=getchar())!= '\n' )
```

```
{ if((c>= 'a' && c<= 'z' ) ||
```

```
(c>= 'A' && c<= 'Z' ))
```

```
{ c=c+4;
```

```
if(c> 'Z' && c<= 'Z' +4 || c> 'z' )
```

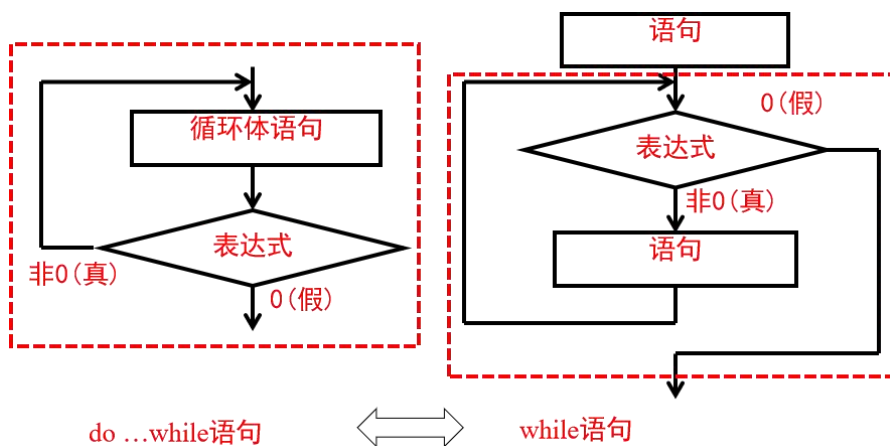
```
c=c-26;  
  
}  
printf("%c",c);  
}  
}
```

China!✓
Glmre!

5.8 提高部分

5.8.1 while 和 do...while 循环的比较

- 凡是能用 while 循环处理的情况，都能用 do...while 循环处理
- do...while 循环结构可以转换成 while 循环结构



例 5.11 while 和 do...while 循环的比较

以下两个程序，循环体是相同的

程序（1）用 while 循环

程序（2）用 do-while 循环

运行时，在有的情况下结果相同，而另一些情况下结果不同，请仔细分析

程序(1)

```
#include <stdio.h>
```

```
void main ( )
```

```
{int s=0,i;
```

```
scanf( "%d" ,&i);
```

```
while (i<=10) {
```

```
    s=s+i;
```

```
    i++;
```

```
}
```

```
printf( "s=%d" ,s);
```

```
}
```

1✓
s=55
再运行一次：
11✓
s=0

程序(2)

```
#include <stdio.h>
void main( )
{int s=0,i;
 scanf("%d",&i);
 do
 { s=s+i;
  i++;
 }while (i<=10);
 printf("s=%d",s);
}
```

1✓
s=55
再运行一次：
11✓
s=11

当while后面的表达式的第一次的值为“真”时，两种循环得到的结果相同；否则，二者结果不相同(相同循环体时)。

5.8.2 for 语句的各种形式

在实际编程中，for 语句相当灵活，形式变化多样

前面介绍过 for 语句的一般形式为

for(表达式 1；表达式 2；表达式 3)语句

表达式 1;

for(; 表达式 2；表达式 3) 语句

for(表达式 1； ; 表达式 3)

if(表达式 2) 语句

for(表达式 1；表达式 2；)

{ 语句

表达式 3;

}

表达式 1;

for(; 表达式 2；)

{ 语句

表达式 3;

}

表达式 1;

for(; ;)

{ if(表达式 2) 语句

 表达式 3;

}

for(表达式1 ; 表达式2 ; 表达式3) 语句

可以是设置循环变量初值的赋值表达式

也可以是与循环变量无关的其他表达式

for(表达式1 ; 表达式2 ; 表达式3) 语句

可以是与循环控制无关的任意表达式

for(表达式1 ; 表达式2 ; 表达式3) 语句

可以是一个简单的表达式
也可以是逗号表达式

for(表达式1 ; 表达式2 ; 表达式3) 语句

一般是关系表达式或逻辑表达式
也可以是数值表达式或字符表达式
只要其值为非0，就执行循环体

for(i=0; (c=getchar())!='\n'; i+=c);

作用：不断输入字符，将它们的ASCII码相加，
直到输入一个“换行”符为止

讲解归纳与举例

1.判断题：do-while 循环至少要执行一次循环体。

【正确答案：对】

【答案解析】do-while 循环的理解。

2.下面程序的功能是：输出 100 以内能被 3 整除且个位数为 6 的所有整数，试填空：

```
#include "stdio.h"

main()
{ int i,j;

  for(i=0;          ;i++)

    { j=i*10+6;

      if(          )

        continue;

      printf( "%d  ",j);

    }

}
```

【正确答案：i<10;j%3!=0】

【答案解析】for 循环的应用。

3.简答题：三种循环的比较？

【参考答案：

- (1) 一般情况下，3 种循环可以互相代替
- (2) 在 while 和 do---while 循环中，循环体应包含使循环趋于结束的语句。
- (3) 用 while 和 do---while 循环时，循环变量初始化的操作应在 while 和 do---while 语句之前完成。而 for 语句可以在表达式 1 中实现循环变量的初始化。
- (4) 三种循环语句都可以使用 break 和 continue 语句。】

【答案解析】三种循环的理解。

本章小结

- (1) 理解循环结构的基本概念。
- (2) 掌握三种循环结构的基本语法，能应用三种循环结构解决实际问题。
- (3) 使用 break，continue 语句来改变循环状态。
- (4) 循环可以嵌套。
- (5) 能应用循环编写程序。

第 6 章 利用数组处理批量数据

本章重难点分析

本章主要介绍数组的基本概念；一维数组的定义和引用；二维数组的定义和引用；字符数组的定义初始化以及使用；字符串的处理以及相关字符串函数等。

本章内容注意以下几点：

一维数组的定义和引用	重点&难点
二维数组的定义和引用	重点
字符数组的定义和引用	重点&难点
字符串及其相关函数	

6.1 为什么要用数组

- 前几章使用的变量都属于基本类型，例如整型、字符型、浮点型数据，这些都是简单的数据类型。
- 对于有些数据，只用简单的数据类型是不够的，难以反映出数据的特点，也难以有效地进行处理。
- 30 名学生成绩，需要用 30 个变量。
- 100 名学生成绩，需要用多少个变量？
- 用 s1, s2, s3, ……，s30 表示成绩，能体现内在联系。
- C 语言用方括号中的数字表示下标，如用 **s[15]** 表示。

一组具有同一属性的数据

- 数组是一组有序数据的集合。数组中各数据的排列是有一定规律的，下标代表数据在数组中的序号。
- 用一个**数组名和下标**唯一确定数组中的元素。
- 数组中的每一个元素都属于**同一个数据类型**。

6.2 怎样定义和引用一维数组

6.2.1 怎样定义一维数组

- 一维数组是最简单的数组
- 数组元素只有 1 个下标的是一维数组
- 数组元素 2 个下标的是二维数组
- 定义数组的方法与定义变量的方法类似。
- 所不同的是一次定义一批有关联的变量。
- 在定义数组时需要指定这批变量的类型、数组名称，数组中包含变量的个数。
- 如 `int a[10];`
- 其中 `a` 是数组名，10 是数组长度（元素个数）

- 定义一维数组的方式为：

类型符 数组名[常量表达式];

注意：常量表达式可以是常量和符号常量，不能包含变量。

- 数组名的命名规则和变量名相同
- 常量表达式给出元素的个数
- 下标从 0 开始，如 `int a[10];`

`a[0],a[1],a[2],...,a[9]`

`int n;`

`scanf("%d",&n);`

`int a[n];`

不合法

6.2.2 怎样引用一维数组的元素

- 必须先定义数组，才能引用数组中的元素。
- 只能逐个引用数组元素而不能一次引用整个数组中的全部元素。
- 引用数组元素的表示形式为：

数组名 [下标]

`a[0]=a[5]+a[2+1]-a[2*3]` 合法

`int n,a[10];` 合法


```
n=a[6];  
a[4]=20;
```

例 6.1 引用数组元素。利用循环给数组元素 a[0]~a[9]赋值为 0~9，然后按逆序输出各元素的值。

- 解题思路：
- 先用循环给数组元素 a[0]~a[9]赋值 0~9，这样，每个数组元素都有固定的值了，然后按 a[9]到 a[0]的顺序输出各元素的值。

```
#include <stdio.h>
```

```
void main()
```

```
{    int i,a[10];  
    for(i=0;i<=9;i++)  
        a[i]=i;  
    for(i=9;i>=0; i--)  
        printf("%d ",a[i]);  
    printf("\n");  
}
```

a[0]a[1]a[2]a[3]a[4]a[5]a[6]a[7]a[8]a[9]

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

6.2.3 一维数组的初始化

- 对数组元素的赋值既可以通过赋值语句来实现，也可以在定义数组时同时给予初值，这就称为数组的初始化。

(1) 定义数组时对全部数组元素赋初值

```
int a[10]={0,1,2,3,4,5,6,7,8,9};
```

(2) 可以只给一部分元素赋值

```
int a[10]={0,1,2,3,4};
```

相当于

```
int a[10]={0,1,2,3,4,0,0,0,0,0};
```

-3

```
int a[5]={1,2,3,4,5};
```

可写为

```
int a[]={1,2,3,4,5};
```

6.2.4 一维数组程序举例

例 6.2 用数组来处理求 Fibonacci 数列问题。Fibonacci 数列问题的含义见例 5.8。

解题思路：

- 建立一个数组，将数列中第 1 个数放在数组第 1 个元素中，数列第 2 个数放在数组第 2 个元素中，…
- 数组序号为 i 的元素的值是其前两个元素值之和。即： $f[i] = f[i-2] + f[i-1]$
- 用循环来求出数组各元素之和。

```
#include <stdio.h>
```

```
void main()
```

```
{ int i;
```

```
    int f[20]={1,1};
```

```
    for(i=2;i<20;i++)
```

$f[0]$ 和 $f[1]$ 的值 1, 1
求 $f[2]$ 和 $f[19]$ 的值

```
        f[i]=f[i-2]+f[i-1];
```

```
    for(i=0;i<20;i++)
```

每行输出 5 个

```
    { if(i%5==0) printf("\n" );
```

```
        printf("%12d",f[i]);
```

```
    }
```

```
    printf("\n");
```

```
}
```

- 例 5.8 程序在顺序求出并输出各个数后，不能保存这些数据，如果要单独输出第 10 个数，是比较困难的。
- 本例用数组处理时，把每个数据都保存在各数组元素中，如果要单独输出第 10 个数，是很容易的，直接输出 $f[9]$ 即可。

例 6.3 假如有 n 个人，各人年龄不同，希望按年龄将他们从小到大排列。

➤ 解题思路：

- ◆ 排序的规律有两种：一种是“升序”，从小到大；另一种是“降序”，从大到小。
- ◆ 把题目抽象为：“对 n 个数按升序排序”。

◆ 采用起泡法排序。

a[0]	9	8	8	8	8	8
a[1]	8	9	5	5	5	5
a[2]	5	5	9	4	4	4
a[3]	4	4	4	9	2	2
a[4]	2	2	2	2	9	0
a[5]	0	0	0	0	0	9

大数沉淀，小数起泡

```
for(i=0;i<5;i++)
    if (a[i]>a[i+1])
        { t=a[i];a[i]=a[i+1];a[i+1]=t; }
```

a[0]	8	5	5	5	5
a[1]	5	8	4	4	4
a[2]	4	4	8	2	2
a[3]	2	2	2	8	0
a[4]	0	0	0	0	8
a[5]	9	9	9	9	9

```
for(i=0;i<4;i++)
    if (a[i]>a[i+1])
        { t=a[i];a[i]=a[i+1];a[i+1]=t; }
```

a[0]	5	4	4	4
a[1]	4	5	2	2
a[2]	2	2	5	0
a[3]	0	0	0	5
a[4]	8	8	8	8
a[5]	9	9	9	9

```
for(i=0;i<3;i++)
    if (a[i]>a[i+1])
        { t=a[i];a[i]=a[i+1];a[i+1]=t; }
```

a[0]	4	2	2
a[1]	2	4	0
a[2]	0	0	4
a[3]	5	5	5
a[4]	8	8	8
a[5]	9	9	9

```
for(i=0;i<2;i++)
    if (a[i]>a[i+1])
        { t=a[i];a[i]=a[i+1];a[i+1]=t; }
```

a[0]	2	0
a[1]	0	2
a[2]	4	4
a[3]	5	5
a[4]	8	8
a[5]	9	9

```
for(i=0;i<1;i++)
    if (a[i]>a[i+1])
        { t=a[i];a[i]=a[i+1];a[i+1]=t; }
```

```
for(i=0;i<5;i++)
    if (a[i]>a[i+1])
```

```
{ .....}
```

```
for(i=0;i<4;i++)
```

```
    if (a[i]>a[i+1])
```

```
    { .....}
```

```
for(i=0;i<1;i++)
```

```
    if (a[i]>a[i+1])
```

```
    { .....}
```

```
for(j=0;j<5;j++)
```

```
for(i=0;i<5-j;i++)
```

```
    if (a[i]>a[i+1])
```

```
    { .....}
```


6.3 怎样定义和引用二维数组

6.3.1 怎样定义二维数组

`float a[3][4], b[5][10];` //定义

a 为 3×4 (3 行 4 列) 的数组

b 为 5×10 (5 行 10 列) 的数组

➤ 二维数组定义的一般形式为

类型符 数组名[常量表达式][常量表达式];

逻辑存储

<code>a[0][0]</code>	<code>a[0][1]</code>	<code>a[0][2]</code>	<code>a[0][3]</code>
<code>a[1][0]</code>	<code>a[1][1]</code>	<code>a[1][2]</code>	<code>a[1][3]</code>
<code>a[2][0]</code>	<code>a[2][1]</code>	<code>a[2][2]</code>	<code>a[2][3]</code>

内存中的存储顺序

6.3.2 怎样引用二维数组的元素

➤ 二维数组元素的表示形式为：

数组名[下标][下标]

`b[1][2]=a[2][3]/2` 合法

`int a[3][4];`

`a[3][4]=3;` 不合法

6.3.3 二维数组的初始化

`int a[3][4]={{1,2,3,4},{5,6,7,8},{9,10,11,12}};`

`int a[3][4]={1,2,3,4,5,6,7,8,9,10,11,12};`

`int a[3][4]={{1},{5},{9}};` 等价于

`int a[3][4]={{1,0,0,0},{5,0,0,0},{9,0,0,0}};`

`int a[3][4]={ {1},{5,6}};`相当于

`int a[3][4]={ {1},{5,6},{0}};`

`int a[3][4]={1,2,3,4,5,6,7,8,9,10,11,12};`

等价于：

`int a[ ][4]={1,2,3,4,5,6,7,8,9,10,11,12};`

`int a[ ][4]={ {0,0,3},{ },{0,10}};`合法

6.3.4 二维数组程序举例

例 6.4 将一个二维数组 a 的行和列的元素互换，存到另一个二维数组 b 中。

$$a = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \Rightarrow b = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

编程思路：

➤ 将 a 数组中第 i 行 j 列元素赋给 b 数组中 j 行 i 列元素。

例如 a[0][0] 赋给 b[0][0]

a[0][1] 赋给 b[1][0]

a[0][2] 赋给 b[2][0]，……

➤ 可以用双层循环来处理，用外循环控制行的变化，内循环控制列的变化。

```
#include <stdio.h>
```

```
void main()
```

```
{ int a[2][3]={ {1,2,3},{4,5,6}};
```

```
int b[3][2],i,j;
```

```
printf("array a:\n");
```

```
for (i=0;i<2;i++)
```

```
{ for (j=0;j<3;j++)
```

```
{ printf("%5d",a[i][j]);
```

```
b[j][i]=a[i][j];
```

```
}
```

```
printf("\n");
```

```
}
```

i 控制行数的变化

j 控制列数的变化

输出a的各元素

a元素值赋给b相应元素

```
printf("array b:\n");           输出b的各元素

for (i=0;i<3;i++)

{   for(j=0;j<2;j++)

        printf("%5d",b[i][j]);

    printf("\n");

}

array a:
    1 2 3
    4 5 6
Array b:
    1 4
    2 5
    3 6
```

例 6.5 有一个班 30 个学生，已知每个学生有 5 门课的成绩，要求输出平均成绩最高的学生的成绩以及该学生的序号。

解题思路：

- 用二维数组，行代表学生，列代表一门课的成绩。
- 要存放 30 个学生 5 门课的成绩和平均成绩，数组的大小应该是 30×6 。
- 为方便操作，我们简化为五个学生 5 门课程。

```
int i,j,max_i;

float sum,max=0;

float s[5][6]={ {78,82,93,74,65},

                {91,82,72,76,67},

                {100,90,85,72,98},

                {67,89,90,65,78},

                {77,88,99,45,89},

                };
```

假设五个学生。第 6 列为 0，用来存放平均成绩。

求每人平均成绩，放在数组每一行的最后一列中

```
for (i=0;i<5;i++)

{ sum=0;

    for (j=0;j<5;j++)

        sum=sum+s[i][j];

    s[i][5]=sum/5;

}
```


找出最高的平均分和该学生的序号

```
for (i=0;i<5;i++)  
    if (s[i][5]>max)  
        {max=s[i][5];max_i=i;}  
  
输出最高的平均分， 和该学生的序号  
  
printf ( “stu_order=%d\n” , max_i ) ;  
printf ( “max=%7.2f\n” , max ) ;  
  
stu_order=2  
max= 89.00
```

讲解归纳与举例

1.以下不能正确定义二维数组的选项是（ ）。

- A.int a[2][2]={ {1},{2}};
- B.int a[][2]={1,2,3,4};
- C.int a[2][2]={ {1},2,3};
- D.int a[2][]={ {1,2},{3,4}};

【正确答案： D】

【答案解析】：二维数组的定义和初始化。a[2][]错误，第二维大小不能省略。

2.下面程序执行后的输出结果是（ ）。

```
#include <stdio.h>  
  
int main()  
{int a[3][3] = { { 1, 2 }, { 3, 4 }, { 5, 6 } }, i, j, s = 0;  
    for ( i = 1; i < 3; i++)  
        for ( j = 0; j <= i; j++)  
            s += a[i][j];  
    printf( ""%d"" , s );  
    return(0);  
}
```

- A.18 B.19

C.20 D.21

【正确答案：A】

【答案解析】：将二维数组的第 1 行和第 2 行元素累加。

6.4 字符数组

6.4.1 怎样定义字符数组及其初始化

- 用来存放字符数据的数组是字符数组
- 字符数组中的一个元素存放一个字符
- 定义字符数组的方法与定义数值型数组的方法类似

```
char c[10];
```

```
c[0]= 'I' ;   c[1]= ' ' ;
```

```
c[2]= 'a' ;   c[3]= 'm' ;
```

```
c[4]= ' ' ;   c[5]= 'h' ;
```

```
c[6]= 'a' ;   c[7]= 'p' ;
```

```
c[8]= 'p' ;   c[9]= 'y' ;
```

```
c[0]c[1]c[2]c[3]c[4]c[5]c[6]c[7]c[8]c[9]
```

I		a	m		h	a	p	p	y
---	--	---	---	--	---	---	---	---	---

```
char c[10]={ 'c' , ' ' , 'p' , 'r' , 'o' , 'g' , 'r' , 'a' , 'm' };
```

```
c[0]c[1]c[2]c[3]c[4]c[5]c[6]c[7]c[8]c[9]
```

c		p	r	o	g	r	a	m	\0
---	--	---	---	---	---	---	---	---	----

也可以写成：

```
char c[]={ 'I' , ' ' , 'a' , 'm' , ' ' , 'h' , 'a' , 'p' , 'p' , 'y' };
```

```
char diamond[5][5]=
```

```
{ { ' ' , ' ' , '*' },
```

```
{ ' ' , '*' , ' ' , '*' },
```

```
{ '*' , ' ' , ' ' , ' ' , '*' },
```

```
{ ' ' , '*' , ' ' , '*' },
```

```
{ ' ' , ' ' , '*' }
```

```
};
```

6.4.2 怎样引用字符数组

例 6.6 输出一个菱形图。

```
#include <stdio.h>
```

```
void main()
```

```
{char diamond[5][5]=
```

```
{ { ' ', ' ', '*', },
```

```
{ ' ', '*', ' ', '*' },
```

```
{ '*', ' ', ' ', ' ', '*' },
```

```
{ ' ', '*', ' ', '*' },
```

```
{ ' ', ' ', '*' }
```

```
};
```

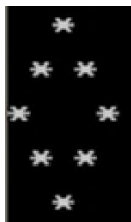
```
int i,j;
```

```
for (i=0;i<5;i++)
```

```
{ for(j=0;j<5;j++)
```

```
printf( "%c" ,diamond[i][j]);
```

```
printf("\n"); }
```



6.4.3 字符串和字符串结束标志

- 在 C 语言中，是将字符串作为字符数组来处理的
- 关心的是字符串的有效长度而不是字符数组的长度
- 为了测定字符串的实际长度，C 语言规定了字符串结束标志 ‘\0’
- ‘\0’ 代表 ASCII 码为 0 的字符
- 从 ASCII 码表可以查到，ASCII 码为 0 的字符不是一个可以显示的字符，而是一个“空操作符”，即它什么也不做
- 用它作为字符串结束标志不会产生附加的操作或增加有效字符，只起一个供辨别的标志。

```
char c[]={ "I am happy" };
```

可写成

```
char c[] = " I am happy";
```

相当于

```
char c[11] = { " I am happy" };
```

```
char c[] = { 'I', ' ', 'a', 'm', ' ', 'h', 'a', 'p', 'p', 'y', '\0' };
```

而与下面的等价（前者长度 11，后者长度 10）：

```
char c[] = { 'I', ' ', 'a', 'm', ' ', 'h', 'a', 'p', 'p', 'y' };
```

```
char c[10] = { " China" };
```

可写成

```
char c[10] = " China";
```

从 c[5] 开始，元素值均为 \0

C	h	i	n	a	\0	\0	\0	\0	\0
---	---	---	---	---	----	----	----	----	----

6.4.4 字符数组的输入输出

➤ 字符数组的输入输出可以有两种方法：

逐个字符输入输出（%c）

整个字符串一次输入输出（%s）

```
char c[] = "China";
```

```
Printf( "%s", c);
```

C	h	i	n	a	\0
---	---	---	---	---	----

➤ 在使用字符串输入输出时要注意有关规定，否则容易出错。

输出字符不包括结束符 '\0'

用 printf 输出：printf("%s", c);

```
char c[10] = "China";
```

```
printf( "%s", c); //输出 China。注意，不用加&符号
```

```
Printf( "%s", c[0]); //不对
```

```
char c[10] = "boy\0girl\0";
```

```
printf( "%s" ,c); //输出 boy
```

➤ 用 scanf 函数输入：scanf("%s" ,c);

```
char c[6];
```

```
scanf( "%s" ,c);
```

China✓ 合法

abcdef✓ 不合法

```
char str[13];
```

```
scanf( "%s" ,str);
```

How are you✓

H	o	w	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0
---	---	---	----	----	----	----	----	----	----	----	----	----	----	----

6.4.5 字符串处理函数

- 在程序中往往需要对字符串作某些操作处理，两个字符串连接、两个字符串进行比较等。
- 在 C 函数库中提供了一些字符串处理函数，使用很方便。

➤ 函数形式	➤ 功能
➤ gets (字符数组)	➤ 从终端输入一个字符串到字符数组
➤ puts (字符数组)	➤ 将一个字符串 (以 '\0' 结束的字符序列) 输出到终端
➤ strcat (字符数组1, 字符数组2)	➤ 连接两个字符数组中的字符串把字符串2接到字符串1的后面
➤ strcpy (字符数组1, 字符串2)	➤ 将字符串2复制到字符数组1中去

➤ strcmp (字符串1, 字符串2)	➤ 比较串1和串2。若串1=串2, 则函数值为0; 若串1>串2, 则函数值为一个正整数; 若串1<串2, 则函数值为一个负整数
➤ strlen (字符数组)	➤ 测试字符串长度
➤ strlwr (字符串)	➤ 将字符串中大写字母换成小写字母
➤strupr (字符串)	➤ 将字符串中小写字母换成大写字母

6.4.6 字符数组应用举例

例 6.7 有 3 个字符串，要求找出其中“最大”者。

解题思路：

- 按英文字典的排列，后面出现的串大。
- 要求处理 3 个字符串，需要定义一个二维的字符数组。
- 假定每个字符串不超过 19 个字符，则可定义二维的大小为 3×20。
- 可以把 str[0],str[1],str[2]看作 3 个一维字符数组。
- 它们各有 20 个元素。
- 可以把它们如同一维数组那样进行处理。

str[0]	C	h	i	n	a	\0	\0	\0	...	\0
str[1]	J	a	p	a	n	\0	\0	\0	...	\0
str[2]	I	n	d	i	a	\0	\0	\0	...	\0

```
#include<stdio.h>
```

```
#include<string.h>
```

```
void main ( )
```

```
{ char string[20],str[3][20];   int i;
```

```
    for (i=0;i<3;i++)    gets (str[i]);
```

```
    if (strcmp(str[0],str[1])>0)
```

```
        strcpy(string,str[0]);
```

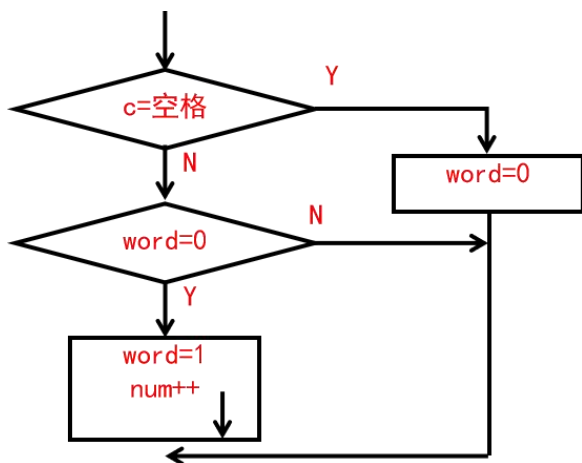
CHINA✓
HOLLAND✓
AMERICA✓
 The largest string is:
HOLLAND

```
else strcpy(string,str[1]);  
  
if (strcmp(str[2],string)>0)  
    strcpy(string,str[2]);  
  
printf( "The largest string  
        is:\n%s\n" ,string);  
  
}
```

例 6.8 输入一行字符，统计其中有多少个单词，单词之间用空格分隔开。

解题思路：

- 通过空格统计单词：由空格出现的次数（连续的空格作为一次空格；一行开头的空格不统计在内）决定单词数目。
- 从第一个字符开始逐个检查是否为空格。
 - （1）如测出某字符为非空格，而它前面的字符是空格，则表示“新的单词开始”。设变量 **num**，用来累计单词数，初值为 0。当发现“新的单词开始”，使 **num** 累加 1，表示增加一个单词。
 - （2）如果当前字符为非空格而其前面的字符也是非空格，则意味着仍然是原来那个单词的继续，**num** 不应再累加 1。
- 设变量 **word**，用来表示前一个字符是否空格，以 0 代表前一个字符是空格；**word** 等于 1，意味着前一个字符为非空格，**word** 的初值置为 0。



核心代码：

```
if(c==' ')  
    word=0;  
else if(word==0)
```



```
{ word=1;

    num++;

}
```

如果输入 I am a boy.对每个字符的有关参数状态如下表所示。

当前字符	I		a	m		a		b	o	y	.
是否空格	否	是	否	否	是	否	是	否	否	否	否
word原值	0	1	0	1	1	0	1	0	1	1	1
新单词开始 否	是	否	是	否	否	是	否	是	否	否	否
word新值	1	0	1	1	0	1	0	1	1	1	1
num值	1	1	2	2	2	3	3	4	4	4	4

```
#include <stdio.h>

void main()

{ char string[81];  int i,num=0,word=0;

    char c;

    gets(string);

    for (i=0;(c=string[i])!= '\0' ;i++)

        if(c== ' ' ) word=0;

        else if(word==0)

            { num++;      word=1;  }

    printf( "There are %d words in this

line.\n" ,num);

}
```

讲解归纳与举例

1.执行以下程序段后，程序的运行结果是 。

```
#include<stdio.h>

void main()

{      char str[ ]= "1a2b3c" ; int i;
```

```
for(i=0;str[i]!='\0';i++)  
    if (str[i]>= '0' && str[i]<= '9' )  
        printf( "%c" ,str[i]);    }
```

【正确答案：123】

【答案解析】：字符数组的理解与应用。

2.填空题：将字符数组 a 中下标值为偶数的元素从小到大排列，其他元素不变。

```
#include "stdio.h"  
  
#include "string.h"  
  
main()  
{ char a[]="clanguage",t;  
    int i,j,k;  
    k=strlen(a);  
    for(i=0;i<=k-2;i+=2)  
        for(j=i+2;j<=k;        )  
            if(        )  
                { t=a[i];a[i]=a[j];a[j]=t;}  
    puts(a);  
    printf( "\n" );  
}
```

【正确答案：j=j+2 ； a[i]>a[j]】

【答案解析】：字符数组的理解与应用。

6.5 提高部分

6.5.1 为什么在定义二维数组时为何采用两对双括号的形式

如 `int a[3][4];`

好处是：

- 使二维数组可被看作是一种特殊的一维数组，它由 3 个一维数组组成。
- 例如，`a` 是一个二维数组，可以把它看作是一个一维数组，它包括 3 个元素：`a[0]`、`a[1]`、`a[2]`，每个元素又是一个包含 4 个元素的一维数组。

```
a[0]  a[0][0]  a[0][1]  a[0][2]  a[0][3]
a[1]  a[1][0]  a[1][1]  a[1][2]  a[1][3]
a[2]  a[2][0]  a[2][1]  a[2][2]  a[2][3]
```

`char str[3][20];`

`scanf(“%s”,str[0]);` 合法

`printf(“%s\n”,str[0]);` 合法

比较 `str[0]`和 `str[1]` 合法

6.5.2 对 C 的字符串函数的详细说明

使用字符串函数时，在程序开头用`#include <string.h>`

1.gets 函数（读入字符串函数）

- 其一般形式为：

`gets（字符数组）`

- 作用是输入一个字符串到字符数组

`gets（str）；`

Computer✓

2.puts 函数（输出字符串函数）

其一般形式为：

`puts（字符数组）`

作用是将一个字符串输出到终端

`char str[20]=“China”；`

```
puts(str);
```

输出 China

```
char str[ ]= "China\nBeijing" ;
```

```
puts(str);
```

输出 China

Beijing

3.strcat 函数（字符串连接函数）

➤ 其一般形式为：

strcat（字符数组 1，字符数组 2）

➤ 其作用是把两个字符串连接起来，把字符串 2 接到字符串 1 的后面，结果放在字符数组 1 中。

```
char str1[30]= "People" ; //str1 要足够大
```

```
char str2[]= "China" ;
```

```
printf( "%s" , strcat(str1,str2));
```

输出：PeopleChina

4.strcpy 和 strncpy 函数（字符串复制）

➤ strcpy 一般形式为：

strcpy（字符数组 1，字符串 2）

➤ 作用是将字符串 2 复制到字符数组 1 中去

```
char str1[10],str2[]= "China" ;
```

```
strcpy(str1,str2); //str1 要足够大
```

也可以是：

```
strcpy(str1, "China" );
```

C	h	i	n	a	\0	\0	\0	\0	\0
---	---	---	---	---	----	----	----	----	----

```
char str1[10],str2[]= "China" ;
```

```
str1= "China" ; //错误
```

```
str1=str2; //错误
```

只能用 strcpy 函数将一个字符串复制到另一个字符数组中去。

Char a[5];

a[0]= 'c' ;//合法

- 可以用 `strcpy` 函数将字符串 2 中前面 n 个字符复制到字符数组 1 中去。
- `strcpy (str1, str2, 2) ;`
- ◆ 作用是将 str2 中最前面 2 个字符复制到 str1 中，取代 str1 中原有的最前面 2 个字符。
- ◆ 复制的字符个数 n 不应多于 str1 中原有的字符。

5.strcmp 函数（字符串比较函数）

- 其一般形式为

`strcmp(字符串 1, 字符串 2)`

- 作用是比较字符串 1 和字符串 2
- `strcmp(str1,str2);`
- `strcmp(“China”, “Korea” );`
- `strcmp(str1, “Beijing” );`
- 字符串比较的规则是：将两个字符串自左至右逐个字符相比，直到出现不同的字符或遇到 ‘\0’ 为止。
- 如全部字符相同，认为两个字符串相等。
- 若出现不相同的字符，则以第一对不相同的字符的比较结果为准。

A” < “ B” “ a” > “ A

computer” > “ compare

these” > “ that

36+54” > “ ! \$ &#

CHINA” > “ CANADA

DOG” < “ cat

- 比较的结果由函数值带回

如果字符串 1=字符串 2，则函数值为 0

如果字符串 1>字符串 2，则函数值为一个正整数

如果字符串 1<字符串 2，则函数值为一个负整数

`if(str1>str2) printf(“yes” );` 错误

```
if(strcmp(str1,str2)>0)
```

```
    printf( "yes" );    正确
```

6.strlen 函数（测字符串长度函数）

➤ 其一般形式为：

```
strlen （字符数组）
```

➤ 它是测试字符串长度的函数

➤ 函数的值为字符串中的实际长度

```
char str[10]= "China" ;
```

```
printf( "%d" ,strlen(str));
```

➤ 输出结果是 5

➤ 也可以直接测试字符串常量的长度

```
strlen( "China" );
```

7.strlwr 函数（转换为小写函数）

➤ 其一般形式为

```
strlwr（字符串）
```

➤ 函数的作用是将字符串中大写字母换成小写字母

8.strupr 函数（转换为大写函数）

➤ 其一般形式为

```
strupr（字符串）
```

➤ 函数的作用是将字符串中小写字母换成大写字母

讲解归纳与举例

1.面程序段执行后的输出结果是（ ）。

```
char c[] = "\t\b\\0will\n";
```

```
printf( "%d", strlen( c ) );
```

A.14 B.3

C.9 D.6

【正确答案：B】

【答案解析】：strlen 返回字符串 s 的长度。s 可以是字符串常量、字符数组或指向字符数组的指针，数到第一个空字符认为字符串结束。因此字符串 c 的长度为 3，包括 3 个有效字符，分别为 ‘\t’ ‘\b’ ‘\’。

2.下面程序执行后的输出结果是（ ）。

```
#include <stdio.h>

#include <string.h>

int main()
{
    char arr[2][4];

    strcpy( arr, "you" );

    strcpy( arr[1], "me" );

    arr[0][3] = '&';

    printf( "%s", arr );

    return(0);
}
```

A.you&me B.you

C.me D.错误

【正确答案：A】

【答案解析】：二维数组的存储可以作为两层一维数组的存储看待。即在存储空间中先存储 arr[0]，然后接着存储 arr[1]。

3.编程题：从键盘输入一句话保存在字符数组中，统计其包含的大小写英文字母个数并输出统计结果。

【正确答案】：见下一页

【答案解析】：数组综合应用。

```
#include <stdio.h>

#include <string.h>

int main()
{char s[20];int i,count=0;

gets(s);puts(s);

for(i=0; s[i]!='\0';i++){

    if((s[i]>='a' && s[i]<='z') || (s[i]>='A' && s[i]<='Z')){
```

```
count++;  
  
    }  
  
}  
  
printf("count=%d",count);  
  
}
```

本章小结

- (1) 理解数组的基本概念。
- (2) 一维数组的定义和引用。
- (3) 二维数组的定义和引用。
- (4) 字符数组的定义和引用。
- (5) 字符串相关函数。
- (6) 能应用数组解决实际问题，编写程序。

第7章 用函数实现模块化程序设计

本章重难点分析

本章主要介绍为什么要使用函数，函数的基本定义和调用；函数的嵌套调用和递归调用；数组作为函数参数的使用；变量的作用域和生存期；内部函数和外部函数等。

本章内容注意以下几点：

函数的基本定义和调用 **重点&难点**

函数的嵌套调用和递归调用 **重点**

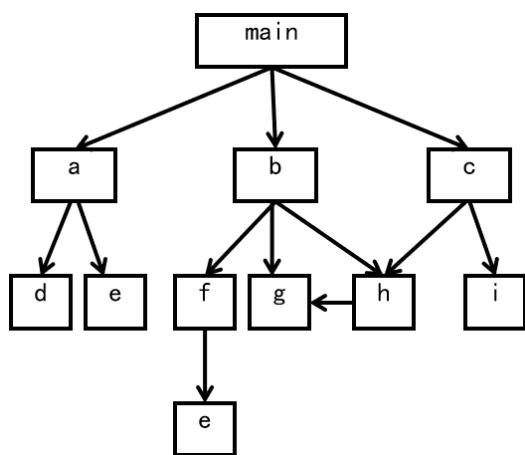
数组作为函数参数的使用 **难点**

7.1 函数是什么

- 如果程序的功能比较多，规模比较大，把所有的程序代码都写在一个主函数中，就会使主函数变得庞大、头绪不清，使阅读和维护程序变得困难。
- 有时程序中要多次实现某一功能，就需要多次重复编写实现此功能的程序代码。这使程序冗长，不精炼。
- 采用“组装”的办法简化程序设计过程
- 事先编好一批函数实现各种不同的功能
- 用到什么函数就直接装使用就可以
- 这就是模块化的程序设计
- 函数就是功能（Function）
- 每一个函数用来实现一个特定的功能
- 函数的名字应反映其代表的功能

在设计一个较大的程序时，往往把它分为若干个程序模块，每一个模块包括一个或多个函数，每个函数实现一个特定的功能。

一个C程序可由**一个主函数和若干个其他函数构成**。由主函数调用其他函数，其他函数也可以互相调用。同一个函数可以被一个或多个函数调用任意多次。



除了可以使用库函数外，还可以编写一些本领域或本单位常用到一些专用函数，供本领域或本单位的人员使用。

在程序设计中要善于利用函数，可以减少各人重复编写程序段的工作量，同时可以方便地实现模块化的程序设计。

例 7.1 输出以下的结果，用函数调用实现。

```
*****
```

```
How do you do!
```

```
*****
```

➤ 解题思路：

在输出的文字上下分别有一行“*”号，显然不必重复写这段代码，用一个函数 `print_star` 来实现输出一行“*”号的功能。

再写一个 `print_message` 函数来输出中间一行文字信息。

用主函数分别调用这两个函数。

```
#include <stdio.h>
```

```
void main(
```

```
{ void print_star(); //函数声明
```

```
    void print_message(); //函数声明
```

```
    print_star(); //函数调用
```

```
    print_message(); //函数调用
```

```
    print_star(); //函数调用
```

```
}
```

```
//函数定义
```

```
void print_star ()  
{printf(" *****\n" );}
```

//函数定义

```
void print_message()  
{printf(" How do you do!\n" );}
```

说明：

（1）一个 C 程序由一个或多个程序模块组成，**每一个程序模块作为一个源程序文件**。对于较大的程序，一般不把所有内容全放在一个源程序文件中，而是将它们分别放在若干个源文件中，由若干个源程序文件组成一个 C 程序。这样便于分别编写、分别编译，提高调试效率。一个源程序文件可以为多个 C 程序所调用。

（2）一个源程序文件由一个或多个函数以及其他有关内容组成。一个源程序文件是一个编译单位，在程序编译时是以源程序文件为单位进行编译的，而不是以**函数**为单位进行编译的。

（3）不论 main 函数出现在什么位置，总是**从 main 函数开始执行**。如果在 main 函数中调用其他函数，在调用后流程返回到 main 函数，在 main 函数中结束整个程序的运行。

（4）所有函数都是平行的，即在定义函数时是分别进行的，是互相独立的。一个函数并不从属于另一个函数，即**函数不能嵌套定义**。函数间可以互相调用，但不能调用 main 函数。main 函数是由系统调用的。

（5）从用户使用的角度看，函数有两种。

库函数，它是由系统提供的，用户不必自己定义而直接使用它们。应该说明，不同的 C 语言编译系统提供的库函数的数量和功能会有一些不同，当然许多基本的函数是共同的。

用户自己定义的函数。它是用以解决用户专门需要的函数。

（6）从函数的形式看，函数分两类。

①**无参函数**。函数没有参数，一般用来执行固定的一组操作。无参函数可以带回或不带回函数值，但一般以不带回函数值的居多。

②**有参函数**。在调用函数时，要给出实参。主调函数在调用被调用函数时，通过参数向被调用函数传递数据，一般情况下，执行被调用函数时会得到一个函数值，供主调函数使用。

讲解归纳与举例

1. 以下关于函数的叙述中正确的是（ ）。

A. 每个函数都可以被其它函数调用（包括 main 函数）

- B.每个函数都可以被单独编译
- C.每个函数都可以单独运行
- D.在一个函数内部可以定义另一个函数

【正确答案：B】

【答案解析】：主函数除外，用户定义的函数或库函数都可以互相进行调用。所以选项 A 错误。每个函数可以单独被编译成二进制代码，但不是所有的函数都可以单独运行，程序的运行需要从主函数 `main` 开始。所以选项 C 错误。C 语言规定，不能在一个函数内部再定义函数，所以选项 D 错误。

7.2 函数的定义和调用

7.2.1 为什么要定义函数

C 语言要求，在程序中用到的所有函数，必须“先定义，后使用”。

指定函数名字、函数返回值类型、函数实现的功能以及参数的个数与类型，将这些信息通知编译系统。

指定函数的名字，以便以后按名调用。

指定函数类型，即函数返回值的类型。

指定函数参数的名字和类型，以便在调用函数时向它们传递数据。

指定函数的功能。这是最重要的，这是在函数体中解决的。

如果程序中要调用库函数，只需用#include 指令把有关的头文件包含到本文件模块中即可。

如果想使用库函数中没有的函数，需要程序设计者在程序中自己定义。

7.2.2 怎样定义函数

1.怎样定义无参函数

函数名后面圆括号中空的，没有参数

定义无参函数的一般形式为：

```
类型名 函数名 ( )  
{ 函数体  
}
```

2.怎样定义有参函数

定义有参函数的一般形式为：

```
类型名 函数名 (形式参数表列)  
{  
    函数体  
}
```

```
int max (int x,int y)
```

```
{int z;  
    if(x>y) z=x;  
    else z=y;
```

```
    return(z);  
}
```

7.2.3 怎样调用函数

1.调用无参函数的形式

函数名 ()

如 `print_star ( )`

2.调用有参函数的形式

函数名 (实参表列)

如 `max(a,b)`

例 7.2 输入两个整数，输出二者中的大者。要求在主函数中输入两个整数，用一个函数 `max` 求出其中的大者，并在主函数中输出此值。

➤ 解题思路：

题目要求用一个 `max` 函数实现比较两个整数，并将得到的大数带回主函数。显然，两个整数中的大者也应该是整数，因此 `max` 函数应当是 `int` 型。

两个数是在主函数中输入的，在 `max` 函数中进行比较，因此应该定义为有参函数，在函数调用时进行数据的传递。

```
#include <stdio.h>  
void main()  
{ int max(int x,int y);  函数声明;  
  int a,b,c;  
  printf("please input two number:");  
  scanf("%d,%d",&a,&b);  
  c = max(a,b);          函数调用  
  printf("max is %d\n",c); }  
int max(int x,int y)     函数定义  
{ int z;  
  if (x>y) z=x;  
  else z=y;  
  return(z);}
```

```
int max(int x,int y)
{
    int z;
    if (x>y) z=x;
    else z=y;
    return(z);
}
```

if (x>y) return(x);

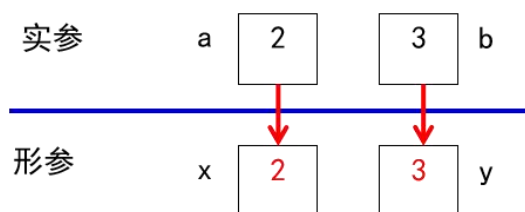
else return(y);

```
#include <stdio.h>
void main()
{ int max(int x,int y); int a,b,c;
  printf( "please input two number:");
  scanf("%d,%d",&a,&b);
  c = max(a,b);
  printf("max is %d\n",c);}
int max(int x,int y)
{ int z;
  if (x>y) z=x;
  else z=y;
  return(z);}
```

实际参数

形式参数

在定义函数中指定的形参，在未出现函数调用时，它们并不占内存中的存储单元。在发生函数调用时，函数 max 的形参被临时分配内存单元。将实参的值传递给形参。



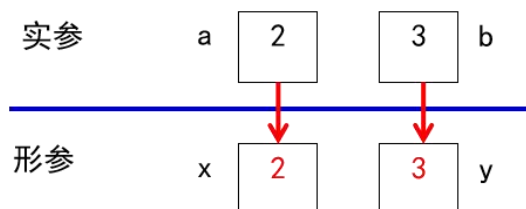
在执行 max 期间，比较形参的值，把较大的值赋给 z。

用 return 语句将函数的返回值带回到主函数。

调用结束，形参单元被释放。

实参单元仍保留并维持原值，没有改变。

如果在执行一个被调用函数时，形参的值发生改变，不会改变主调函数的实参的值。



按函数在程序中出现的位置来分，可以有以下 3 种函数调用方式

1.函数语句

调用没有返回值的函数，函数调用单独作为一个语句

如例 7.1 中的 “`print_star();`”

2.函数表达式

函数出现在一个表达式中，这种表达式称为函数表达式

如例 “`c=2*max(a,b);`” 或者 “`c=max(a,b);`”

3.函数参数

函数调用作为一个函数的实参

如 `printf(" %d",max(a,b));`

7.2.4 对被调用函数的声明和函数原型

在一个函数中调用另一个函数需要具备如下条件：

- (1) 被调用函数必须是已经定义的函数（是库函数或用户自己定义的函数）。
- (2) 如果使用库函数，应该在本文件开头加相应的`#include` 指令。
- (3) 如果使用自己定义的函数，而该函数的位置在调用它的函数后面，应该进行函数声明。

已定义的函数的首部，再加一个分号，就形成了对函数的“声明”，函数声明称为“**函数原型**”。

函数原型的一般形式有两种：

如 `int max(int x,int y);`

`int max(int,int);`

如果被调函数的定义出现在主调函数之前，可以不必加声明。

原型说明可以放在文件的开头，这时本文件中所有函数都可以使用此函数。

讲解归纳与举例

1.C 语言规定，函数返回值的类型是由（ ）决定的。

- A.return 语句中的表达式类型
- B.调用该函数时的主调函数
- C.调用该函数时由系统临时
- D.在定义函数时所指定的函数类型

【正确答案：D】

【答案解析】：函数返回值的类型是由函数定义中的返回类型来决定的。当 **return** 表达式的类型与此不合同时，返回时会进行隐式类型转换；如果不能转换，则出现编译错误。

2.若已定义的函数有返回值，则以下关于该函数调用的叙述中错误的是（ ）。

- A.函数调用可以作为独立的语句存在
- B.函数调用可以作为一个函数的实参
- C.函数调用可以出现在表达式中
- D.函数调用可以作为一个函数的形参

【正确答案：D】

【答案解析】：对函数调用的三种形式的理解。

3.判断题：函数必须有返回值，否则不能定义成函数。

【正确答案：错误】

【答案解析】：函数可以不返回数据，此时返回类型应写成 **void**，表示没有返回值。

4.程序执行的结果是（ ）

```
#include <stdio.h>

void fun( int x, int y, int z ) {
    z = x * x + y * y;
}

int main() {
    int a = 31;
```

```
fun( 5, 2, a );  
printf( "%d", a );  
return(0); }
```

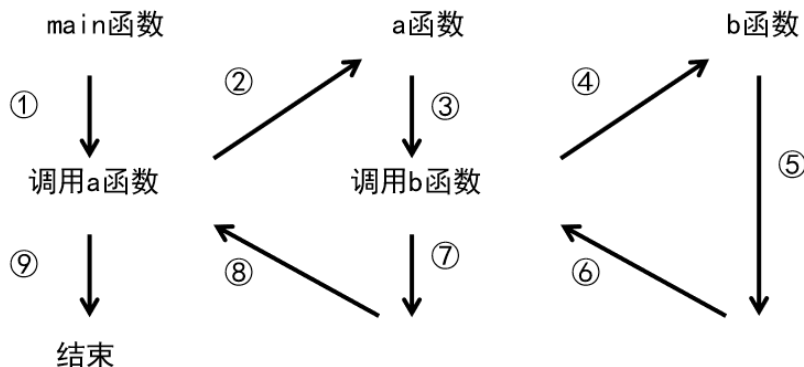
【正确答案：31】

【答案解析】：值传递时，实参数传递给形参是单向传递，即只能由实参传递给形参，而不能由形参传回给实参。

7.3 函数的嵌套调用和递归调用

7.3.1 函数的嵌套调用

调用一个函数的过程中，又可以调用另一个函数，称为**函数的嵌套调用**。



例 7.3 输入 4 个整数，找出其中最大的数。用一个函数来实现。

➤ 解题思路：

定义 max_4 函数，找 4 个数中最大者。

max_4 中再多次调用 max，找 4 个数中的大者，然后把它作为函数值返回 main 函数。

```
#include <stdio.h> 对max_4函数声明
```

```
void main()
```

```
{ int max_4(int a,int b,int c,int d);
```

```
int a,b,c,d,max;
```

```
printf("4 interger numbers:");
```

```
scanf("%d%d%d%d",&a,&b,&c,&d);
```

```
max=max_4(a,b,c,d); //函数调用
```

```
printf("max=%d \n",max);
```

```
} 主函数
```

```
int max_4(int a,int b,int c,int d)
{ int max(int a,int b);
  int m;
  m=max(a,b);
  m=max(m,c); //函数调用
  m=max(m,d);
  return(m);
}
```

对max函数声明

max_4函数

```
int max(int x,int y)
{ if(x>y)
    return x;
  else
    return y;
}
```

return(x>y?x:y);

max函数

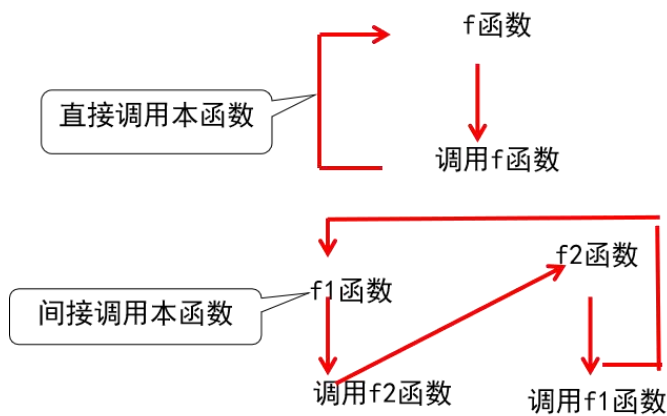
7.3.2 函数的递归调用

在调用一个函数的过程中又出现直接或间接地调用该函数本身，称为函数的递归调用。

C 语言的特点之一就在于允许函数的**递归调用**。

```
int f(int x)
{
    int y,z;
    z=f(y);
    return (2*z);
}
```

在 f 函数体内又调用了 f 函数。



应使用 if 语句控制结束调用

例 7.6 有 5 个学生坐在一起

问第 5 个学生多少岁？他说比第 4 个学生大 2 岁

问第 4 个学生岁数，他说比第 3 个学生大 2 岁

问第 3 个学生，又说比第 2 个学生大 2 岁

问第 2 个学生，说比第 1 个学生大 2 岁

最后问第 1 个学生， he 说是 10 岁

请问第 5 个学生多大

➤ 解题思路：

要求第 5 个年龄，就必须先知道第 4 个年龄

要求第 4 个年龄必须先知道第 3 个年龄

第 3 个年龄又取决于第 2 个年龄

第 2 个年龄取决于第 1 个年龄

每个学生年龄都比其前 1 个学生的年龄大 2

➤ 解题思路：

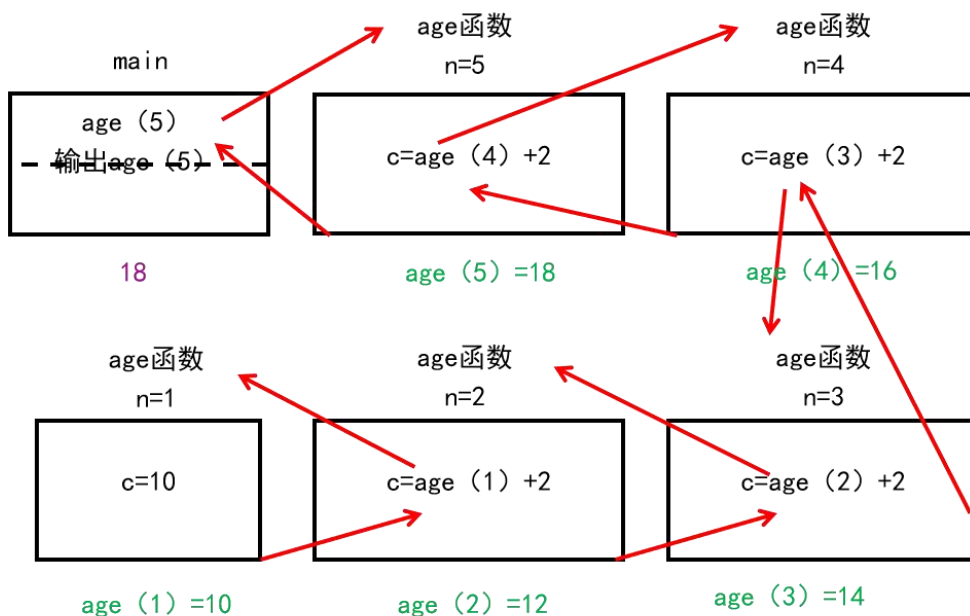
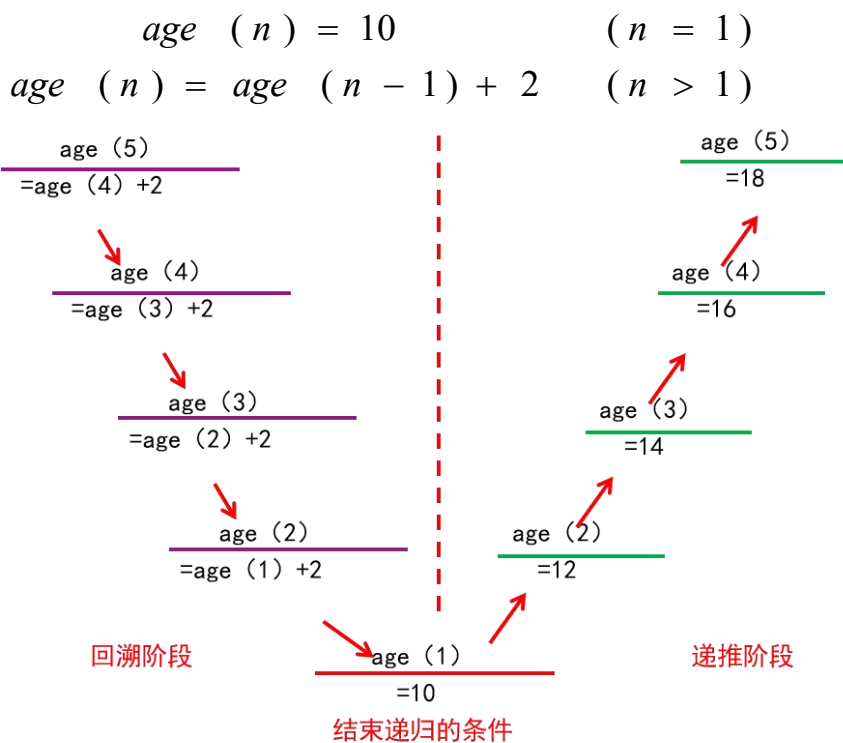
$$\text{age}(5) = \text{age}(4) + 2$$

$$\text{age}(4) = \text{age}(3) + 2$$

$$\text{age}(3) = \text{age}(2) + 2$$

$$\text{age}(2) = \text{age}(1) + 2$$

$$\text{age}(1) = 10$$



例 7.5 分别用递推方法和递归方法求 $n!$ ，即 $1 \times 2 \times \dots \times n$ 。

1. 用递推方法求 $n!$

➤ 解题思路：

从 1 开始，乘 2，再乘 3……一直乘到 n 。这种方法容易理解，也容易实现。递推法的特点是从一个已知的事实出发，按一定规律推下一个事实，再从这个新的已知的事实出发，再向下推出一个新的事实……这是和递归不同的。

```
#include <stdio.h>

void main()
{ long fac(int n);

  int n;

  long fact=0;

  printf( "input an integer number:" );

  scanf( "%d" ,&n);

  fact=fac(n);

  printf( "%d!=%ld\n" ,n,fact);

}
```

```
long fac(int n)
{
  int i;

  long fact=1;

  for(i=1;i<=n;i++)

    fact=fact*i;

  return fact;
}
```

2.用递归方法求 n!

解题思路：

递归的思路和递推是相反的，并不是先求 1 再求 1×2 再×3…，直到×n，而是直接从目标出发提出问题：

5! 等于 4! ×5，而 4! =3! ×4…，而 1! 是已知的，不必再回溯了。

$$n! = \begin{cases} n! = 1 & (n = 0,1) \\ n \cdot (n-1)! & (n > 1) \end{cases}$$

```
#include <stdio.h>

void main()

{ long fac(int n);

  int n,y;

  printf("input an integer number:");
```

```
scanf("%d",&n);

y=fac(n);

printf("%d!=%ld\n",n,y);

}

long fac(int n)

{ long f;

  if(n<0)

    printf( "n<0,data error!" );

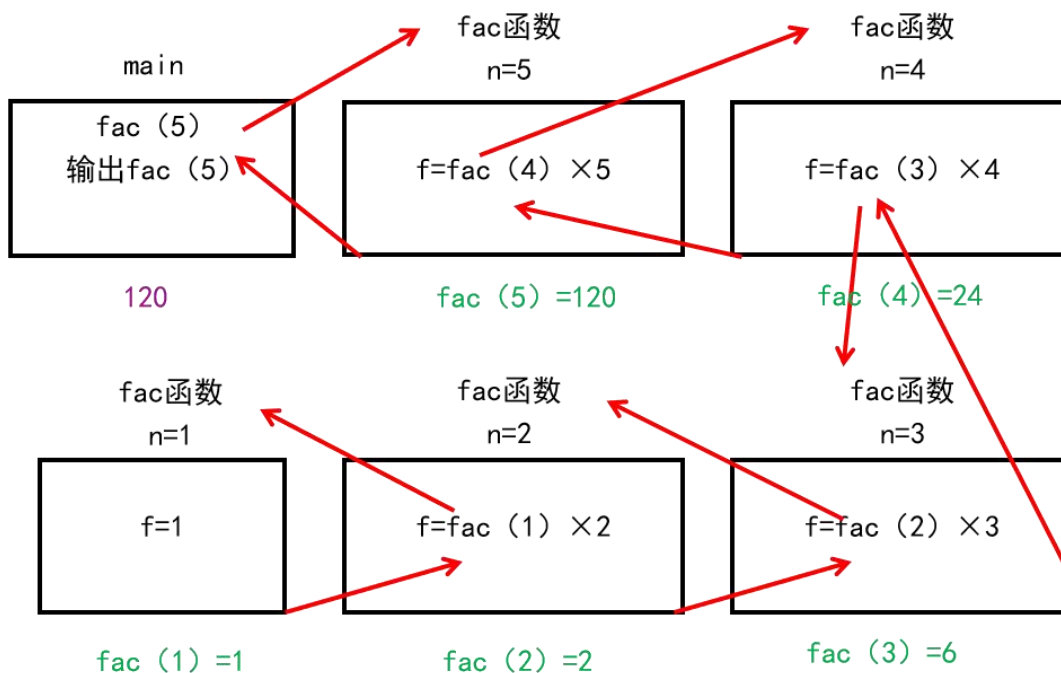
  else if(n==0 || n==1)

    f=1;

  else f=fac(n-1)*n;

  return(f);

}
```



递归调用的特点：

执行“未知→未知→……递归边界条件已知→已知→已知”的过程。

用递归方法解题的条件：

- (1) 所求解的问题能转化为用同一方法解决的子问题。
- (2) 子问题的规模比原问题的规模小。

(3) 必须要有递归结束条件, 停止递归, 否则形成无穷递归, 系统无法实现。

讲解归纳与举例

1. 程序执行的结果是 ()

```
#include <stdio.h>

int f( int n ) {
    if ( n == 1 ) return(1);
    return(f( n - 1 ) + 1); }

int main() {
    int i, j = 0;
    for ( i = 1; i < 3; i++ )
        j += f( i ); printf( "" "%d", j ); return(0); }
```

【正确答案: 3】

【答案解析】: 函数递归执行可知, $f(1)=1, f(2)=2, j=f(1)+f(2)$, 所以输出结果为 3。

2. 以下叙述中正确的是 ()。

- A. 函数的定义可以嵌套, 但函数的调用不可以嵌套
- B. 函数的定义不可以嵌套, 但函数的调用可以嵌套
- C. 函数的定义和函数的调用均不可以嵌套
- D. 函数的定义和函数的调用均可以嵌套

【正确答案: B】

【答案解析】: C 语言不允许在函数体内嵌套定义函数。在调用一个函数的过程中, 又调用另一个函数, 称为函数嵌套调用, C 语言允许函数多层嵌套调用, 只要在函数调用前有函数声明即可。

3. 程序输出结果 ()。

```
#include <stdio.h>

int SUM( int n ) {
    if ( n == 1 ) return(1);
    else return(n * n + SUM( n - 1 )); }

int main() {
    printf( "SUM=%d", SUM( 5 ) ); }
```

```
return(0); }
```

【正确答案：SUM=55】

【答案解析】：从 1 到 n 的整数平方和。

7.4 数组作为函数参数

7.4.1 用数组元素作函数实参

由于实参可以是表达式，而数组元素可以是表达式的组成部分，因此数组元素可以作为函数的实参。与变量作为实参一样，传递方式是**单向值传递**。

例 7.6 有两个运动队 a 和 b，各有 10 个队员，每个队员有一个综合成绩。将两个队的每个队员的成绩按顺序一一对应地逐个比较（即 a 队第 1 个队员与 b 队第 1 个队员比，……）。如果 a 队队员的成绩高于 b 队相应队员成绩的数目多于 b 队队员成绩高于 a 队相应队员成绩的数目（例如，a 队赢 6 次，b 队赢 4 次），则认为 a 队胜。统计出两队队员比较的结果（a 队高于、等于和低于 b 队的次数）。

➤ 解题思路：

设两个数组 a 和 b，各有 10 个元素，分别存放 10 个队员的成绩。

将两个数组的相应元素逐个比较，用 3 个变量 n，m，k 分别累计 a 队队员高于、等于和低于 b 队队员的次数。

用一个函数 higher 来判断每一次比较的结果，如果 a 队员高于 b 队员，结果为 1，二者相等，结果为 0，a 队员低于 b 队员，结果为 -1。最后比较 n 和 k 即可得到哪队胜的结果。

```
#include <stdio.h>
```

```
void main()
```

```
{ int higher(int x,int y);
```

```
    int a[10],b[10],i,n=0,m=0,k=0;
```

```
    printf("enter array a:\n");
```

输入a队队员成绩

```
    for(i=0;i<10;i++) scanf("%d",&a[i]);
```

```
    printf("\n");
```

```
    printf( "enter array b:\n" );
```

输入b队队员成绩

```
    for(i=0;i<10;i++) scanf("%d",&b[i]);
```

```
    printf("\n");
```

```
for(i=0;i<10;i++)    比较10个队员
{ if(higher(a[i],b[i])==1)
    n++; 与变量作为实参一样
    else
        if(higher(a[i],b[i])==0) m++;
        else k=k+1;
}
printf("a higher b %d times\n",n);
printf("a equal to b %d times\n",m);
printf("b higher a %d times\n",k);

if(n>k)

    printf("'a wins!\n");

    else

        if (n<k)

            printf("'b wins!\n");

        else

            printf("'a is equal to b\n");

}
```

```
higher(int x,int y)
```

```
{ int flag;

    if(x>y) flag=1;

    else if(x<y) flag=-1;

    else flag=0;

    return(flag);

}
```

7.4.2 用数组名作函数参数

希望在函数中处理整个数组的元素时，可以用数组名作为函数实参。

注意，此时只是将数组的首元素的地址传递给所对应的形参，因此对应的形参应当是数组名或者指针变量（见第8章）。

例 7.7 有 10 个学生成绩，用一个函数求全体学生的平均成绩。

➤ 解题思路：

在主函数中定义一个实型数组 `score`，将输入的 10 个学生成绩存放在数组中。

设计函数 `average`，用来求学生平均成绩。

需要把数组有关信息传递给 `average` 函数。

采取用数组名作为实参，把数组地址传给 `average` 函数，在该函数中对数组进行处理。

```
#include <stdio.h>

void main()
{
    float average(float array[10]);

    float score[10], aver;

    int i;

    printf("input 10 scores:\n");

    for(i=0; i<10; i++)

        scanf("%f", &score[i]);

    aver=average(score);           数组名作实参

    printf("average score is %5.2f\n", aver);
}

    实参、形参都是float型
float average(float array[10])
{
    int i;           与score共占同一存储单元
    float aver, sum=array[0];
    for(i=1; i<10; i++)
        sum=sum+array[i];
    aver=sum/10;
    return(aver);
}
```

相当于score[0]

相当于score[i]

例 7.8 有两个班，学生数不同，编写一个函数，用来分别求各班的平均成绩。

➤ 解题思路：

问题的关键是用同一个函数求不同人数的班级平均成绩。

在定义形参时不指定大小，函数对不同人数的班级都是适用。

由于数组名传递的是数组首地址，可以利用同一个函数求人数不同的班平均成绩。

在定义 average 函数时，增加一个参数 n，用来指定当前班级的人数。

```
#include <stdio.h>
```

```
void main()
```

```
{ float average(float array[ ],int n);
```

```
    float score_1[5]={98.5,97,91.5,60,55};
```

```
    float score_2[10]={67.5,89.5,99,69.5,                          77,89.5,76.5,54,60,99.5};
```

```
    printf( "%6.2f\n" ,average(score_1,5));
```

```
    printf( "%6.2f\n" ,average(score_2,10));
```

```
}
```

调用形式为average(score_1,5)时

```
float average(float array[ ],int n)
```

```
{ int i;
```

n是5

```
    float aver,sum=array[0];
```

```
    for(i=1;i<n;i++)
```

相当于score_1[0]

```
        sum=sum+array[i];
```

```
    aver=sum/n;
```

相当于score_1[i]

```
    return(aver);
```

```
}
```

调用形式为average(score_2,10)时

```
float average(float array[ ],int n)
```

```
{ int i;
```

n是10

```
    float aver,sum=array[0];
```

```
    for(i=1;i<n;i++)
```

相当于score_2[0]

```
        sum=sum+array[i];
```

```
    aver=sum/n;
```

相当于score_2[i]

```
    return(aver);
```

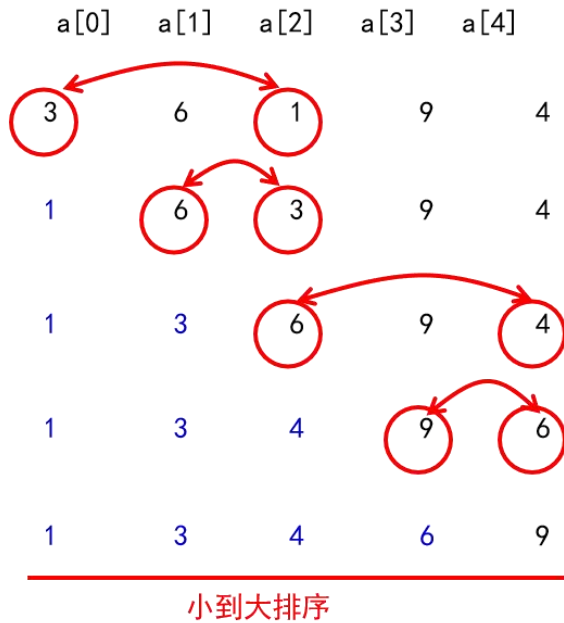
```
}
```

例 7.9 用一个函数实现用选择法对 10 个整数按升序排列。

➤ 解题思路：

所谓**选择法**就是先将 10 个数中最小的数与 a[0]对换；再将 a[1]到 a[9]中最小的数与 a[1]对换……每比较一轮，找出一个未经排序的数中最小的一个。

共比较 9 轮。



```
#include <stdio.h>
```

```
void main()
```

```
{ void sort(int array[],int n);
```

```
    int a[10],i;
```

```
    printf("enter the array:\n");
```

```
    for(i=0;i<10;i++) scanf("%d",&a[i]);
```

```
    sort(a,10);
```

```
    printf("The sorted array:\n");
```

```
    for(i=0;i<10;i++) printf("%d ",a[i]);
```

```
    printf("\n");
```

```
}
```

```
void sort(int array[],int n)
```

```
{ int i,j,k,t;
```

```
    for(i=0;i<n-1;i++)
```

```
    { k=i;
```

在sort[i]~sort[9]中，
找最小数下标

```
        for(j=i+1;j<n;j++)
```

```
            if(array[j]<array[k]) k=j;
```

```
        t=array[k];
```

```
        array[k]=array[i];
```

```
        array[i]=t;
```

```
    }
```

在sort[i]~sort[9]中，最
小数与sort[i]对换

```
}
```

例 7.10 有 4 个学生，5 门课的成绩，设计一个函数，用来求出其中的最高成绩。

➤ 解题思路：

先使变量 `max` 的初值为二维数组中第一个元素的值，然后将二维数组中各个元素的值与 `max` 相比，每次比较后都把“大者”存放在 `max` 中，取代 `max` 的原值。全部元素比较完后，`max` 的值就是所有元素的最大值。

```
#include <stdio.h>

void main()
{
    float highest_score(float array[4][5]);

    float score[4][5]={ {61,73,85.5,87,90},
                        {72,84,66,88,78},
                        {75,87,93.5,81,96},
                        {65,85,64,76,71}
                      };

    printf("%6.2f\n",highest_score(score));
}

float highest_score(float array[4][5])
{
    int i,j;

    float max;

    max=array[0][0];

    for(i=0;i<4;i++)
        for(j=0;j<5;j++)
            if(array[i][j]>max)
                max=array[i][j];

    return (max);
}
```


7.5 变量的作用域和生存期

7.5.1 变量的作用域——局部变量和全局变量

1. 局部变量

在函数和复合语句内定义的变量，称为**内部变量或局部变量**。

只在本函数或复合语句内范围内有效（从定义点开始到函数或复合语句结束）。

在此函数或复合语句以外是不能使用这些变量的。

➤ 说明：

- （1）主函数中定义的变量也只在主函数中有效，主函数也不能使用其他函数中定义的变量。
- （2）不同函数中可以使用相同名字的变量，它们代表不同的对象，互不干扰。
- （3）形式参数也是局部变量。在函数中可以使用本函数定义的形参，在函数外不能引用它。
- （4）在一个函数内部，可以在复合语句中定义变量，这些变量只在本复合语句中有效。

2. 全局变量

一个程序可以包含一个或若干个源程序文件（即程序模块），而一个源文件可以包含一个或若干个函数。

在函数之外定义的变量是外部变量，也称为**全局变量（或全程变量）**。

全局变量的有效范围为从定义变量的位置开始到本源文件结束，在此范围内可以为本文件中所有函数所共用。

在一个函数中既可以使用本函数中的局部变量，又可以使用有效的全局变量。

如果在同一个源文件中，外部变量与局部变量同名，则在局部变量的作用范围内，**外部变量被“屏蔽”了**，即它不起作用，此时局部变量是有效的。

例 7.11 有 4 个学生，5 门课的成绩，要求输出其中的最高成绩以及它属于第几个学生、第几门课程。

➤ 解题思路：

在例 7.10 中，通过调用 `highest_score` 函数，得到最高分。

除了输出最高分以外，还要输出该分数是属于第几个学生、第几门课的信息，即需要输出 3 个结果。

调用一个函数只能得到一个函数值，因此例 7.10 程序无法解决这个问题。

可以使用全局变量，通过全局变量从函数中得到所需要的值。

```
#include <stdio.h>
```

```
int Row,Column; 定义全局变量
```

```
void main()
```

```
{ float highest_score(float array[4][5]);

    float score[4][5]={ {61,73,85.5,87,90},
        {72,84,66,88,78},{75,87,93.5,81,96},
{65,85,64,76,71}}};

    printf("he highest score is %6.2f\n",
highest_score(score));

    printf("Student No.is %d\n
        Course No. is %d\n",Row,Column);

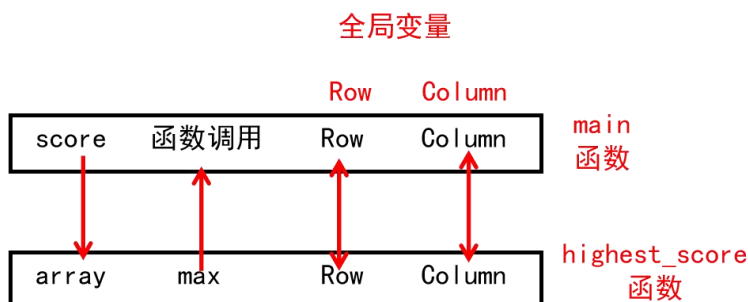
}
```

```
float highest_score(float array[4][5])
{ int i,j; float max;

    max=array[0][0];
    for(i=0;i<4;i++)
        for(j=0;j<5;j++)
            if(array[i][j]>max)
                { max=array[i][j];
                    Row=i;
                    Column=j;
                }

    return (max);
}
```

行的序号赋给全局变量Row
 将列的序号赋给全局变量Column



建议不在必要时不要使用全局变量

7.5.2 变量的存储方式和生存期

变量的生存期：变量值存在的时间

变量的两种存储方式：静态存储方式和动态存储方式

静态存储方式是指在程序运行期间由系统分配固定的存储空间的方式。

动态存储方式是在程序运行期间根据需要进行动态的分配存储空间的方式。

全局变量采用静态存储方式，在程序开始执行时给全局变量分配存储区，程序执行完毕释放。在程序执行过程中它们占据固定的存储单元，而不是动态地进行分配和释放。

在函数中定义的变量，在函数调用开始时分配动态存储空间，函数结束时释放这些空间。在程序执行过程中，这种分配和释放是动态的。

每一个变量和函数都有两个属性：数据类型和数据的存储类别

数据类型，如整型、浮点型等

存储类别指的是数据在内存中存储的方式（如静态存储和动态存储）

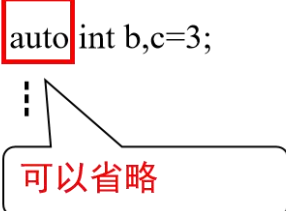
1.auto—声明自动变量（auto 变量）

函数中的形参和在函数中定义的变量（包括在复合语句中定义的变量），都属于此类。

在调用该函数时，系统给这些变量分配存储空间，在函数调用结束时就自动释放这些存储空间。因此这类局部变量称为自动变量。

自动变量用关键字 auto 作存储类别的声明。

```
int f(int a)
{
    auto int b,c=3;
    ...
}
```



2.static—声明静态变量

以下情况需要指定 static 存储类别：

希望函数中的局部变量值在函数调用结束后不消失而继续保留原值，即其占用的存储单元不释放，在下一

次该函数调用时，该变量已有值，就是上一次函数调用结束时的值。这时就应用关键字 `static` 指定该局部变量为“静态局部变量”。

例 7.12 输出 1 到 5 的阶乘值。

➤ 解题思路：

可以编一个函数用来进行一次累乘

如第 1 次调用时进行 1 乘 1

第 2 次调用时再乘以 2

第 3 次调用时再乘以 3

依此规律进行下去

这时希望上一次求出的连乘值保留，以便下一次再乘上一个数。可以用 `static`。

```
#include <stdio.h>
```

```
void main()
```

```
{ int fac(int n);
```

```
    int i;
```

```
    for(i=1;i<=5;i++)
```

```
        printf( "%d!=%d\n" ,i,fac(i));
```

```
}
```

```
int fac(int n)
```

```
{ static int f=1;
```

```
    f=f*n;
```

```
    return(f);
```

```
}
```

调用5次，每调用一次，
开辟新n，但f不是

```
#include <stdio.h>
```

```
void main()
```

```
{ int fac(int n);
```

```
    int i;
```

```
    for(i=1;i<=5;i++)
```

```
        printf( "%d!=%d\n" ,i,fac(i));
```

```
}
```

```
int fac(int n)
```

i=1：第一次调用

f72

1

```
{static int f=1;  
  
    f=f*n;  
  
    return(f);  
  
}
```

```
#include <stdio.h>
```

```
void main()
```

```
{ int fac(int n);  
  
    int i;  
  
    for(i=1;i<=5;i++)  
  
        printf( "%d!=%d\n" ,i,fac(i));  
  
}
```

```
int fac(int n)  
{static int f=1;  
  
    f=f*n;  
  
    return(f);  
  
}
```

i=2: 第二次调用

f

2

```
#include <stdio.h>
```

```
void main()
```

```
{ int fac(int n);  
  
    int i;  
  
    for(i=1;i<=5;i++)  
  
        printf( "%d!=%d\n" ,i,fac(i));  
  
}
```

```
int fac(int n)  
{static int f=1;  
  
    f=f*n;  
  
    return(f);  
  
}
```

i=3: 第三次调用

f

6

```
#include <stdio.h>

void main()
{ int fac(int n);

  int i;

  for(i=1;i<=5;i++)

    printf( "%d!=%d\n" ,i,fac(i));

}

int fac(int n)
{ static int f=1;

  f=f*n;

  return(f);

}
```

i=4: 第四次调用

f

24

```
#include <stdio.h>

void main()
{ int fac(int n);

  int i;

  for(i=1;i<=5;i++)

    printf( "%d!=%d\n" ,i,fac(i));

}

int fac(int n)
{ static int f=1;

  f=f*n;

  return(f);

}
```

i=5: 第五次调用

f

120

对静态局部变量的说明：

(1) 静态局部变量属于静态存储类别，在静态存储区内分配存储单元。在程序整个运行期间都不释放。而自动变量（即动态局部变量）属于动态存储类别，占动态存储区空间而不占静态存储区空间，函数调用结束后即释放。

(2) 对静态局部变量是在编译时赋初值的，即只赋初值一次，在程序运行时它已有初值。以后每次调用

函数时不再重新赋初值而只是保留上次函数调用结束时的值。而对自动变量赋初值，不是在编译时进行的，而是在函数调用时进行，每调用一次函数重新给一次初值，相当于执行一次赋值语句。

(3) 如在定义局部变量时不赋初值的话，则对静态局部变量来说，编译时自动赋初值 0（对数值型变量）或空字符（对字符变量）。而对自动变量来说，如果不赋初值则它的值是一个不确定的值。这是由于每次函数调用结束后存储单元已释放，下次调用时又重新另分配存储单元，而所分配的单元中的值是不可知的。

(4) 虽然静态局部变量在函数调用结束后仍然存在，但其他函数是不能引用它的。因为它是局部变量，只能被本函数引用，而不能被其他函数引用。

(5) 用静态存储要多占内存（长期占用不释放，而不能像动态存储那样一个存储单元可供多个变量使用，节约内存），而且降低了程序的可读性，当调用次数多时往往弄不清静态局部变量的当前值是什么。因此，若非必要，不要多用静态局部变量。

3.register—声明寄存器变量

一般情况下，变量（包括静态存储方式和动态存储方式）的值是存放在内存中的。

寄存器变量允许将局部变量的值放在 CPU 中的寄存器中。

现在的计算机能够识别使用频繁的变量，从而自动地将这些变量放在寄存器中，而不需要程序设计者指定。

4.extern—声明外部变量的作用范围

(1) 在一个文件内扩展外部变量的作用域

如果外部变量不在文件的开头定义，其有效的作用范围只限于定义处到文件结束。

如果由于某种考虑，在定义点之前的函数需要引用该外部变量，则应该在引用之前用关键字 **extern** 对该变量作“外部变量声明”。

(2) 将外部变量的作用域扩展到其他文件

在任一个文件中定义外部变量，而在另一文件中用 **extern** 对该变量作外部变量声明。

7.5.3 关于作用域和生存期的小结

对一个变量的属性可以从两个方面分析，一是变量的作用域，一是变量的生存期。

前者是从空间的角度，后者是从时间的角度。

二者有联系但不是同一回事。

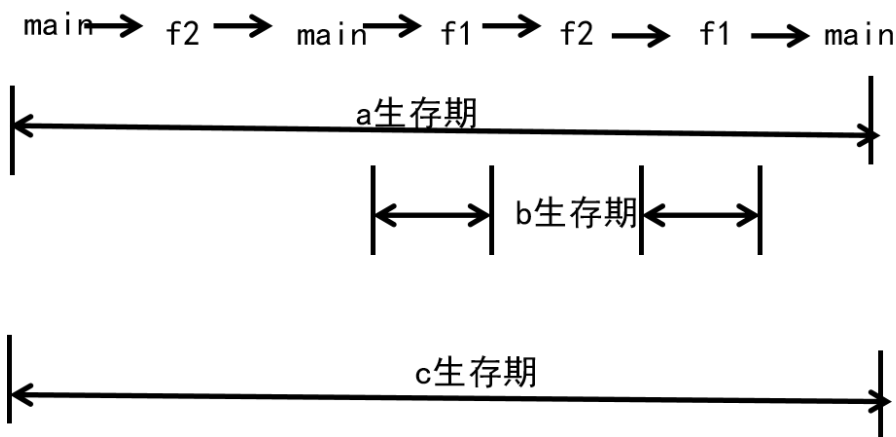
文件file1.c

```
int a;
void main( )
{ ...f2();...f1();... }
void f1( )
{ auto int b;
  ... f2();
  ...
}
void f2( )
{ static int c;
  .....
}
```

b的作用域

a的作用域

c的作用域



各种类型变量的作用域和存在性的情况

变量存储类别	函数内		函数外	
	作用域	存在性	作用域	存在性
自动变量和寄存器变量	√	√	×	×
静态局部变量	√	√	×	√
静态外部变量	√	√	√ (只限本文件)	√
外部变量	√	√	√	√

讲解归纳与举例

1.若变量定义时未初始化，则其值不确定的是（ ）。

- A.静态全局变量
- B.局部变量
- C.静态局部变量
- D.静态变全局变量

【正确答案：B】

【答案解析】：未初始化的静态变量会在程序运行前进行填充 0。

2.若函数中局部变量的值经函数调用后仍然保留，则该局部变量定义为（ ）。

- A.自动变量
- B.内部变量
- C.外部变量
- D.静态变量

【正确答案：D】

【答案解析】：局部变量可以定义成 static 变量，使用静态存储机制。

3.在某源程序文件中，若全局变量与局部变量同名，则（ ）。

- A.视为同一个变量
- B.变量作用域不确定
- C.不允许
- D.允许

【正确答案：D】

【答案解析】：在不同区域中允许定义相同名字的变量，但本质上它们是不同的变量。如果一个变量所处区域的子区域中有同名的变量，则该变量在子区域无效，有效的是子区域的变量，称为定义屏蔽。

7.6 内部函数和外部函数

7.6.1 什么是内部函数

如果一个函数只能被本文件中其他函数所调用，它称为**内部函数**。

在定义内部函数时，在函数名和函数类型的前面加 `static`，即：

`static` 类型标识符 函数名（形参表）

内部函数又称静态函数，因为它用 `static` 声明的。

通常把只能由本文件使用的函数和外部变量放在文件的开头，前面都冠以 `static` 使之局部化，其他文件不能引用。

提高了程序的可靠性。

7.6.2 什么是外部函数

如果在定义函数时，在函数首部的最左端加关键字 `extern`，则此函数是外部函数，可供其他文件调用。

如函数首部可以为

`extern int fun (int a, int b)`

如果在定义函数时省略 `extern`，则默认为外部函数。

例 7.13 有一个字符串，内有若干个字符，今输入一个字符，要求程序将字符串中该字符删去。用外部函数实现。

假设删除空格，删除空格的思路为

l		a	m		h	a	p	p	y	\0	...
---	--	---	---	--	---	---	---	---	---	----	-----

l	a	m	h	a	p	p	y	\0
---	---	---	---	---	---	---	---	----

```
#include <stdio.h>
void main()
{ extern void enter_string(char str[]);
  extern void delete_string(char str[],
    char ch);
  extern void print_string(char str[]);
  char c,str[80];
  enter_string(str);
  scanf("%c",&c);
  delete_string(str,c);
  print_string(str);
}
```

声明在本函数中将要调用的已在其他文件中定义的3个函数

file2（文件2）

```
void enter_string(char str[80])
{ gets(str); }
```

file3（文件3）

```
void delete_string(char str[],char ch)
{ int i,j;
  for(i=j=0;str[i]!='\0';i++)
    if(str[i]!=ch) str[j++]=str[i];
  str[j]='\0';
}
```

file4（文件4）

```
void print_string(char str[])
{ printf("%s\n",str); }
```

讲解归纳与举例

1.如果在一个源文件中定义的函数，只能被本文件中的函数调用，而不能被同一程序其它文件中的函数调用，则说明这个函数为（ ）。

- A.私有函数
- B.内部函数
- C.外部函数

D.库函数

【正确答案：B】

【答案解析】：在函数定义前加上 `static` 修饰，则函数称为内部函数。内部函数仅在包含它的文件中有效。

2.判断题：C 程序中，有调用关系的所有函数必须放在同一个源程序文件中。

【正确答案：错误】

【答案解析】：在调用另一个文件中的函数时，需要用 `extern` 声明此函数是外部函数。

7.7 提高部分

7.7.1 实参求值的顺序

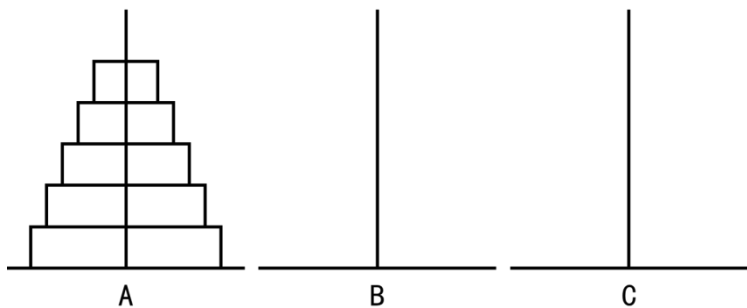
如果实参表列包括多个实参，对实参求值的顺序并不是确定的

有的系统按自左至右顺序求实参的值

有的系统（如 Turbo C 2.0, Turbo C++ 3.0, VC++ 6.0）则按自右至左顺序求值。

7.7.2 递归的典型例子--Hanoi（汉诺）塔问题

例 7.8 Hanoi（汉诺）塔问题。古代有一个梵塔，塔内有 3 个柱子 A、B、C，开始时 A 柱上有 64 个盘子，盘子大小不等，大的在下，小的在上。有一个老和尚想把这 64 个盘子从 A 柱移到 C 柱，但规定每次只允许移动一个盘，且在移动过程中在 3 个柱上都始终保持大盘在下，小盘在上。在移动过程中可以利用 B 柱。要求编程输出移动一盘子的步骤。



➤ 解题思路：

要把 64 个盘子从 A 柱移动到 C 柱，需要移动大约 2^{64} 次盘子。一般人是不能直接确定移动盘子的每一个具体步骤的。

老和尚会这样想：假如有另外一个和尚能有办法将上面 63 个盘子从一个柱移到另一柱。那么，问题就解决了。此时老和尚只需这样做：

- （1）命令第 2 个和尚将 63 个盘子从 A 柱移到 B 柱。
- （2）自己将 1 个盘子（最底下的、最大的盘子）从 A 柱移到 C 柱。
- （3）再命令第 2 个和尚将 63 个盘子从 B 柱移到 C 柱。

➤ 由上面的分析可知：将 n 个盘子从 A 柱移到 C 柱可以分解为以下 3 个步骤：

- （1）将 A 柱上 $n-1$ 个盘借助 C 柱先移到 B 柱上。
- （2）把 A 柱上剩下的一个盘移到 C 柱上。
- （3）将 $n-1$ 个盘从 B 柱借助于 A 柱移到 C 柱上。

➤ 把上面 3 个步骤分成两类操作：

(1) 将 $n-1$ 个盘从一个柱移到另一个柱上 ($n>1$)。这就是大和尚让小和尚做的工作，它是一个递归的过程，即和尚将任务层层下放，直到第 64 个和尚为止。

(2) 将 1 个盘子从一个座上移到另一柱上。这是大和尚自己做的工作。

➤ 编写程序

用 hanoi 函数实现第 1 类操作（即模拟小和尚的任务）。

用 move 函数实现第 2 类操作（模拟大和尚自己移盘）。

函数调用 hanoi (n,x,y,z) 表示将 n 个盘子从 x 柱移到 z 柱的过程（借助 y 柱）。

函数调用 move (a,b) 表示将 1 个盘子从 a 柱移到 b 柱的过程。 a 和 b 是代表 A、B、C 柱之一，根据每次不同情况分别取 A、B、C 代入。

```
#include <stdio.h>
```

```
void main()
```

```
{ void hanoi(int n,char x,  
                char y,char z);
```

```
    int m;
```

```
    printf( "the number of dishes:");
```

```
    scanf("%d",&m);
```

```
    printf("move %d dishes:\n",m);
```

```
    hanoi(m,'A','B','C');
```

```
}
```

```
void hanoi(int n,char x,char y, char z)
```

```
{ void move(char a,char b);
```

```
    if(n==1)
```

```
        move(x,z);
```

```
    else
```

```
    { hanoi(n-1,x,z,y);
```

```
        move(x,z);
```

```
        hanoi(n-1,y,x,z);
```

```
    }
```

```
}
```

```
void move(char a,char b)
```

```
{
```

```
printf("%c-->%c\n",a,b);  
}
```

the number of dishes:3 ✓

moving 3 dishes:

A-->C

A-->B

C-->B

A-->C

B-->A

B-->C

A-->C

本章小结

- (1) 为什么要使用函数。
- (2) 两种函数：库函数和自定义函数。
- (3) 函数的定义、声明、调用、函数原型。
- (4) 函数的嵌套调用。
- (5) 函数的递归调用。
- (6) 数组作为函数参数。
- (7) 变量的作用域、生存周期。
- (8) 内部函数和外部函数。
- (9) 应用函数解决实际问题，提升编程能力。

第8章 善于使用指针

本章重难点分析

本章主要介绍什么是指针，为什么要使用指针；什么是指针变量，如何定义引用指针变量以及用指针变量作为函数参数解决问题；通过指针引用数组；通过指针引用字符串；指向多维数组的指针；指向函数的指针；返回指针值的函数；指针数组；多重指针等等。

本章内容注意以下几点：

指针的概念	重点
指针变量	难点
指针引用数组	重点
指针引用字符串	难点

8.1 什么是指针

在定义变量时，系统就给这个变量分配内存单元。

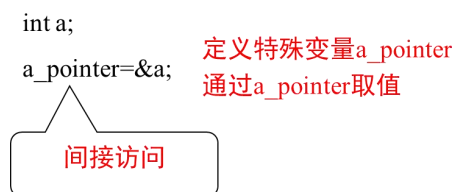
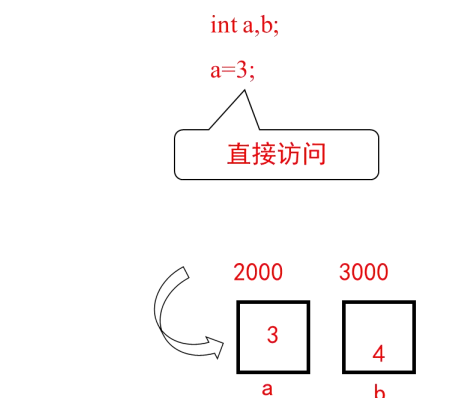
编译系统根据程序中定义的变量类型，分配一定长度的空间。

内存区的每一个字节有一个编号，这就是内存单元的“地址”。

在地址所标志的内存单元中存放数据。

由于通过地址能找到所需的变量单元，我们可以说，地址指向该变量单元。

在 C 语言中，将地址形象化地称为“指针”。意思是通过它能找到以它为地址的内存单元。



➤ 为了表示将数值 3 送到变量中，可以有两种表达方法：

- (1) 将 3 直接送到变量 a 所代表的单元中。
- (2) 将 3 送到变量 a_pointer 所指向的单元（即 a 所代表的存储单元）。

➤ 指向就是通过地址来体现的

假设 a_pointer 中的值是变量 a 的地址（2000），这样就在 a_pointer 和变量 a 之间建立起一种联系，即通过 a_pointer 能知道 a 的地址，从而找到变量 a 的内存单元。

➤ 一个变量的地址称为该变量的“指针”

例如，地址 2000 是变量 a 的指针。

- 如果有一个变量专门用来存放另一变量的地址（即指针），则它称为“**指针变量**”。
- a_pointer 就是一个指针变量。**指针变量就是地址变量，用来存放地址的变量，指针变量的值是地址（即指针）。**
- 指针”和“指针变量”是不同的概念。
- 可以说变量 a 的指针是 2000，而不能说 a 的指针变量是 2000。
- **指针是一个地址，而指针变量是存放地址的变量。**

讲解归纳与举例

1.在 C 程序中，指针变量能够赋___值或___值。

【正确答案：第 1 空：地址，第 2 空：0[或者]NULL】

【答案解析】：指针变量的理解。

2.判断题：指针变量就是存放变量的地址的变量。

【正确答案：对】

【答案解析】：指针变量的理解。

8.2 指针变量

8.2.1 使用指针变量访问变量的例子

例 8.1 通过指针变量访问整型变量。

```
#include <stdio.h>
void main()
{ int a,b;
  int *pointer_1,*pointer_2;
  a=100;b=10;
  pointer_1=&a;
  pointer_2=&b;
  printf("a=%d,b=%d\n",a,b);
  printf("*pointer_1=%d,*pointer_2=
    %d\n",*pointer_1,*pointer_2);
}
```

定义两个指针变量

使pointer_1指向a

直接输出变量a和b的值

借助指针间接输出变量a和b的值

```
#include <stdio.h>
void main()
{ int a,b;
  int*pointer_1,*pointer_2;
  a=100;b=10;
  pointer_1=&a;
  pointer_2=&b;
  printf("a=%d,b=%d\n",a,b);
  printf("*pointer_1=%d,*pointer_2=
    %d\n",*pointer_1,*pointer_2);
}
```

此处*与类型名在一起。此时共同定义指针变量。

此处*与指针变量一起使用。此时代表指针变量所指向的变量。

8.2.2 怎样定义指针变量

定义指针变量的一般形式为：

基类型 * 指针变量名；

如：int *pointer_1, *pointer_2;

int 是为指针变量指定的“基类型”。

基类型指定指针变量可指向的变量类型。

如 pointer_1 可以指向整型变量，但不能指向浮点型变量。

➤ 下面都是合法的定义和初始化：

```
float *pointer_3;
```

```
char *pointer_4;
```

```
int a,b;
```

```
int *pointer_1=&a,*pointer_2=&b;
```

pointer_3=2000; 错误

pointer_3=&a; 错误

*pointer_1=&a; 错误

pointer_1=&a; 正确

8.2.3 怎样引用指针变量

➤ 在引用指针变量时，可能有三种情况：

◆ 给指针变量赋值。如：p=&a; 使p指向a

◆ 引用指针变量指向的变量。如有

```
p=&a; *p=1;
```

*p相当于a

则执行printf("%d",*p); //将输出1

◆ 引用指针变量的值。如：printf("%o",p);

以八进制输出a的地址

➤ 要熟练掌握两个有关的运算符：

(1) & 取地址运算符。

&a 是变量 a 的地址

(2) * 指针运算符（“间接访问”运算符）

如果：p 指向变量 a，则*p 就代表 a。

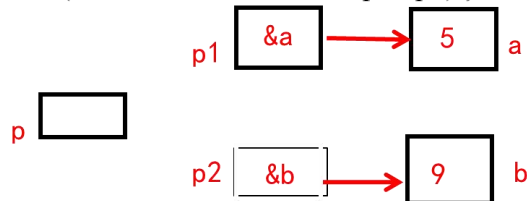
k=*p; (把 a 的值赋给 k)

*p=1; (把 1 赋给 a)

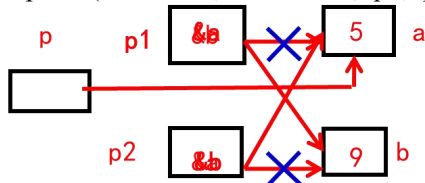
例 8.2 输入 a 和 b 两个整数，按先大后小的顺序输出 a 和 b。

➤ 解题思路：用指针方法来处理这个问题。不交换整型变量的值，而是交换两个指针变量的值。

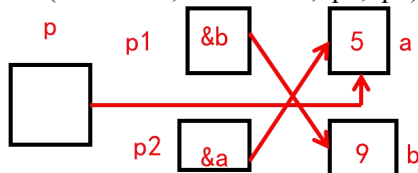
```
#include <stdio.h>
void main()
{ int *p1,*p2,*p,a,b;
  scanf("%d,%d",&a,&b);
  p1=&a; p2=&b;
  if(a<b)
  { p=p1; p1=p2; p2=p; }
  printf("a=%d,b=%d\n",a,b);
  printf("max=%d,min=%d\n",*p1,*p2);}
```



```
#include <stdio.h>
void main()
{ int *p1,*p2,*p,a,b;
  scanf("%d,%d",&a,&b);
  p1=&a; p2=&b;
  if(a<b)
  { p=p1; p1=p2; p2=p; }
  printf("a=%d,b=%d\n",a,b);
  printf("max=%d,min=%d\n",*p1,*p2);}
```



```
#include <stdio.h>
void main()
{ int *p1,*p2,*p,a,b;
  scanf("%d,%d",&a,&b);
  p1=&a; p2=&b;
  if(a<b)
  { p=p1; p1=p2; p2=p; }
  printf("a=%d,b=%d\n",a,b);
  printf("max=%d,min=%d\n",*p1,*p2);}
```



```
#include <stdio.h>
```

```
void main()
```

```
{ int *p1,*p2,*p,a,b;
```

```
  scanf( "%d,%d" ,&a,&b);
```

```
  p1=&a;    p2=&b;
```

```
if(a<b)
{
    p=p1; p1=p2; p2=p;
}
printf( "a=%d,b=%d\n" ,a,b);
printf( "max=%d,min=%d\n" ,*p1,*p2);
}
```

可否改为 `p1=&b; p2=&a;?`

➤ 注意：

a 和 b 的值并未交换，它们仍保持原值。

但 p1 和 p2 的值改变了。p1 的值原为 &a，后来变成 &b，p2 原值为 &b，后来变成 &a。

这样在输出 *p1 和 *p2 时，实际上是输出变量 b 和 a 的值，所以先输出 9，然后输出 5。

8.2.4 指针变量作为函数参数

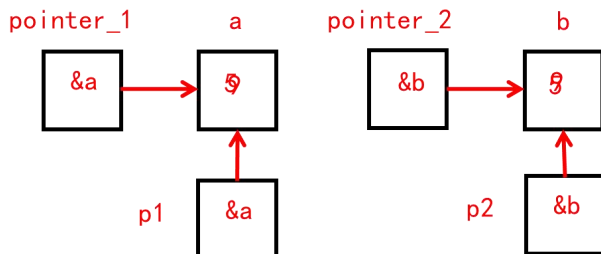
例 8.3 题目要求同例 8.2，即对输入的两个整数按大小顺序输出。要求用函数处理，用指针变量作函数参数。

解题思路：将指向两个变量的指针变量（内放两个变量的地址）作为实参传递给形参的指针变量，在形参中通过指针交换两个变量的值。

```
#include <stdio.h>
void main()
{void swap(int *p1,int *p2);
 int a,b; int*pointer_1,*pointer_2;
 scanf("%d,%d",&a,&b); 5,9
 pointer_1=&a;
 pointer_2=&b;
 if (a<b) swap(pointer_1,pointer_2);
 printf("max=%d,min=%d\n",a,b);}
pointer_1  a  pointer_2  b
```



```
void swap(int *p1,int *p2)
{ int temp;
  temp=*p1; //temp的值为5
  *p1=*p2;
  *p2=temp;
}
```



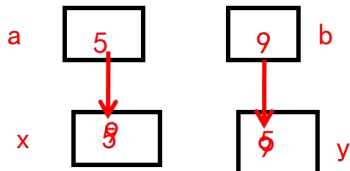
```
void swap(int *p1,int *p2)
{ int temp;
  temp=*p1;
  *p1=*p2;
  *p2=temp;
}
```

错!!!
无确定的指向

```
void swap(int *p1,int *p2)
{ int *temp;
  *temp=*p1;
  *p1=*p2;
  *p2=*temp;
}
```

```
#include <stdio.h>
void main()
{.....
  if (a<b) swap(a,b);
  printf("max=%d,min=%d\n",a,b);
}
void swap(int x,int y)
{ int temp;
  temp=x; x=y; y=temp;
}
```

错!!!
无法交换a, b



➤ 如果想通过函数调用得到 n 个要改变的变量：

- ①在主调函数中设 n 个变量，用 n 个指针变量指向它们。
- ②设计一个函数，有 n 个指针形参。在这个函数中改变这 n 个形参的值。
- ③在主调函数中调用这个函数，在调用时将这 n 个指针变量作实参，将它们的地址传给该函数的形参。

- ④在执行该函数的过程中，通过形参指针变量，改变它们所指向的 n 个变量的值。
- ⑤主调函数中就可以使用这些改变了值的变量。

例 8.4 企图通过改变指针形参的值来改变指针实参的值。

不能实现！

```
#include <stdio.h>
void main()
{
    .....
    pointer_1=&a; pointer_2=&b;
    if(a<b)
        swap(pointer_1,pointer_2);
    ..... 调用结束后指向不变
}
void swap(int *p1,int *p2)
{
    int *p;
    p=p1; p1=p2; p2=p;
}
```

错!!!
只交换形参
指向

例 8.5 输入 3 个整数 a , b , c ，要求按大小顺序将它们输出。用函数实现改变这 3 个变量的值。

解题思路：采用例 8.3 的方法在函数中交换两个变量的值。

```
#include <stdio.h>
void main()
{
    void exchange(int *q1, int *q2, int *q3);
    int a,b,c,*p1,*p2,*p3;
    printf("please enter three numbers:");
    scanf("%d,%d,%d",&a,&b,&c);
    p1=&a;p2=&b;p3=&c;
    exchange(p1,p2,p3);
    printf("%d,%d,%d\n",a,b,c);
}
```

9, 0, 10

调用结束后不会
改变指针的指向


```
void exchange(int *q1, int *q2, int *q3)
{void swap(int *pt1, int *pt2);
 if(*q1<*q2) swap(q1,q2);
 if(*q1<*q3) swap(q1,q3);
 if(*q2<*q3) swap(q2,q3);
}
void swap(int *pt1, int *pt2)
{ int temp;
 temp=*pt1; *pt1=*pt2; *pt2=temp;
}
```

交换指针指向的变量值

9, 0, 10 ✓
10, 9, 0

讲解归纳与举例

1. 写出下面程序执行后的运行结果___。

```
#include <stdio.h>
```

```
int c;
```

```
func( int *a, int b ) {
    c = (*a) * b;
    *a = b - 1;
    b++;
    return(*a + b + 1); }
```

```
int main() {
    int a = 4, b = 2, p = 0;
    p = func( &b, a );
    printf( "%d,%d,%d,%d", a, b, c, p );
    return(0); }
```

【正确答案： 4, 3, 8, 9 】

【答案解析】：调用函数 func 时，传入了变量 b 的地址，因此在 func 函数中通过*a 修改了 main 中 b 的值。

8.3 通过指针引用数组

8.3.1 数组元素的指针

一个变量有地址，一个数组包含若干元素，每个数组元素都有相应的地址。

指针变量可以指向数组元素（把某一元素的地址放到一个指针变量中）。

所谓数组元素的指针就是数组元素的地址。

➤ 可以用一个指针变量指向一个数组元素

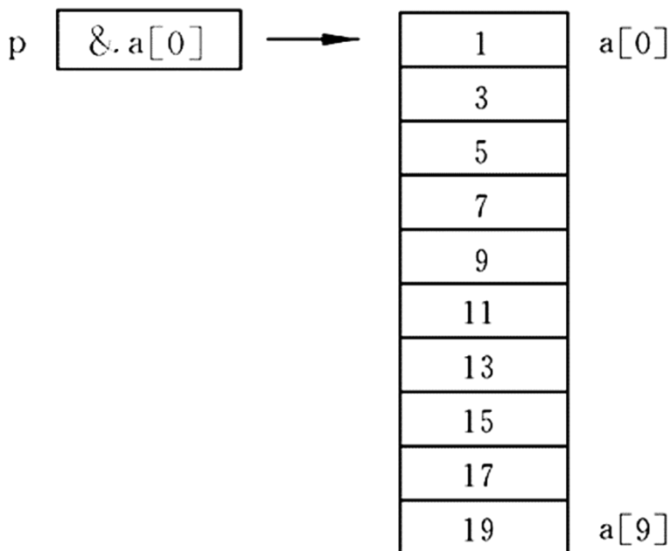
```
int a[10]={1,3,5,7,9,11,13,15,17,19};
```

```
int *p;  
p=&a[0]; //等价于p=a;
```

等价于 `int *p=a;`
或 `int *p=&a[0];`

注意：数组名a不代表整个数组，只代表数组首元素的地址。

“`p=a;`”的作用是“把a数组的首元素的地址赋给指针变量p”，而不是“把数组a各元素的值赋给p”。



8.3.2 通过指针引用数组元素

引用数组元素，可用下面两种方法：

(1) 下标法，用数组名加下标 如 `a[i]`

(2) 指针法，即地址法。

`*(a+i)` 或 `*(p+i)`

（其中初值 `p=a`）

例 8.6 有一个数组存放 10 个学生的年龄，用不同的方法输出数组中的全部元素。

➤ 解题思路：设整型数组 `a[10]`，可以用下面方法实现输出全部学生的年龄。

①用数组名加下标。

②通过数组名计算数组元素地址，找到元素。

③通过指针变量计算数组元素地址，找到元素。

④用指针变量先后指向各数组元素。

```
#include <stdio.h>

void main()

{int a[10]={19,17,20,18,16,22,24,15,23,25};

    int i,*p=a;

    for(i=0;i<10;i++) printf( "%d  ",a[i]);

    printf( "\n");

    for(i=0;i<10;i++) printf( "%d  ",*(a+i));

    printf( "\n");

    for(i=0;i<10;i++)printf( "%d  ",*(p+i));

    printf( "\n");

    for(p;p<(a+10);p++)printf( "%d  ",*p);

    printf( "\n"); }
```

➤ 指针运算：通过指针的运算，可以方便地引用数组中的元素。

（1）如果指针变量 `p` 已指向数组中的一个元素，则 `p+1` 指向同一数组中的下一个元素，`p-1` 指向同一数组中的上一个元素。

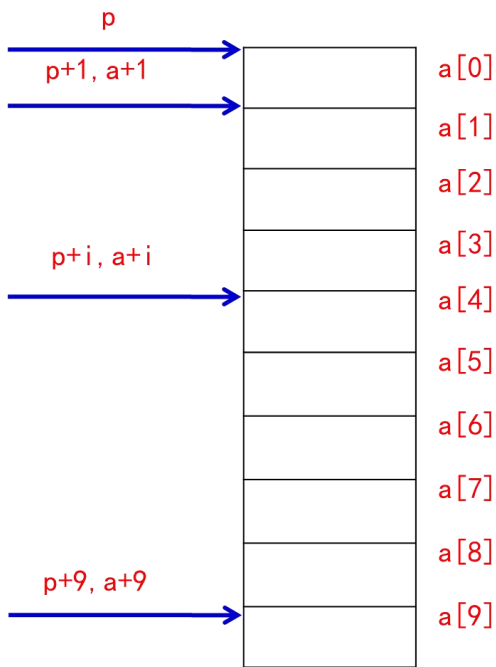
```
float a[10],*p=a;
```

假设 `a[0]` 的地址为 2000，则

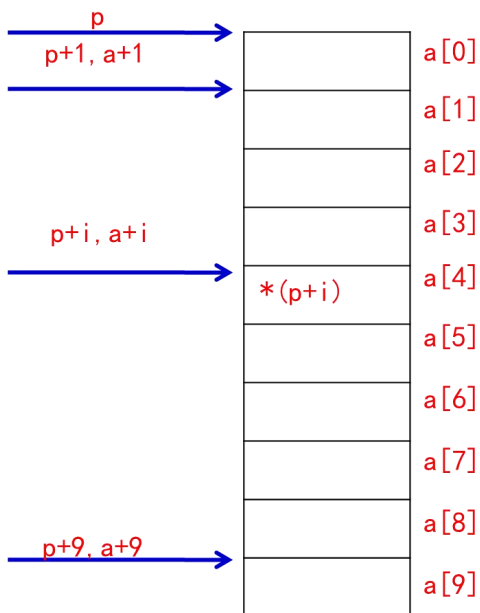
`p` 的值为 2000

`p+1` 的值为 2004

（2）如果 `p` 的初值为 `&a[0]`，则 `p+i` 和 `a+i` 就是数组元素 `a[i]` 的地址，或者说，它们指向 `a` 数组序号为 `i` 的元素。



(3) $*(p+i)$ 或 $*(a+i)$ 是 $p+i$ 或 $a+i$ 所指向的数组元素，即 $a[i]$ 。

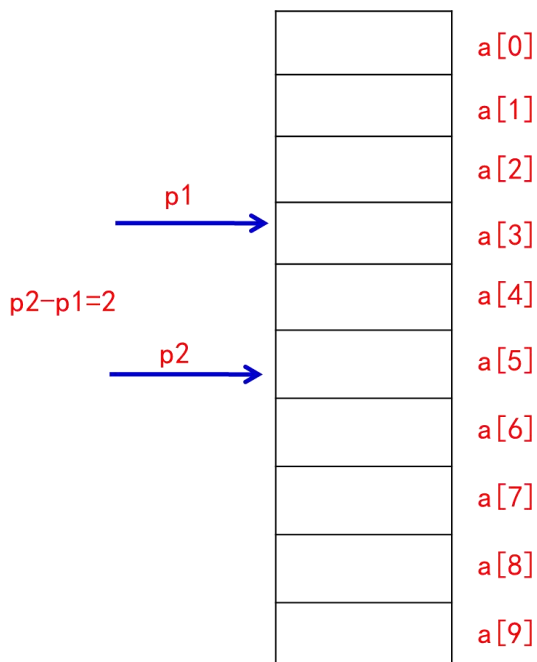


(4)如果 p 指向数组 $a[0]$ ，指向 $p++$ 后 p 的值改变了，指向下一个数组 $a[1]$ 。

(5) 如果指针 $p1$ 和 $p2$ 都指向同一数组。

可以执行: $p2-p1$,结果是两个地址之差除以数组的长度。表示 $p2$ 所指的元素与 $p1$ 所指的元素之间差几个元素。(下一页图)

不能 $p1+p2$ ，两个地址相加无实际意义。



例 8.7 通过指针变量读入数组的 10 个元素，然后输出这 10 个元素。

```
#include <stdio.h>

void main()
{int *p,i,a[10];

  p=a;

  for(i=0;i<10;i++) scanf("%d",p++);

  for(i=0;i<10;i++,p++)

    printf("%d ",*p);

  printf("\n");
}
```

程序输出结果错误！第一个 `for` 循环执行完后，指针 `p` 指向数组的末尾了。第二个循环开始执行，`p` 的起始值不是 `&a[0]`。

```
#include <stdio.h>

void main()
{int *p,i,a[10];

  p=a;

  for(i=0;i<10;i++) scanf("%d",p++);

  p=a;//使指针 p 重新指向&a[0]

  for(i=0;i<10;i++,p++)
```

```
printf("%d ",*p);  
  
printf("\n");  
  
}
```

8.3.3 用数组名作函数参数

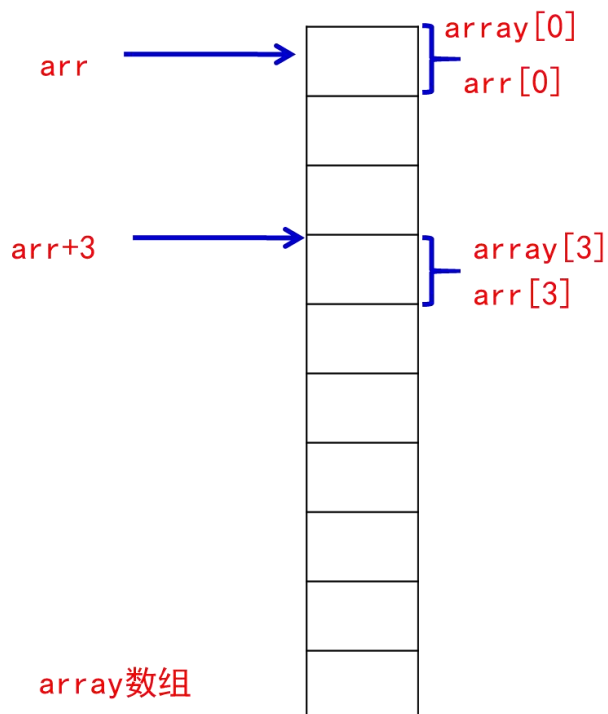
用数组名作函数参数时，因为实参数组名代表该数组首元素的地址，形参应该是一个指针变量。

C 编译都是将形参数组名作为指针变量来处理的。

```
void fun(int arr[],int n)  
{  
    :  
}
```

*fun(int *arr,int n)*

```
void main()  
{  
    void fun(int arr[],int n);  
    int array[10];  
    :  
    fun (array,10);  
}
```



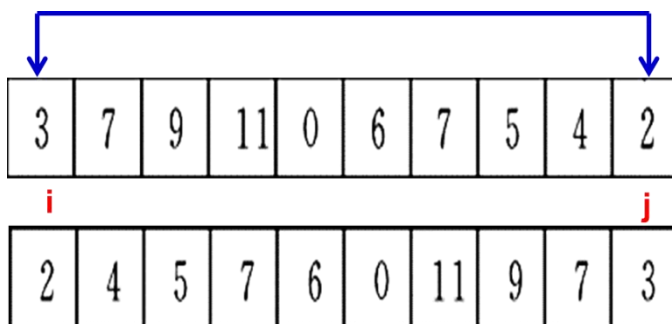
- 实参数组名是指针常量，但形参数组名是按指针变量处理。
- 在函数调用进行虚实结合后，它的值就是实参数组首元素的地址。

➤ 在函数执行期间，形参数组可以再被赋值。

```
void fun (arr[ ],int n)
{ printf( " %d\n" , *arr);
  arr=arr+3;
  printf( " %d\n" , *arr);
}
```

例 8.8 将数组 a 中 n 个整数按相反顺序存放。

解题思路：将 a[0]与 a[n-1]对换，……将 a[4]与 a[5]对换。



```
#include <stdio.h>

void main()
{ void inv(int x[ ],int n);

  int i, a[10]={3,7,9,11,0,6,7,5,4,2};

  for(i=0;i<10;i++) printf( "%d " ,a[i]);

  printf("\n");

  inv(a,10);

  printf( " the array has been inverted:\n" );

  for(i=0;i<10;i++) printf( "%d " ,a[i]);

  printf("\n");

  return 0;
}

void inv(int x[ ],int n)
{ int temp,i,j,m=(n-1)/2;

  for(i=0;i<=m;i++)
  { j=n-1-i;
```

优化

```
void inv(int *x,int n)
{int *p,temp,*i,*j,m=(n-1)/2;
 i=x;j=x+n-1;p=x+m;
 for(;i<=p;i++,j--)
 { temp=*i; *i=*j; *j=temp; }
}
```

```
    temp=x[i];x[i]=x[j];x[j]=temp;
}
}
```

例 8.9 编写一个函数用选择法对 10 个整数按由大到小顺序排序，用数组名作实参。

编程思路：

用选择法排序，其算法前已介绍。

```
#include <stdio.h>
```

```
void main()
```

```
{ void sort(int x[],int n);
```

```
    int *p,i,a[10];
```

```
    p=a;
```

```
    for(i=0;i<10;i++) scanf( "%d" ,p++);
```

```
    p=a;
```

```
    sort(p,10);
```

```
    for(p=a,i=0;i<10;i++)
```

```
    { printf( "%d " ,*p);    p++;    }
```

```
    printf("\n");
```

```
}
```

```
void sort(int x[],int n)
```

```
{ int i,j,k,t;
```

```
void sort(int *x,int n)
```

```
    for(i=0;i<n-1;i++)
```

```
    { k=i;
```

```
        for(j=i+1;j<n;j++)
```

```
        if(x[j]>x[k]) k=j;
```

```
if (*(x+j)>*(x+k)) k=j;
```

```
        if(k!=i)
```

```
        { t=x[i];x[i]=x[k];x[k]=t; }
```

```
    }
```

```
}
```

```
{t=*(x+i);
*(x+i)=*(x+k);
*(x+k)=t;}
```

讲解归纳与举例

1.函数调用时，若形参是一个指针变量，而对应的实参是一个数组名，则函数参数的传递方式是__传递。

【正确答案：地址】

【答案解析】数组名代表数组首元素的地址。

2.若有定义 `int w[10]={23,54,10,33,47,98,72,80,61},*p=w;`则不移动指针 `p`，且通过指针 `p` 引用值为 98 的数组元素的表达式是_____。

【正确答案：*(p+5)】

【答案解析】：指针引用数组元素。

3.下列程序执行后的输出结果是_____。

```
void func (int *a,int b [])  
{ b[0]=*a+6; }  
  
main ()  
{int a, b[5];  
    a=0;  
    b[0]=3;  
    func (&a,b) ;  
    printf ("%d \n",b[0]) ;}
```

【正确答案：6】

【答案解析】：b 是数组名，传递的是数组首元素地址。b[0]=*a+6 即 0+6,结果是 6。

8.4 通过指针引用字符串

8.4.1 字符串的表示形式

➤ 可以用两种方法访问一个字符串：

(1) **用字符数组存放一个字符串**，然后用字符数组名和下标可以访问字符数组中的元素，也可以通过字符数组名用%s 格式符输出一个字符串。

(2) **用字符指针指向一个字符串**。可以不定义字符数组，而定义一个字符指针。用字符指针指向字符串中的字符。

例8.10 定义字符指针，使它指向一个字符串。

```
#include <stdio.h>
void main()
{ char *string="I love China!";
  printf("%s\n",string);
}
```

char *string;
string= "I love China!";



例 8.11 有一字符数组 a，内存有字符串“I am a boy.”，要求把该字符串复制到字符数组 b 中。

解题思路：

从第一个字符开始，将数组 a 中的字符逐个复制到数组 b 中，直到遇到 a 数组中的某一元素值为 ‘\0’ 为止。此时表示数组 a 中的字符串结束，然后在已复制到 b 数组中的字符最后加一个 ‘\0’，表示字符串结束。

```
#include <stdio.h>
void main()
{char a[ ]="I am a boy.",b[20];
  int i;
  for(i=0;*(a+i)!='\0';i++)
    *(b+i)=*(a+i);
  *(b+i)= '\0' ;
  printf("string a is:%s\n",a);
```

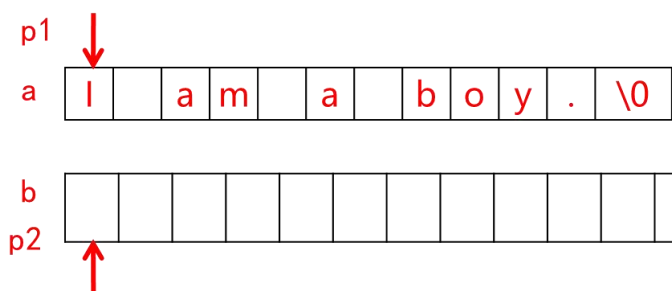
相当于b[i]=a[i];

```
printf("string b is:");  
  
for(i=0;b[i]!='\0';i++)  
  
printf( "%c" ,b[i]);  
  
printf("\n");  
  
}
```

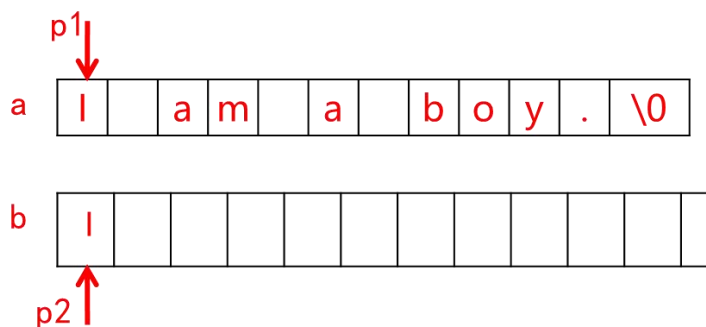
例 8.12 用指针变量来处理例 8.11 问题。

编程思路：

- (1) 使两个指针 p1 和 p2 分别指向字符串 a 和 b 的第一个字符

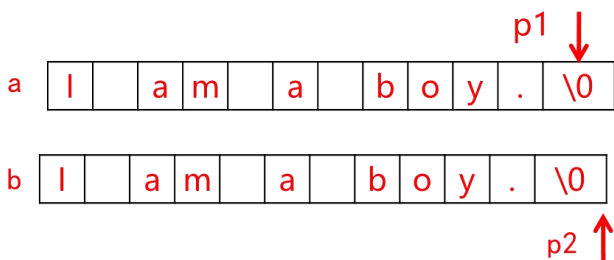


- (2) 将 *p1 (p1 指向的 a 串中的一个字符) 复制到 *p2 (p2 指向的 b 串中的一个字符) 中。
- (3) 使 p1 和 p2 分别下移一个位置，分别指向字符串 a 和 b 的第二个字符。
- (4) 将 *p1 复制到 *p2 中。
- (5) 再使 p1 和 p2 分别下移一个位置。



……发现 p1 指向的字符是 ' \0' 时不再进行复制

- (6) 在 p2 所指的位上赋予 ' \0'



```
#include <stdio.h>
void main()
{char a[]="I am a boy.",b[20],*p1,*p2;
 int i;
 p1=a; p2=b; //用指针实现
 for( ;*p1!='\0';p1++,p2++) *p2=*p1;
 p2='\0';
 printf("string a is:%s\n",a);
 printf("string b is:");
 for(i=0;b[i]!='\0';i++) printf("%c",b[i]);
 printf("\n");
}
```

`printf("string b is:%s\n",b);`

8.4.2 字符指针作函数参数

- 如果想把一个字符串从一个函数“传递”到另一个函数，可以用地址传递的办法，即用字符数组名作参数，也可以用字符指针变量作参数。
- 在被调用的函数中可以改变字符串的内容。
- 在主调函数中可以引用改变后的字符串。

例 8.13 任务同例 8.11，即复制字符串，但要求用函数调用来实现。

```
#include <stdio.h>

void main()

{void copy_string(char *from, char *to);

 char *a="I am a teacher.";

 char b[]="You are a student.";

 char *p=b;

 printf( "a=%s\n b=%s\n",a,p);

 printf( "copy string a to string b:\n ");

 copy_string(a,p);

 printf( "a=%s\n b=%s\n",a,p);

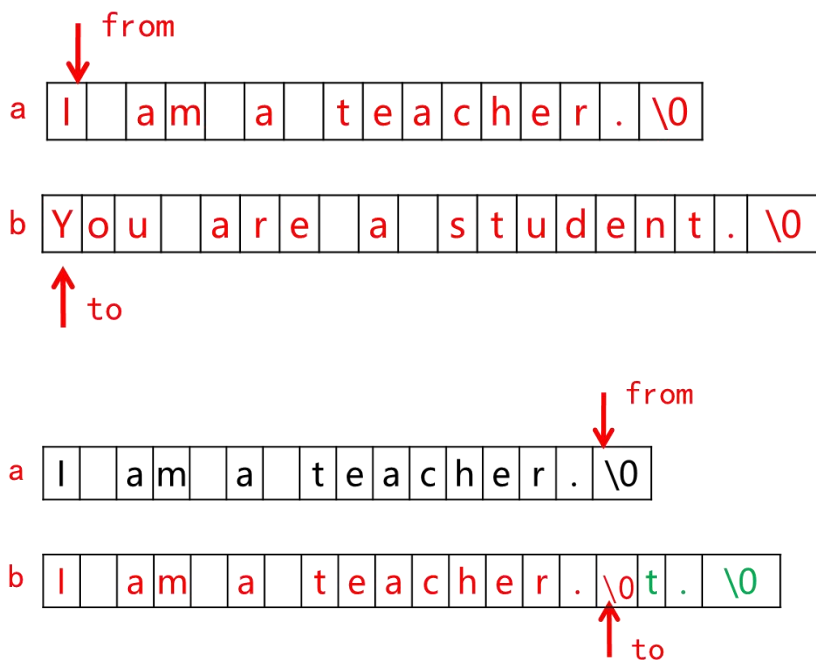
}

void copy_string(char *from,char *to)

{ for( ;*from!= '\0' ;from++,to++)

    { *to=*from;}
```

```
*to='\0';
}
```



例 8.14 有字符串 a= “I am a teacher.”，字符串 b= “You are a student.”，要求把字符串 b 连接到字符串 a 的后面。即字符串 a 的内容为 “I am a teacher. You are a student.”

➤ 解题思路：

- (1) 使 p1 和 p2 分别指向字符串 a 和 b。
- (2) 使 p1 下移到字符串 a 的 ‘\0’ 处。
- (3) 从 p1 指向的元素处开始，将字符串 b 中的字符逐个复制到字符数组 a 中。

用一个函数来实现字符串连接的功能。

```
#include <stdio.h>

void main()
{
    void link_string(char *arr1, char *arr2);

    char a[40]= “I am a teacher.” ;
    char b[]= “You are a student.” ;

    char *p1=a, *p2=b;

    printf( “a:%s\n b:%s\n” ,p1,p2);

    link_string(p1,p2);

    printf( “a:%s\n b:%s\n” ,a,b);
}
```

```
void link_string(char *arr1, char *arr2)
{ int i;
  for( i=0;*arr1!='\0';i++)
    arr1++;
  for(;*arr2!= '\0' ;arr1++,arr2++)
    *arr1=*arr2;
  *arr1= '\0' ;
}
```

调用函数时实参与形参的对应关系

实 参	形 参
字符数组名	字符数组名
字符数组名	字符指针变量
字符指针变量	字符指针变量
字符指针变量	字符数组名

8.4.3 使用字符指针变量和字符数组的区别

虽然用字符数组和字符指针变量都能实现字符串的存储和运算，但它们二者之间是有区别的，不应混为一谈。主要有以下几点。

（1）字符数组由若干个元素组成，每个元素中放一个字符，而字符指针变量中存放的是地址（字符串第 1 个字符的地址），决不是将字符串放到字符指针变量中。

（2）赋值方式。对字符数组只能对各个元素赋值，**不能用以下办法对字符数组赋值：**

```
char str[14];

str= "I love China!" ;
```

而**对字符指针变量，可用下面方法赋值：**

```
char *a;

a= "I love China!" ;
```

但注意赋给 a 的不是字符，而是字符串第一个元素的地址。

（3）对字符指针变量赋初值：

```
char *a= "I love China!" ;
```

等价于

```
char *a;a= "I love China!" ;
```

而对数组的初始化:

```
char str[14]={ "I love China!" };
```

不能等价于

```
char str[14];
```

```
str[]= "I love China!" ;
```

(4) 如果定义了一个字符数组，在编译时为它分配内存单元，它有确定的地址。

而定义一个字符指针变量时，给指针变量分配内存单元，在其中可以放一个字符变量的地址。

```
char str[10];
```

```
scanf( "%s" ,str); 对
```

```
char *a; scanf( "%s" ,a); 错
```

```
char *a,str[10]; a=str;
```

```
scanf( "%s" ,a); 对
```

(5) 指针变量的值是可以改变的。

例 8.15 改变指针变量的值。

➤ 解题思路:

先使指针变量 a 指向字符串第 1 个字符，然后改变指针变量 a 的值，使之指向字符串中第 1 个字符，输出其后面的字符。

```
#include <stdio.h>
void main()
{ char *a="I love China!";
  a=a+7;
  printf("%s\n",a);
}
```

不能改为:
char str[]="I love China!";
str=str+7;
因为数组名代表首地址,
是常量, 值不能改变

a=a+7; 使a指向输出字符串的第8个字符。

输出: China!

(6) 对字符数组可以用下标法和地址法引用数组元素*(a+5)。如果字符指针变量 p=a，则也可以用指针变量带下标的形式和地址法引用*(p+5)。

```
char *a= "I love Cahina!" ;
```

则 a[5]的值是第 6 个字符，即字母 ‘e’

(7) 字符数组中各元素的值是可以改变的（可以对它们再赋值），但字符指针变量指向的字符串常量中的内容是不可以被取代的（不能对它们再赋值）。如：

```
char a[] = "House" ;  
char *b = "House" ;  
a[2] = 'r' ;    对  
b[2] = 'r' ;    错
```

讲解归纳与举例

1. 下面程序段中，for 循环的执行次数是（ ）。

```
char *s = "\ta\018bc";  
for (; *s != '\0'; s++)  
    printf( "*" );
```

【正确答案：6】

【答案解析】：s 所指的字符串长度为 6，其所指的字符串包含的有效字符分别为 ‘\t’ ‘a’ ‘\01’ ‘8’ ‘b’ ‘c’ 。

2. 下面程序段执行后的输出结果是（ ）。

```
char a[] = "language", *p;  
p = a;  
while ( *p != 'u' ) {  
    printf( "%c", *p - 32 );  
    p++;  
}
```

【正确答案：LANG】

【答案解析】：将 u 字符之前的字符小写转大写输出。

3. 编程题：输入字符串，要求将每个单词的第一个字母改为大写字母，并输出。如“hello world!”改为“Hello World ”。

【正确答案：见下一页】

【答案解析】：通过指针应用字符串的理解与应用。

```
#include <stdio.h>

void main() {
    char s[255],prrior=' ',*p;
    gets(s);
    for(p=s; *p!='\0'; p++)    { if(prrior==' ' &&
        *p>='a' && *p <='z')
        *p-=32;
        prrior=*p; }
    puts(s);
}
```

8.5 提高部分

8.5.1 指针使用的技巧

例 8.13 中的 copy_string 函数可以改写：

例8. 13中的copy_string函数可以改写：

```
void copy_string(char *from, char *to)
```

```
{ for( ;*from!='\0';from++,to++)  
  { *to=*from; }  
  *to='\0';  
}
```

(1)
while((*to=*from)!='\0')
{ to++; from++; }

(2)
while((*to++=*from++)!='\0');

(3)
{while(*from!='\0')
 *to++=*from++;
 *to='\0'; }

(4)
{while(*from)
 *to++=*from++;
 *to='\0'; }

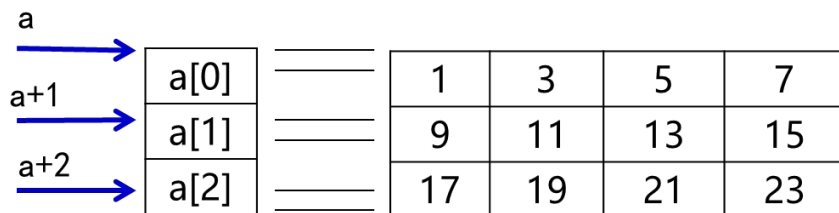
(5)
While(*to++ = *from++);

(6)
for(;*to++=*from++);
或：
for(*to++=*from++);

8.5.2 多维数组的指针

指针变量可以指向一维数组中的元素，也可以指向多维数组中的元素。

```
int a[3][4]={1,3,5,7},  
          {9,11,13,15},{17,19,21,23}};  
  
int (*p)[4]; //指向一维数组的指针变量  
  
p=a+1; //p 指向 a[1]行的开头
```



8.5.3 指向函数的指针

如果在程序中定义了一个函数，在编译时，编译系统为函数代码分配一段存储空间，这段存储空间的起始地址，称为这个函数的指针。

可以定义一个指向函数的指针变量，用来存放某一函数的起始地址，这就意味着此指针变量指向该函数。

例如：

```
int (*p)(int,int);
```

定义 p 是指向函数的指针变量，它可以指向的函数类型为整型且有两个整型形参。

➤ 定义的一般形式为：

数据类型（*指针变量名）（函数参数表列）；

➤ 如果要用指针调用函数，必须先使指针变量指向该函数。如：

```
p=max;
```

把 max 函数的入口地址赋给指针变量 p

➤ 调用函数时，只需将(*p)代替函数名即可。

例如：c=(*p)(a,b);

相当于：c=max(a,b);

指向函数的指针变量的一个重要用途是把函数的入口地址作为实参传递给形参，此时形参是指向函数的指针变量。这样就能够对被调用的函数中使用实参函数。

8.5.4 返回指针值的函数

- 一个函数也可以返回指针型的数据。

例如：`int *a(int x,int y);`

a 是函数名，*表示此函数值是指针。

最前面的 int 表示返回的指针是指向整型变量的。

调用它以后能得到一个指向整型数据的指针(地址)。

- 返回指针值的函数一般定义形式为

类型名 *函数名(参数表列);

- 这种形式与定义指向函数的指针变量很相似，但请注意：在*a 两侧没有括号。有括号就成指向函数的指针变量了。

8.5.5 指针数组

一个数组，若其元素均为指针类型数据，称为指针数组。

例如：`int *p[4];`

p 是数组，有 4 个元素

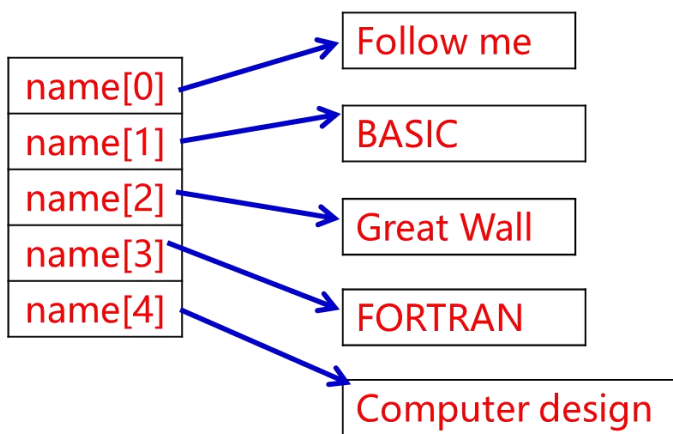
*表示此数组是指针类型的，每个数组元素（相当于一个指针变量）都可指向一个整型变量。

- 一维指针数组的定义的一般形式为

类型名 数组名[数组长度];

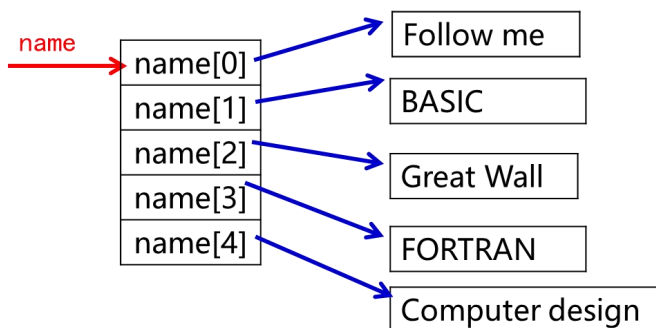
- 指针数组比较适合用来指向若干个字符串，使字符串处理更加方便灵活。
- 可以分别定义一些字符串，然后用指针数组中的元素分别指向各字符串。
- 由于各字符串长度一般是不相等的，所以比用二维数组节省内存单元。

- 图书馆有若干本书，想把书名放在一个数组中，然后要对这些书目进行排序和查询。



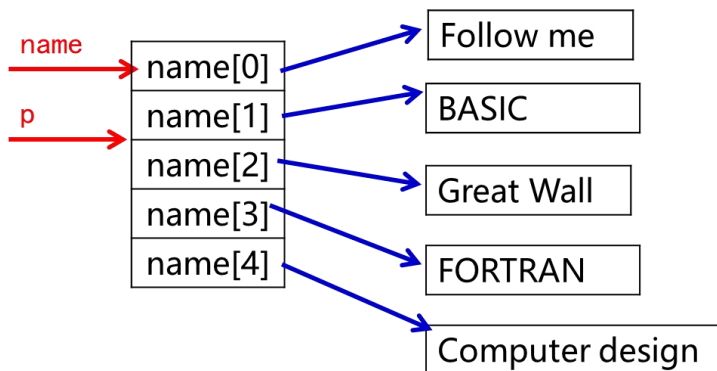
8.5.6 多重指针——指向指针的指针

指向另一个指针数据的指针变量，称为**指向指针的指针**。



```
char **p;
```

```
p=name+2;
```



本章小结

- (1) 理解指针以及指针变量的概念。
- (2) 掌握指针的基本定义和引用。
- (3) 掌握指针变量作为函数参数的应用。
- (4) 掌握指向数组的指针，通过指针引用数组元素，用数组名作为函数参数。

- (5) 掌握指针运算。
- (6) 通过指针引用字符串。
- (7) 字符指针和字符数组的区别。
- (8) 各种指针变量定义形式的比较 P281 表 8.3。

第9章使用结构体类型处理组合数据----用户自定义数据类型

本章重难点分析

C语言允许用户根据需要自己建立数据类型，用它来定义变量。本章主要介绍结构体的概念和定义、结构体变量、结构体指针、结构体数组、用结构体变量和指针作为函数参数、用指针处理链表、枚举类型以及共用体等。

本章内容注意以下几点：

结构体以及结构体变量	重点&难点
结构体指针	重点
结构体数组	重点&难点
用结构体指针处理链表	难点

9.1 定义和使用结构体变量

9.1.1 自己建立结构体类型

用户自己建立由不同类型数据组成的组合型的数据结构，它称为**结构体**。

例如，一个学生的学号、姓名、性别、年龄、成绩、家庭地址等项，是属于同一个学生的，因此组成一个**组合数据**，如 student_1 的变量，反映它们之间的内在联系。

```
struct student
{
    int num;

    char name[20];

    char sex;

    int age;

    float score;

    char addr[30];
};
```

由程序设计者指定了一个**结构体类型 struct Student**。

它包括 num,name,sex,age,score,addr 等不同类型的成员。

➤ 声明一个结构体类型的一般形式为：

```
struct 结构体名
{ 成员表列 };
```

➤ 其中成员的声明形式：

类型名 成员名；

➤ 说明：

（1）结构体类型并非只有一种，而是可以设计出许多种结构体类型，例如：

```
struct teacher

struct worker

struct date 等结构体类型
```

各自包含不同的成员。

（2）成员可以是另一个结构体类型。

```
struct date
{
    int month;

    int day;
```



```
int year; };\n\nstruct student\n{\n    int num;char name[20];\n\n    char sex;  int age;\n\n    struct date birthday;\n\n    // birthday 是 struct date 类型\n\n    char addr[30];\n\n};
```

9.1.2 定义结构体类型变量

前面只是建立了一个结构体类型，它相当于一个模型，并没有定义变量，其中并无具体数据，系统对之也不分配存储单元。

相当于设计好了图纸，但并未建成具体的房屋。为了能在程序中使用结构体类型的数据，应当定义结构体类型的变量，并在其中存放具体的数据。

1.先声明结构体类型，再定义该类型变量

声明结构体类型 `struct student`，可以用它来定义变量。

```
struct student  student1,student2;
```



`student1`

10001	Zhang Xin	M	19	90.5	Shanghai
-------	-----------	---	----	------	----------

`student2`

10002	Wang Li	F	20	98	Beijing
-------	---------	---	----	----	---------

2.在声明类型的同时定义变量

```
struct student\n{\n    int num;\n\n    char name[20];\n\n    char sex;
```

```
int age;  
  
float score;  
  
char addr[30];  
  
} student1, student2;
```

3. 不指定类型名而直接定义结构体类型变量

➤ 其一般形式为：

```
struct  
{ 成员表列 } 变量名表列;  
  
指定了一个无名的结构体类型。
```

➤ 说明：

- (1) 结构体类型与结构体变量是不同的概念，不要混同。只能对变量赋值、存取或运算，而不能对一个类型赋值、存取或运算。在编译时，对类型是不分配空间的，只对变量分配空间。
- (2) 结构体类型中的成员名可以与程序中的变量名相同，但二者不代表同一对象。
- (3) 对结构体变量中的成员（即“域”），可以单独使用，它的作用与地位相当于普通变量。

9.1.3 结构体变量的初始化和引用

例 9.1 把一个学生的信息放在一个结构体变量中，然后输出这个学生的信息。

➤ 解题思路：

先在程序中自己建立一个结构体类型，包括有关学生信息的各成员。

然后用它来定义结构体变量，同时赋以初值。

最后输出该结构体变量的各成员。

```
#include <stdio.h>
```

```
void main()
```

```
{struct student
```

```
{ int num;    char name[20];
```

```
    char sex;    char addr[20];
```

```
} student1 = {10101, "Li Lin", 'M',
```

```
    "123 Beijing Road" };
```

//定义结构体的同时定义变量并初始化

```
printf("NO.: %d\n name: %s\n
```

```
sex:%c\naddress:%s\n",
student1.num, student1.name,
student1.sex, student1.addr);
}
// student1.num 结构体成员的引用
#include <stdio.h>
void main()
{struct student
{ int num;    char name[20];
char sex;    char addr[20]; }student1={10101, "LiLin", 'M', "123 Beijing Road" };
.....
}
student1.num=10010; 对
printf( "%s\n" ,student1); 不对
#include <stdio.h>
void main()
{struct student {struct date birthday;
{ int num;  char name[20];
char sex;  char addr[20]; }student1={10101,"LiLin",'M',
"123 Beijing Road"};
.....
}
student1.num=10010; 对
student1.birthday.month=11; 对
```

```
#include <stdio.h>
void main()
{struct student
{ int num;  char name[20];
  char sex;  char addr[20]; }student1={10101,"LiLin",'M',
  "123 Beijing Road"};
  .....
  },student2;
student2.num=student1.num; 对
sum=student1.num+student2.num; 对
student1.num++; 对
```

```
#include <stdio.h>
void main()
{struct student
{ int num;  char name[20];
  char sex;  char addr[20]; }student1={10101,"LiLin",'M',
  "123 Beijing Road"};
  .....
  },student2;
student2=student1; 对
scanf("%d",&student1.num); 对
scanf("%o",&student1); 对
scanf("%d,%s,%c,%s",&student1); 错
```

例 9.2 输入两个学生的学号、姓名和成绩，输出成绩较高学生的学号、姓名和成绩。

➤ 解题思路：

- (1) 定义两个结构相同的结构体变量 student1 和 student2;
- (2) 分别输入两个学生的学号、姓名和成绩;
- (3) 比较两个学生的成绩，如果学生 1 的成绩高于学生 2，就输出学生 1 的全部信息，如果学生 2 的成绩高于学生 1，就输出学生 2 的全部信息。如果二者相等，输出 2 个学生的全部信息。

```
#include <stdio.h>
void main()
{ struct student
  { int num;
    char name[20];
    float score;
  } student1, student2;
  scanf("%d%s%f", &student1.num,
    student1.name, &student1.score);
  scanf("%d%s%f", &student2.num,
    student2.name, &student2.score);
  printf("The higher score is:\n");
  if (student1.score > student2.score)
    printf("%d %s %6.2f\n", student1.num,
      student1.name, student1.score);
  else if (student1.score < student2.score)
    printf("%d %s %6.2f\n", student2.num,
      student2.name, student2.score);
  else
  { printf("%d %s %6.2f\n", student1.num,
    student1.name, student1.score);
    printf("%d %s %6.2f\n", student2.num,
      student2.name, student2.score);
  }
}
```

不能加&

讲解归纳与举例

1. 有如下结构体说明，以下叙述中错误的是（ ）。

```
struct stu {
    int a; float b;
} stutype;
```

A.struct 是结构体类型的关键字

B.struct stu 是用户定义的结构体类型

C.stutype 是用户定义的结构体类型名

D.a 和 b 都是结构体成员名

【正确答案：C】

【答案解析】：stutype 是结构体类型 struct stu 的一个变量。

2.若 a、b 都是结构体变量，语句 a=b; 能够执行的条件是___。

【参考答案：两者数据类型相同】

【答案解析】：结构体基本概念。

9.2 结构体数组

例 9.3 有 3 个候选人，每个选民只能投票选一人，要求编一个统计选票的程序，先后输入被选人的名字，最后输出各人得票结果。

➤ 解题思路：

设一个结构体数组，数组中包含 3 个元素。

每个元素中的信息应包括候选人的姓名（字符型）和得票数（整型）。

输入被选人的姓名，然后与数组元素中的“姓名”成员比较，如果相同，就给这个元素中的“得票数”成员的值加 1。

输出所有元素的信息。

```
#include <string.h>
#include <stdio.h>
struct person
{ char name[20];
  int count;
} leader[3]={"Li",0,"Zhang",
0,"Sun",0};
```

全局的结构体数组

leader[0]

name	count
Li	0
Zhang	0
Sun	0

```
void main()
```

```
{ int i,j;   char leader_name[20];
```

```
    for (i=1;i<=10;i++)
```

```
{ scanf("%s",leader_name);
```

```
    for(j=0;j<3;j++)
```

```
if(strcmp(leader_name,leader[j].name)==0)
```

```
    leader[j].count++; }
```

```
//leader[j].count=leader[j].count+1;
```

```
printf("\nResult :\n");
```

```
    for(i=0;i<3;i++)
```

```
        printf("%5s:%d\n",leader[i].name,
```

```
leader[i].count);
```

```
}
```

```
void main()
{ int i,j; char leader_name[20];
  for (i=1;i<=10;i++)
  { scanf("%s",leader_name);
    for(j=0;j<3;j++)
    if(strcmp(leader_name,leader[j].name)==0)
      leader[j].count++; }
  for(i=0;i<3;i++)
    printf("%5s:%d\n",leader[i].name,
    leader[i].count);
}
```

```
Li
Li
Sun
Zhang
Zhang
Sun
Li
Sun
Zhang
Li
```

```
Li:4
Zhang:3
Sun:3
```

➤ 说明：

(1) 定义结构体数组一般形式是

① **struct 结构体名**

{成员表列} 数组名[数组长度];

② 先声明一个结构体类型，然后再用此类型定义结构体数组：

结构体类型 数组名[数组长度];

如：

```
struct person leader[3];
```

(2) 对结构体数组初始化的形式是在定义数组的后面加上：

```
={初值表列};
```

如：

```
struct person leader[3]= {"Li",0,"Zhang",0, "Sun",0};
```

例 9.4 有 N 个学生的信息（包括学号、姓名、成绩），要求按照成绩的高低顺序输出各学生的信息。

解题思路：用结构体数组存放 N 个学生信息，采用选择法对各元素进行排序（进行比较的是各元素中的成绩）。

```
#include <stdio.h>
```

```
#define N 5
```

```
struct student
```



```
{ int num; char name[20]; float score;  };

void main()

{ struct student stu[N]

    ={{10101,"Zhang",78 },

      {10103,"Wang",98.5},

      {10106,"Li ",86},

      {10108, "Ling", 73.5},

      {10110, "Sun", 100}  };

    struct student temp;

    int i,j,k;

printf("The order is:\n");

for(i=0;i<N-1;i++)

{ k=i;

    for(j=i+1;j<N;j++)

        if(stu[j].score>stu[k].score)    k=j;

    temp=stu[k];

    stu[k]=stu[i];    stu[i]=temp;

}

for(i=0;i<N;i++)

    printf("%6d %8s %6.2f\n",

        stu[i].num,stu[i].name,stu[i].score);

printf("\n");

}
```

讲解归纳与举例

1.结构体数组（名词解释）

【参考答案：一个结构体变量中可以存放一组有关联的数据（如一个学生的学号、姓名、成绩等数据），如果有 10 个学生的数据需要参与运算，就需要使用数组，这就是结构体数组。】

【答案解析】：结构体数组基本概念的理解。

2.程序输出结果是（ ）。

```
#include <stdio.h>

int main() {

struct complx { int x;  int y; } cnum[2] = { 1, 3, 2, 7 };

printf( "%d", cnum[0].y / cnum[0].x * cnum[1].x );

return(0); }
```

【正确答案： 6】

【答案解析】： 结构体数组的应用和理解。

9.3 结构体指针

所谓**结构体指针**就是指向结构体数据的指针。一个结构体变量的起始地址就是这个结构体变量的指针。指向结构体对象的指针变量既可以指向结构体变量，也可以用来指向结构体数组中的元素。但是，指针变量的基类型必须与结构体变量的类型相同。例如：

```
struct student *pt;
```

例 9.5 通过指向结构体变量的指针变量输出结构体变量中成员的信息。

➤ 解题思路：在已有的基础上，本题要解决两个问题：

怎样对结构体变量成员赋值；

怎样通过指向结构体变量的指针访问结构体变量中成员。

```
#include <stdio.h>
```

```
#include <string.h>
```

```
void main()
```

```
{ struct student
```

```
{ long num;
```

```
    char name[20];
```

```
    char sex;
```

```
    float score;
```

```
};
```

```
.....
```

```
struct student stu_1;
```

```
struct student * p; //结构体指针变量
```

```
p=&stu_1; //指向结构体变量 stu_1
```

```
stu_1.num=10101;
```

```
strcpy(stu_1.name, "Li Lin" );
```

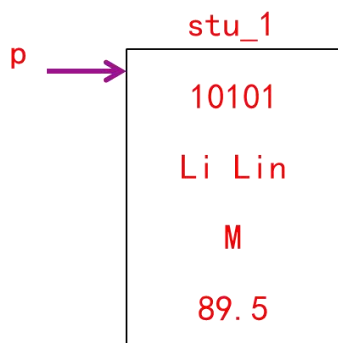
```
stu_1.sex='M ' ;    stu_1.score=89.5;
```

```
printf("No.:%ld\n",stu_1.num);
```

```
printf("name:%s\n",stu_1.name);
```

```
printf("sex:%c\n",stu_1.sex);
```

```
printf("score:%5.1f\n",stu_1.score);  
}
```



```
struct student stu_1;  
struct student * p;  
p=&stu_1;  
stu_1.num=10101;  
strcpy(stu_1.name,"Li Lin");  
stu_1.sex='M'; stu_1.score=89.5;  
printf("No.:%ld\n",stu_1.num);  
printf("name:%s\n",stu_1.name);  
printf("sex:%c\n",stu_1.sex);  
printf("score:%5.1f\n",stu_1.score);  
}
```

//用指针来访问
(*p).num
或p->num

➤ 说明：

为了方便和直观，允许把`(*p).num`用`p->num`来代替。

同样：`(*p).name`等价于`p->name`。

如果 `p` 指向一个结构体变量 `stu`，以下等价：

① `stu.成员名` (如 `stu.num`)

② `(*p).成员名` (如 `(*p).num`)

`p->成员名` (如 `p->num`)

例 9.6 有 3 个学生的信息，放在结构体数组中，要求输出全部学生的信息。

➤ 解题思路：用指向结构体变量的指针处理

(1) 声明 `struct student`，并定义结构体数组、初始化。

(2) 定义指向 `struct student` 类型指针 `p`。

(3) 使 `p` 指向数组首元素，输出元素中各信息。

(4) 使 p 指向下一个元素，输出元素中各信息。

(5) 再使 p 指向结构体数组的下一个元素，输出它指向的元素中的有关信息。

```
#include <stdio.h>
```

```
struct student
```

```
{ int num;    char name[20];
```

```
    char sex;    int age;
```

```
};
```

```
struct student stu[3]={
```

```
    {10101,"Li Lin",'M',18},
```

```
    {10102,"Zhang Fun",'M',19},
```

```
    {10104,"Wang Min",'F',20} };
```

```
void main()
```

```
{ struct student *p;
```

```
    printf(" No.   Name    sex  age\n");
```

```
    for(p=stu;p<stu+3;p++)
```

```
        printf( "%5d %-20s %2c %4d\n" ,
```

```
                p->num, p->name,
```

```
                p->sex, p->age);
```

```
}
```

10101	Li Lin	M	18	stu[0]
10102	Zhang Fang	M	19	stu[1]
10104	Wang Min	F	20	stu[2]

p→	10101	Li Lin	M	18	stu[0]
	10102	Zhang Fang	M	19	stu[1]
	10104	Wang Min	F	20	stu[2]

```
No.   Name                sex age
10101 Li Lin                M   18
```

	10101	Li Lin	M	18	stu[0]
p →	10102	Zhang Fang	M	19	stu[1]
	10104	Wang Min	F	20	stu[2]

```
No.  Name                sex age
10101 Li Lin              M   18
10102 Zhang Fun           M   19
```

	10101	Li Lin	M	18	stu[0]
	10102	Zhang Fang	M	19	stu[1]
p →	10104	Wang Min	F	20	stu[2]

```
No.  Name                sex age
10101 Li Lin              M   18
10102 Zhang Fun           M   19
10104 Wang Min            F   20
```

讲解归纳与举例

1.结构体变量成员的引用方式是使用___运算符，结构体指针变量成员的引用方式是使用___运算符。

【正确答案：第 1 空：点，第 2 空：指向[或者]->】

【答案解析】：结构体变量成员的引用以及结构体指针引用成员的方式。

2.程序输出结果（ ）。

```
#include <stdio.h>

struct st { int x; int *y; } *p;

int main() {
    int dt[4] = { 10, 20, 30, 40 };
    struct st aa[4] = { 50, &dt[0], 60, &dt[1], 60, &dt[2], 60, &dt[3] };
    p = aa;
    printf( "%d", ++(p->x) );
```

```
return(0); }
```

【正确答案： 51】

【答案解析】： p=aa 执行后 p 指向 aa 数组中的第一个结构体对象 aa[0] ， 值为{50,&dt[0]}。

p->x 值为 50， ++(p->x) 为 51。

9.4 用结构体变量和结构体变量的指针作函数参数

将一个结构体变量的值传递给另一个函数，有 3 个方法。

(1) 用结构体变量的成员作参数。

例如，用 `stu[1].num` 或 `stu[2].name` 作函数实参，将实参值传给形参。

用法和用普通变量作实参是一样的，属于“**值传递**”方式。

应当注意实参与形参的类型保持一致。

(2) 用结构体变量作实参。

用结构体变量作实参时，将结构体变量所占的内存单元的内容全部按顺序传递给形参，形参也必须是同类型的结构体变量。

在函数调用期间形参也要占用内存单元。这种传递方式在空间和时间上开销较大。

在被调用函数期间改变形参（也是结构体变量）的值，不能返回主调函数。

一般较少用这种方法。

(3) 用指向结构体变量（或数组）的指针作实参，将结构体变量（或数组元素）的地址传给形参。

例 9.7 有 N 个结构体变量 `stu`，内含学生学号、姓名和 3 门课程的成绩。要求输出平均成绩最高的学生的信息（包括学号、姓名、3 门课程成绩和平均成绩）。

➤ 解题思路：按照功能函数化的思想，分别用 3 个函数来实现不同的功能：

用 `input` 函数输入数据和求各学生平均成绩。

用 `max` 函数找平均成绩最高的学生。

用 `print` 函数输出成绩最高学生的信息。

在主函数中先后调用这 3 个函数，用指向结构体变量的指针作实参。最后得到结果。

本程序假设 `N=3`。

```
#include <stdio.h>
```

```
#define N 3
```

```
struct student
```

```
{ int num;
```

```
    char name[20];
```

```
    float score[3];
```

```
    float aver;
```

```
};
```

4个成员，输入前三个成员的值，计算最后成员值


```
void main()
{ void input(struct student stu[]);

  struct student max(struct student stu[]);

  void print(struct student stu);

  struct student stu[N],*p=stu;

  input(p);

  print(max(p));

}
```

```
void input(struct student stu[])
{ int i;

  printf("请输入各学生的信息：

    学号、姓名、三门课成绩:\n");

  for(i=0;i<N;i++)
  {scanf("%d %s %f %f %f",

    &stu[i].num,stu[i].name,

    &stu[i].score[0],&stu[i].score[1],

    &stu[i].score[2]);

    stu[i].aver=(stu[i].score[0]+

    stu[i].score[1]+stu[i].score[2])/3.0;}}
```

i=0

→ stu	10101	Li	78	89	98	88.33	stu[0]
							stu[1]
							stu[2]

i=1

→ stu	10101	Li	78	89	98	88.33	stu[0]
	10103	Wang	98.5	87	69	84.83	stu[1]
							stu[2]

i=2

	10101	Li	78	89	98	88.33	stu[0]
	10103	Wang	98.5	87	69	84.83	stu[1]
→ stu	10106	Sun	88	76.5	89	84.5	stu[2]

```
struct student max(struct student stu[])
```

```
{int i,m=0;
```

```
for(i=0;i<N;i++)
```

```
if (stu[i].aver>stu[m].aver) m=i;
```

```
return stu[m]; //返回最大值
```

```
}
```

```
void print(struct student stud)
```

```
{ printf("\n 成绩最高的学生是:\n");
```

```
printf("\n学号:%d\n 姓名:%s\n
```

```
三门课成绩:%5.1f,%5.1f,%5.1f\n
```

```
平均成绩:%6.2f\n", stud.num,
```

```
stud.name,stud.score[0], stud.score[1],stud.score[2],stud.aver);
```

```
}
```

```
成绩最高的学生是:
学号:10101
姓名:Li
三门课成绩: 78.0, 89.0, 98.0
平均成绩: 88.33
```

➤ 以上 3 个函数的调用，情况各不相同：

调用 input 函数时，实参是指针变量，形参是结构体数组，传递的是结构体元素的地址，函数无返回值。

调用 max 函数时，实参是指针变量，形参是结构体数组，传递的是结构体元素的地址，函数的返回值是结构体类型数据。

调用 print 函数时，实参是结构体变量，形参是结构体变量，传递的是结构体变量中各成员的值，函数无返回值。

讲解归纳与举例

1.程序输出结果（ ）。

```
#include <stdio.h>

struct STU {

char name[10];

int num;

float TotalScore; };

void f( struct STU *p ) {

struct STU s[2] = {

{ "SunDan", 20044, 550 },

{ "Penghua", 20045, 537 } }, *q = s;

++p; ++q; *p = *q;

}
```

1.程序输出结果（ ）。

```
int main() {

struct STU s[3] = {

{ "YangSan", 20041, 703 },

{ "LiSiGuo", 20042, 580 } };

f( s );

printf( "%s %d %3.0f", s[1].name, s[1].num, s[1].TotalScore );

return(0);

}
```

【正确答案： Penghua 20045 537】

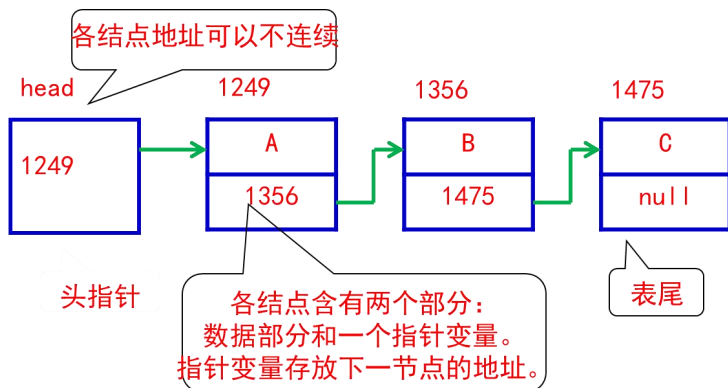
【答案解析】： f 函数借助传递的结构体指针将 main 函数中的 s 数组内容更改了。

9.5 用指针处理链表

9.5.1 什么是线性链表

链表是一种常见的重要的数据结构

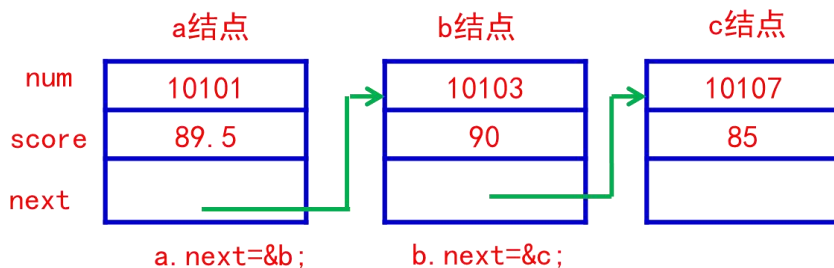
用结构体变量和指针相结合来构造链表。



```
struct student
{
    int num;

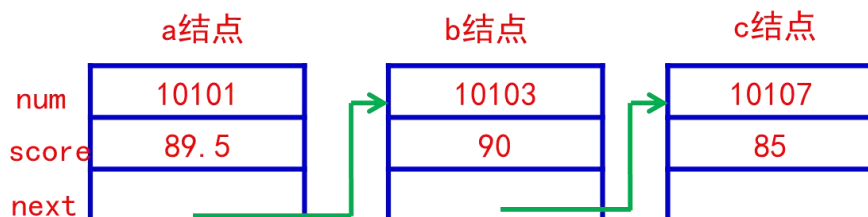
    float score;

    struct student *next;
}a,b,c;
```



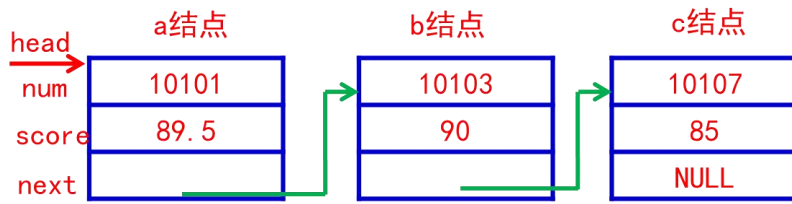
9.5.2 建立简单的静态链表

例 9.8 建立一个如图所示的简单链表，它由 3 个学生数据的结点组成，要求输出各结点中的数据。



➤ 解题思路：

head=&a; a.next=&b;
b.next=&c; c.next=NULL;



```
#include <stdio.h>
```

```
struct student
```

```
{ int num;
```

```
    float score;
```

```
    struct student *next;
```

```
};
```

//定义结构体，三个成员：编号，分数，指向下一节点的指针。

```
void main()
```

```
{ struct student a,b,c,*head,*p;
```

```
    a.num=10101;  a.score=89.5;
```

```
    b.num=10103;  b.score=90;
```

```
    c.num=10107;  c.score=85;
```

```
    head=&a;      a.next=&b;
```

```
    b.next=&c;     c.next=NULL;
```

```
    p=head;
```

```
    do
```

```
    {printf( "%ld%5.1f\n" ,p->num,p->score);
```

```
        p=p->next;
```

```
    } while(p!=NULL);
```

```
}
```

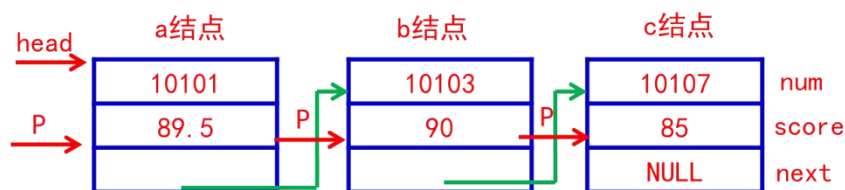
```
p=head;
```

```
do
```

```
{printf( "%ld%5.1f\n" ,p->num,p->score);
```

```
    p=p->next;
```

```
    }while(p!=NULL);
}
```



```
p=head;
do
{printf(“%ld%5.1fn”,p->num,p->score);
  p=p->next;
}while(p!=NULL);
}
```

```
10101 89.5
10103 90.0
10107 85.0
```

9.5.3 建立动态链表

所谓建立动态链表是指在程序执行过程中从无到有地建立起一个链表，即一个一个地开辟结点和输入各结点数据，并建立起前后相链的关系。

例 9.9 建立一个有 2 名学生学号和成绩数据的单向动态链表。

➤ 解题思路：

定义结构体变量，其成员包括学号、成绩和指针变量。

动态地开辟一个新单元（动态开辟内存单元用 **malloc** 函数）。使指针变量 **p** 和 **head** 指向此结点。

向此结点输入数据。

再开辟第 2 个新结点，并使指针变量 **p** 指向此结点。

使第 2 个结点中的指针变量的值为 **NULL**，即不指向任何对象，链表到此为止。

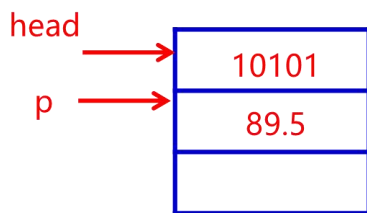
输出两个结点中的数据。

```
#include <stdio.h>
#include <malloc.h>
#define LEN sizeof(struct student)

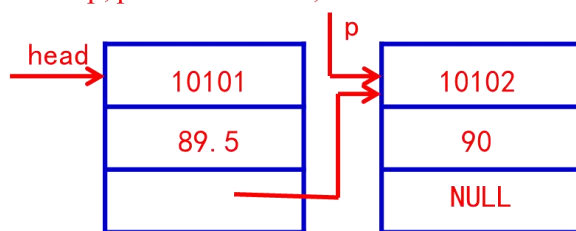
struct student
{
    int num;
    float score;
    struct student *next;
};
```

struct student
类型数据的长度

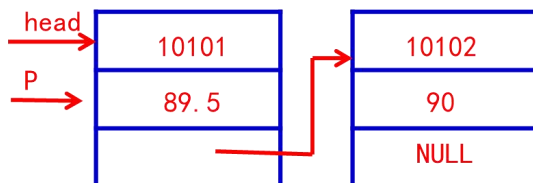
```
void main()
{
    struct student *head,*p;
    head=p=(struct student*) malloc(LEN);
    scanf("%d,%f",&p->num,&p->score);
```



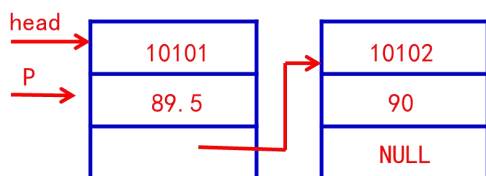
```
void main()
{
    struct student *head,*p;
    head=p=(struct student*) malloc(LEN);
    scanf("%d,%f",&p->num,&p->score);
    p=(struct student*) malloc(LEN);
    scanf("%d,%f",&p->num,&p->score);
    head->next=p; p->next=NULL;
```



```
void main()
{ struct student *head,*p;
head=p=(struct student*) malloc(LEN);
scanf("%d,%f",&p->num,&p->score);
p=(struct student*) malloc(LEN);
scanf("%d,%f",&p->num,&p->score);
head->next=p; p->next=NULL;
p=head;
```

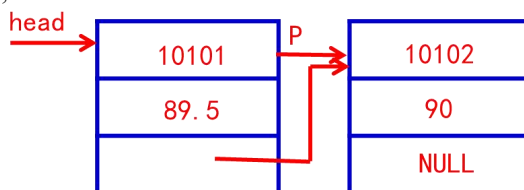


```
void main()
{ .....
printf("结点1:%d,%6.2f\n",
p->num,p->score);
```



结点1: 10101, 89.50

```
void main()
{ .....
printf("结点1:%d,%6.2f\n",
p->num,p->score);
p=p->next;
printf("结点2:%d,%6.2f\n",
p->num,p->score);
}
```



结点1: 10101, 89.50

结点2: 10102, 90.00

讲解归纳与举例

1.单向链表中，指针 p 所指结点为最后一个结点的条件是___。

【正确答案：p->next==NULL】

【答案解析】：链表的基本概念和操作。

2.链表存储结构的每个结点所占存储空间（ ）。

- A.分两部分，一部分存放结点值，另一部分存放表示结点间关系的指针
- B.只有一部分，存放结点值
- C.只有一部分，存储表示结点间关系的指针
- D.分两部分，一部分存放结点值，另一部分存放结点所占单元数

【正确答案：A】

【答案解析】：链表的存储。

9.6 提高部分

9.6.1 共用体类型

有时想用同一段内存单元存放不同类型的变量。

使几个不同的变量共享同一段内存的结构，称为“共用体”类型的结构。



➤ 定义共用体类型变量的一般形式为：

union 共用体名

{ 成员表列

}变量表列；

例如：

```
union data
{ int i;
  char ch;
  float f;
}a,b,c;

union data
{ int i;
  char ch;
  float f;
};

union data a,b,c;
```

- “共用体”与“结构体”的定义形式相似，但它们的含义是不同的。
- 结构体变量所占内存长度是各成员占的内存长度之和，每个成员分别占有其自己的内存单元。而共用体变量所占的内存长度等于最长的成员的长度。
- 只有先定义了共用体变量才能引用它，但应注意，不能引用共用体变量，而只能引用共用体变量中的成员。

例如，前面定义了 a, b, c 为共用体变量，下面的引用方式是正确的：

a.i a.ch a.f

9.6.2 枚举类型

如果一个变量只有几种可能的值，则可以定义为枚举类型。

所谓“枚举”就是指把可能的值一一列举出来，变量的值只限于列举出来的值的范围内。

➤ 声明枚举类型用 **enum** 开头。

例如：

```
enum Weekday{
```

```
sun,mon,tue,  
wed,thu,fri,sat};
```

声明了一个枚举类型 enum Weekday

然后可以用此类型来定义变量

```
enum Weekday  weekday,week_end;
```

```
weekday=mon;          //正确
```

```
week_end=sun;         //正确
```

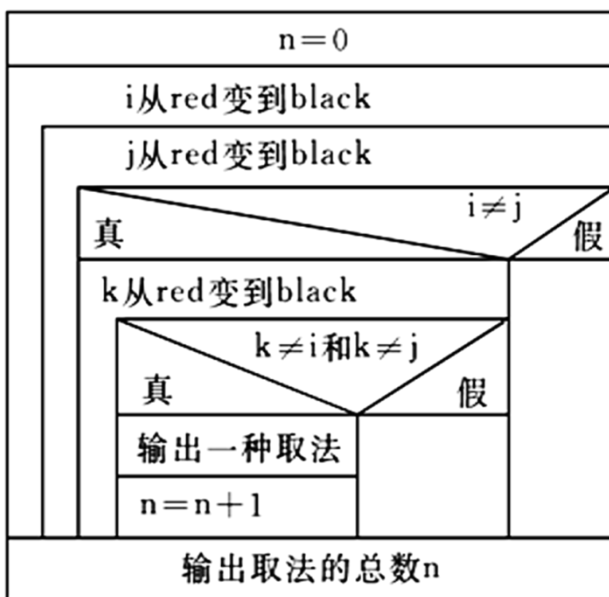
也可以直接定义枚举变量，例如：

```
enum{sun,mon,tue, wed,thu,fri,sat  
}weekday,week_end;
```

其中 sun、mon、…、sat 称为**枚举元素或枚举常量**，它们是用户定义的标识符。

例 9.10 口袋中有红、黄、蓝、白、黑 5 种颜色的球若干个。每次从口袋中先后取出 3 个球，问得到 3 种不同颜色的球的可能取法，输出每种排列的情况。

➤ 解题思路：



loop由1到3				
loop的值				
1	2	3		
i⇒pri	j⇒pri	k⇒pri		
pri的值				
red	yellow	blue	white	black
输出	输出	输出	输出	输出
"red"	"yellow"	"blue"	"white"	"black"

```
#include <stdio.h>
```

```
void main()
```

```
{enum color{red,yellow,blue,white,black};
```

```
enum color i,j,k,pri; int n,loop;
```

```
n=0;
```

```
for (i=red;i<=black;i++)
```

```
for (j=red;j<=black;j++)
```

```
if (i!=j)
```

```
{ for (k=red;k<=black;k++)
```

```
if ((k!=i) && (k!=j))
```

```
{ n=n+1;
```

```
printf( "%-4d" ,n);
```

```
for (loop=1;loop<=3;loop++)
```

```
{switch (loop)
```

```
{ case 1: pri=i;break;
```

```
case 2: pri=j;break;
```

```
case 3: pri=k;break;
```

```
default:break;
```

```
}
```

```
switch (pri)
```

```
{case red:
```

```
printf( "%-10s" , "red" ); break;
```

```
case yellow:

    printf("%-10s","yellow"); break;

case blue:

    printf( "%-10s" , "blue" ); break;

case white:

    printf( "%-10s" , "white" ); break;

case black:

    printf("%-10s","black"); break;

}

}

        printf("\n");

    }

}

printf("\ntotal:%5d\n",n);

}
```

讲解归纳与举例

1.若有定义 union { int b;char a[9];float x;} un;则 un 的内存空间是___字节。

【正确答案： 9】

【答案解析】： 共用体的基本概念。

讲解归纳与举例

2.若有定义 enum color {red,yellow=2,blue,white,black} r=white;执行 printf("%d",r);后的输出结果是（ ）。

【正确答案： 4】

【答案解析】： 默认时枚举常量总是从 0 开始，后续的枚举常量总是前一个的枚举常量加 1。可以在（仅仅在）声明枚举类型时，为命名枚举常量指定值。

本章小结

- (1) 理解结构体类型相关的基本概念。
- (2) 掌握结构体类型的定义、结构体变量的定义、初始化、结构体变量中成员的引用。
- (3) 掌握结构体数组的定义和使用。
- (4) 结构体变量指针的概念、定义、使用。
- (5) 使用结构体变量和结构体变量的指针来建立简单的链表。
- (6) 理解共用体和枚举类型的基本概念和定义。

第 10 章 利用文件保存数据

本章重难点分析

本章主要介绍文件的基本概念以及文件的类型、文件名、文件类型指针；文件的打开与关闭（`fopen, fclose`）；文件的顺序读写；文件的随机读写；系统定义的文件类型指针；回车换行符的转换；`fread`和 `fwrite` 函数；文件读写出错检测等等。

本章内容注意以下几点：

文件相关的基本概念	重点
文件的打开与关闭	重点&难点
文件的读写	难点

10.1 C文件的有关概念

10.1.1 什么是文件

➤ 文件有不同的类型，在程序设计中，主要用到两种文件：

（1）**程序文件**。包括源程序文件（后缀为.c）、目标文件（后缀为.obj）、可执行文件（后缀为.exe）等。这种文件的内容是程序代码。

（2）**数据文件**。文件的内容不是程序，而是供程序运行时读写的数据，如在程序运行过程中输出到磁盘（或其他外部设备）的数据，或在程序运行过程中供读入的数据。如一批学生的成绩数据，或货物交易的数据等。

本章主要讨论的是**数据文件**。

➤ 在以前各章中所处理的数据的输入和输出，从终端的键盘输入数据，运行结果输出到终端显示器上。常常需要将一些数据输出到磁盘上保存起来，以后使用。

这就要用到磁盘文件。

➤ 操作系统把各种设备都统一作为文件处理。

从操作系统的角度看，每一个与主机相联的输入输出设备都看作是文件。例如，

终端键盘是输入文件

显示屏和打印机是输出文件

➤ “**文件**”指存储在外部介质上数据的集合。

一批数据是以文件的形式存放在外部介质上的。

操作系统是以文件为单位对数据进行管理。

想找存放在外部介质上的数据，先按文件名找到所指定的文件，然后再从该文件读数据。

要向外部介质上存储数据也必须先建立一个文件（以文件名作为标志），才能向它输出数据。

➤ 输入输出是数据传送的过程，数据如流水一样从一处流向另一处，因此常将输入输出形象地称为**流（stream）**，即**输入输出流**。流表示了信息从源到目的端的流动。

输入操作时，数据从文件流向计算机内存。

输出操作时，数据从计算机流向文件。

➤ C语言把文件看作是一个字符（或字节）的序列。一个输入输出流就是一个字符流或字节（内容为二进制数据）流。

C的数据文件由一连串的字符（或字节）组成，中间没有分隔符，对文件的存取是以字符（字节）为单位的，允许对文件存取一个字符。输入输出数据流的开始和结束仅受程序控制而不受物理符号（如回车换行符）控制，这就增加了处理的灵活性。这种文件称为“**流式文件**”。

10.1.2 文件名

➤ 文件要有一个唯一的文件标识，以使用户识别和引用。

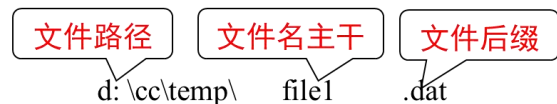
➤ 文件标识包括 3 部分：

(1) 文件路径

(2) 文件名主干

(3) 文件后缀

➤ 文件路径表示文件在外部存储设备中的位置。如：



◆ 表示file1.dat文件存放在d盘中的cc目录下的temp子目录下。

➤ 文件主干名命名规则遵循标识符的命名规则。

后缀一般不超过 3 个字母（doc、txt、dat、c、cpp、obj、exe、ppt、bmp 等）。

10.1.3 文件的分类

➤ 根据数据的组织形式，数据文件可分为 **ASCII 文件**和**二进制文件**。

数据在内存中是以二进制形式存储的，如果不加转换地输出到外存，就是二进制文件。

如果要求在外存上以 ASCII 代码形式存储，则需要在存储前进行转换。

ASCII 文件又称文本文件，每一个字节放一个字符的 ASCII 代码。

ASCII 形式

00110001	00110000	00110000	00110000	00110000
(1)	(0)	(0)	(0)	(0)

二进制形式

00100111	00010000
----------	----------

10.1.4 文件缓冲区

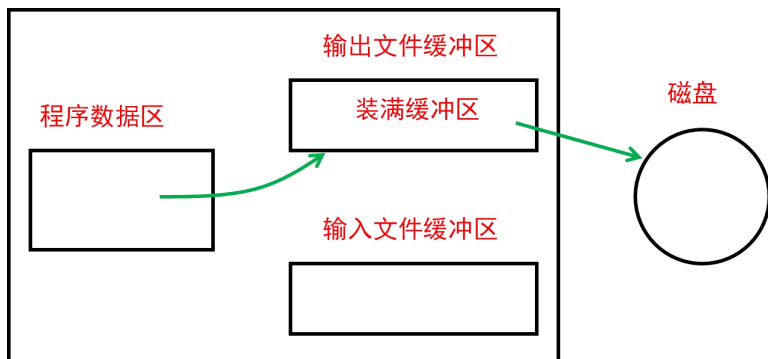
➤ ANSI C 标准采用“**缓冲文件系统**”处理数据文件。

所谓缓冲文件系统是指系统自动地在内存区为程序中每一个正在使用的文件开辟一个**文件缓冲区**。

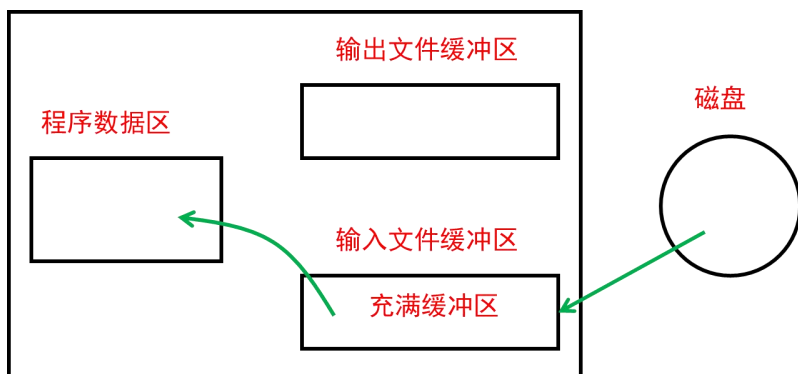
从内存向磁盘输出数据必须先送到内存中的缓冲区，装满缓冲区后才一起送到磁盘去。

如果从磁盘向计算机读入数据，则一次从磁盘文件将一批数据输入到内存缓冲区（充满缓冲区），然后再从缓冲区逐个地将数据送到程序数据区（给程序变量）。

➤ 从内存向磁盘输出数据



➤ 从磁盘向计算机读入数据



10.1.5 文件类型指针

缓冲文件系统中，关键的概念是“文件类型指针”，简称“文件指针”。

每个被使用的文件都在内存中开辟一个相应的文件信息区，用来存放文件的有关信息（如文件的名字、文件状态及文件当前位置等）。

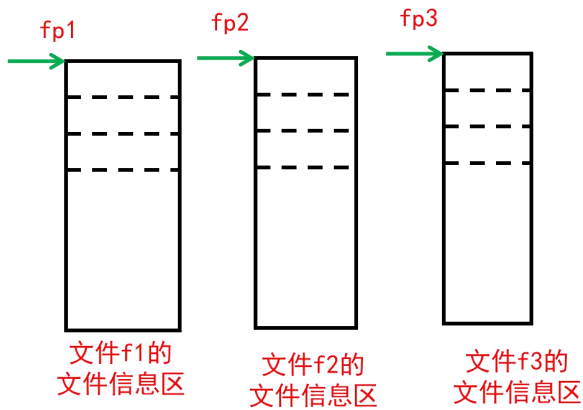
这些信息是保存在一个结构体变量中的。该结构体类型是由系统声明的，取名为 **FILE**。

声明 **FILE** 结构体类型的信息包含在头文件“**stdio.h**”中。

一般设置一个指向 **FILE** 类型变量的指针变量，然后通过它来引用这些 **FILE** 类型变量。

```
FILE *fp1,*fp2,*fp3;
```

//指向文件的指针变量



讲解归纳与举例

1.简要叙述文件指针的含义。

【参考答案：文件指针是用来存放文件的相关信息（文件名、文件状态、当前文件位置等）的数据区的结构指针。】

【答案解析】：文件指针的概念。

2.在 C 文件中，数据可用___和___两种代码形式存放。

【正确答案：第 1 空：ASCII，第 2 空：二进制】

【答案解析】：ASCII 文本文件，数据以字符形式表示，因而便于按字符方式逐个处理，也便于打印输出字符。二进制文件是将数据以内存中的存储形式直接存放到磁盘上。用二进制形式输出数据，可以节省存储空间和避免编码转换。由于一个字节并不对应一个字符，所以不能直接打印输出或编辑二进制文件。

10.2 文件的打开与关闭

10.2.1 用 fopen 函数打开数据文件

➤ 对文件读写之前应该“打开”该文件，在使用结束之后应“关闭”该文件。

所谓“**打开**”是指为文件建立相应的信息区（用来存放有关文件的信息）和文件缓冲区（用来暂时存放输入输出的数据）。

➤ 在编写程序时，在打开文件的同时，一般都指定一个指针变量指向该文件，也就是建立起指针变量与文件之间的联系，这样就可以通过该指针变量对文件进行读写。

所谓“关闭”是指撤销文件信息区和文件缓冲区。

➤ fopen 函数的调用方式为：

fopen(文件名,使用文件方式);

例如：

```
fopen(“a1”, “r”);
```

表示要打开名为“a1”的文件，使用文件方式为“读入”。

fopen 函数的返回值是指向 a1 文件的指针。

➤ 通常将 fopen 函数的返回值赋给一个指向文件的指针变量。如：

```
FILE *fp;
```

```
fp=fopen(“a1”, “r”);
```

fp 和文件 a1 相联系，fp 指向了 a1 文件。

➤ 在打开一个文件时，通知编译系统以下 3 个信息：

①需要访问的文件的名字

②使用文件的方式（“读”还是“写”等）

③让哪一个指针变量指向被打开的文件

使用文件方式参见教材表 10.1。

➤ 说明：

（1）最基本的是“r”，“w”，“a”三种方式。在其后加“b”表示是二进制文件，“+”表示既可读又可写。

（2）如果不能实现“打开”的任务，fopen 函数将会带回一个出错信息。出错的原因可能是用“r”方式打开一个并不存在的文件；磁盘出故障；磁盘已满无法建立新文件等。此时 fopen 函数将带回一个空指针值 **NULL**。

➤ 常用下面的方法打开一个文件：

```
if ((fp=fopen( "file1" , "r" ))==NULL)
{
    printf( "cannot open this file\n" );
    exit(0);//终止正在执行的程序
}
```

10.2.2 用 fclose 函数关闭文件

关闭文件用 fclose 函数。

fclose 函数调用的一般形式为：

fclose(文件指针);

例如：

```
fclose (fp);
```

如果不关闭文件将会丢失数据。

讲解归纳与举例

1.为什么对文件处理前需要打开文件，文件处理后需要关闭该文件？

【参考答案：通过使用文件打开函数，将使得的一个文件指针与具体的文件关联起来，进而可以通过文件指针来操作该文件；文件处理后需要关闭文件，它可以使得在缓冲区中的数据写到磁盘文件中，保证数据的正确性，同时也释放文件指针资源。】

【答案解析】：文件打开与关闭的理解。

2.若使用 fopen 函数打开一个新的二进制文件,对该文件进行读写操作,则文件打开方式字符串应该是_____。

【正确答案: "wb+"[或者]"w+b"】

【答案解析】： "wb+"或"w+b" 创建一个新文件用于读写数据，新文件会覆盖旧文件。

10.3 文件的顺序读写

在顺序写时，先写入的数据存放在文件中前面，后写入的数据存放在文件中后面。

在顺序读时，先读文件中前面的数据，后读文件中后面的数据。

对顺序读写来说，对文件读写数据的顺序和数据在文件中的物理顺序是一致的。

顺序读写需要用库函数实现。

10.3.1 向文件读写字符

➤ 读写一个字符的函数

函数名	调用形式	功能	返回值
fgetc	fgetc(fp)	从fp指向的文件读入一个字符	读成功，带回所读的字符，失败则返回文件结束标志 E O F (即-1)
fputc	fputc(ch,fp)	把字符ch写到文件指针变量fp所指向的文件中	写成功，返回值就是输出的字符；输出失败，则返回 E O F (即-1)

函数名 调用形式 功能 返回值

fgetc **fgetc(fp)** 从 fp 指向的文件读入一个字符 读成功，带回所读的字符，失败则返回文件结束标志 E O F (即-1)

fputc **fputc(ch,fp)** 把字符 ch 写到文件指针变量 fp 所指向的文件中 写成功，返回值就是输出的字符；输出失败，则返回 E O F (即-1)

例 10.1 从键盘输入一些字符，逐个把它们送到磁盘上去，直到用户输入一个“#”为止。

解题思路：用 fgetc 函数从键盘逐个输入字符，然后用 fputc 函数写到磁盘文件即可。

```
#include <stdio.h>
```

```
#include <stdlib.h> //用 exit 函数时加
```

```
void main()
```

```
{ FILE *fp;
```

```
    char ch,filename[10];
```

```
    printf("请输入所用的文件名: ");
```

```
    scanf("%s",filename);
```

//w 表示只写

```
if((fp=fopen(filename, "w"))==NULL)
{
    printf("无法打开此文件\n");
    exit(0);
}

ch=getchar( );

printf( "请输入一个字符串(以#结束): ");

ch=getchar( );
while(ch!= '#' )
{
    fputc(ch,fp);
    putchar(ch);
    ch=getchar();
}

fclose(fp);

putchar(10);//换行字符的 ASCII 代码是 10
}
```

例 10.2 将一个磁盘文件中的信息复制到另一个磁盘文件中。今要求将上例建立的 file1.dat 文件中的内容复制到另一个磁盘文件 file2.dat 中。

➤ 解题思路：处理此问题的算法是：从 file1.dat 文件中逐个读入字符，然后逐个输出到 file2.dat 中。

```
#include <stdio.h>

#include <stdlib.h>

void main( )
{
    FILE *in,*out;

    char  ch,infile[10],outfile[10];

    printf("输入读入文件的名字:");

    scanf("%s",infile);

    printf("输入输出文件的名字:");

    scanf( "%s" ,outfile);

    if((in=fopen(infile, "r"))==NULL)
```

```

{printf("无法打开此文件\n");
    exit(0);}

if((out=fopen(outfile,"w"))==NULL)
{printf("无法打开此文件\n");
    exit(0); }
ch=fgetc(in);
while(!feof(in))
{ fputc(ch,out);
    putchar(ch);
    ch=fgetc(in);
}
putchar(10);
fclose(in);
fclose(out);
return 0; }
    
```

检查当前读写位置是否移到文件末尾，等同于：
 while(ch!=-1) 或者
 While(ch!=EOF)

程序中的r和w如果改为rb和wb，则复制一个二进制文件

10.3.2 向文件读写一个字符串

➤ 读写一个字符串的函数

函数名	调用形式	功能	返回值
fgets	fgets(str,n,fp)	从fp指向的文件读入长度为(n-1)的字符串，存放到字符数组str中	读成功，返回地址str，失败则返回NULL)
fputs	fputs(str,fp)	str所指向的字符串写到文件指针变量fp所指向的文件中	写成功，返回0；否则返回非0值

➤ 说明：

(1) 用 fgets 函数可以从指定的文件读入一个字符串。如：**fgets(str,n,fp);**
 fgets(str,n,fp);中 n 是要求得到的字符个数，但实际上只读 n-1 个字符，然后在最后加一个 ‘\0’ 字符，这样得到的字符串共有 n 个字符，把它们放到字符数组 str 中。
 如果在读完 n-1 个字符之前遇到换行符 “\n” 或文件结束符 EOF，读入即结束，但将所遇到的换行符 “\n” 也作为一个字符读入。
 执行 fgets 成功，返回 str 数组首地址，如果一开始就遇到文件尾或读数据错，返回 NULL。

(2) 用 `fputs` 函数可以向指定的文件输出一个字符串。如：`fputs(" Chian",fp);`

`fputs` 函数中第一个参数可以是字符串常量、字符数组名或字符型指针。

字符串末尾 ‘\0’ 不输出。

输出成功，函数值为 0；失败，函数值为 EOF。

例 10.3 从键盘读入若干个字符串，对它们按字母大小的顺序排序，然后把排好序的字符串送到磁盘文件中保存。

➤ 解题思路：为解决问题，可分为三个步骤：

从键盘读入 n 个字符串，存放在一个二维字符数组中，每一个一维数组存放一个字符串；

对字符数组中的 n 个字符串按字母顺序排序，排好序的字符串仍存放在字符数组中；

将字符数组中的字符串顺序输出。

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
void main()
{ FILE *fp;
  char  str[3][10],temp[10];
  int i,j,k,n=3;
  printf( "Enter strings:\n" );
  for(i=0;i<n;i++)
    gets(str[i]);
  for(i=0;i<n-1;i++)
  { k=i;
    for(j=i+1;j<n;j++)      if(strcmp(str[k],str[j])>0)      k=j;
    if(k!=i)
    { strcpy(temp,str[i]);      strcpy(str[i],str[k]);
      strcpy(str[k],temp);
    }
  }
  if((fp=fopen( "D:\\CC\\string.dat" ,
w" ))==NULL)
```

```
{printf("can't open file!\n");  
    exit(0);}  
printf("\nThe new sequence:\n");  
    for(i=0;i<n;i++){  
        fputs(str[i],fp);  
        fputs( "\n" ,fp);  
        printf( "%s\n" ,str[i]);  
    }  
}
```

- 为了验证输出到磁盘文件中的内容，可以编写出以下的程序，从该文件中读回字符串，并在屏幕上显示。

```
#include <stdio.h>  
#include <stdlib.h>  
void main()  
{ FILE *fp;  char  str[3][10];  int i=0;  
  if((fp=fopen( "D:\\CC\\string.dat" ,  
                "r" ))==NULL)  
  {printf("can't open file!\n");  
   exit(0);}  
  while(fgets(str[i],10,fp)!=NULL)  
  { printf("%s",str[i]);  
    i++;  }  
  fclose (fp);  
}
```

10.3.3 文件的格式化读写

- 一般调用方式为：

fprintf(文件指针，格式字符串，输出表列)；

fscanf(文件指针，格式字符串，输入表列)；

如：

fprintf (fp, "%d,%6.2f" ,i,f);

```
fscanf(fp, "%d,%f", &i, &f);
```

10.3.4 用二进制方式读写文件

➤ 一般调用形式为：

```
fread(buffer, size, count, fp);
```

```
fwrite(buffer, size, count, fp);
```

➤ **buffer**：是一个地址

对 **fread** 来说，它是用来存放从文件读入的数据的存储区的地址。

对 **fwrite** 来说，是要把此地址开始的存储区中的数据向文件输出。

size：要读写的字节数

count：要读写多少个数据项

fp：FILE 类型指针

例 10.4 从键盘输入 10 个学生的有关数据，然后把它们转存到磁盘文件中。

```
#include <stdio.h>
```

```
#define SIZE 10
```

```
struct student_type
```

```
{ char name[10];
```

```
    int num;
```

```
    int age;
```

```
    char addr[15];
```

```
}stud[SIZE];
```

```
void save( )
```

```
{ FILE *fp; int i;
```

```
    if((fp=fopen("stu_dat", "wb"))==NULL)
```

```
    { printf("cannot open file\n");
```

```
        return;
```

```
    }
```

```
    for(i=0; i<SIZE; i++)
```

```
        if(fwrite(&stud[i], sizeof(struct student_type), 1, fp)!=1)
```

```
            printf("file write error\n");
```

```
        fclose(fp);}
```

```
void main()
```

当前路径下的文件

10+4+4+15=33，实际
上开辟36字节，
是4的倍数

```
{ int i;

    printf("enter data of students:\n");

    for(i=0;i<SIZE;i++)

        scanf("%s%d%d%s",

            stud[i].name,&stud[i].num,

            &stud[i].age,stud[i].addr);

    save( );

}
```

- 为了验证在磁盘文件“stu_dat”中是否已存在此数据，可以用以下程序从“stu_dat”文件中读入数据，然后在屏幕上输出。

```
#include <stdio.h>

#define SIZE 10

struct student_type

{ char name[10];

    int num;

    int age;

    char addr[15];

}stud[SIZE];

void main( )

{int i;   FILE *fp;

    if((fp=fopen("stu_dat","rb"))==NULL)

        {printf("cannot open file\n"); return; }

    for(i=0;i<SIZE;i++)

        {fread (&stud[i],

            sizeof(struct student_type),1,fp);

            printf( “ %-10s %4d %4d  %-15s\n” ,

                stud[i].name,stud[i].num,

                stud[i]. age,stud[i].addr);

        }

    fclose (fp);

}
```

- 如果从已有的二进制文件“stu_list”中，读入数据并输出到“stu_dat”文件中，可以编写一个如下的 load 函数，从磁盘文件“stu_list”中读二进制数据，并存放在 stud 数组中。

```
void load()  
{ FILE *fp;   int i;   if((fp=fopen("stu_list","rb"))==NULL)  
    {printf("cannot open infile\n"); return;}  
    for(i=0;i<SIZE;i++)  
        if(fread(&stud[i],sizeof(struct  
student_type),1,fp)!=1)  
            { if(feof(fp))  
                { fclose(fp);   return;   }  
                printf("file read error\n");  
            }  
        fclose (fp);  
}
```

讲解归纳与举例

1. 设文件 file1.c 的内容为 COMPUTER。写出下面程序执行后的运行结果_____。

```
#include <stdio.h>  
  
int main() {  
    char ch;   FILE *fp;  
  
    if ((fp=fopen("file1.c","r"))==NULL) { printf( "不能打开文件\n" );  
        return(-1); }  
  
    while (!feof(fp)){  
        ch = fgetc( fp );  
  
        if ( ch >= 'A' && ch <= 'Z' )   fputc( ch + 32, stdout ); }    fclose( fp );  
  
    return(0); }
```

【正确答案： computer】

【答案解析】：

ch=fgetc(fp);

//从 fp 关联的文件 file1.c 中读取一个字符存入 ch

```
if(ch>='A'&&ch<='Z') fputc(ch+32,stdout);
```

//如果 ch 是大写字母，则把 ch+32 对应的字符（即小写形式）写入到标准输出文件 stdout 即显示器。

10.4 文件的随机读写

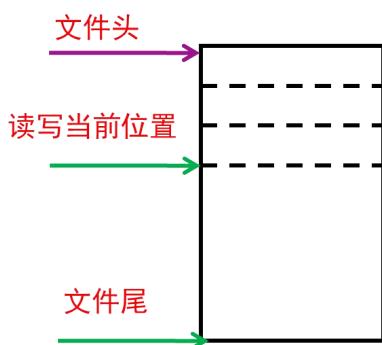
- 对文件进行顺序读写比较容易理解，也容易操作，但有时效率不高。

随机访问不是按数据在文件中的物理位置次序进行读写，而是可以对任何位置上的数据进行访问，显然这种方法比顺序访问效率高得多。

10.4.1 文件位置标记及其定位

1. 文件的位置标记

为了对读写进行控制，系统为每个文件设置了一个**位置指针**，用来指示当前的读写位置。



- 一般情况下，在对字符文件进行**顺序读**时，文件标记指向文件开头，进行读的操作时，就读第一个字符，然后文件标记向后移一个位置，在下一次读操作时，就将位置标记指向的第二个字符读入。依此类推，直到遇文件尾，结束。
- 如果是**顺序写**文件，则每写完一个数据后，文件标记顺序向后移一个位置，然后在下一次执行写操作时把数据写入指针所指的位置。直到把全部数据写完，此时文件位置标记在最后一个数据之后。

可以根据读写的需要，人为地移动了文件标记的位置。文件标记可以向前移、向后移，移到文件头或文件尾，然后对该位置进行读写——**随机读写**。

随机读写可以在任何位置写入数据，在任何位置读取数据。

2. 文件位置指针的定位

可以强制使文件位置指针指向指定的位置。

用以下函数实现：

- (1) 用 **rewind 函数**使文件指针指向文件头：

rewind 函数的作用是使文件指针重新返回文件的开头，此函数没有返回值。

例 10.5 有一个磁盘文件，第一次将它的内容显示在屏幕上，第二次把它复制到另一文件上。

➤ 解题思路：

因为在第一次读入完文件内容后，位置指针已指到文件的末尾，如果再接着读数据，就遇到文件结束标志，`feof` 函数的值等于 1（真），无法再读数据。

必须在程序中用 `rewind` 函数使位置指针返回文件的开头。

```
#include<stdio.h>

void main()
{ FILE *fp1,*fp2;

  fp1=fopen( "file1.dat" , "r" );
  fp2=fopen( "file2.dat" , "w" );
  while(!feof(fp1))
    putchar(getc(fp1));
  putchar(10);
  rewind(fp1); //使文件位置标记返回文件头
  while(!feof(fp1))
    putc(getc(fp1),fp2);
  fclose(fp1);   fclose(fp2);
}
```

（2）用 `fseek` 函数移动位置指针

`fseek` 函数的调用形式为：

`fseek`（文件类型指针，位移量，起始点）

起始点 0 代表“文件开始”，1 为“当前位置”，2 为“文件末尾”。

➤ `fseek` 函数中的“起始点”的表示方法

起始点	名 字	用数字代表
文件开始	SEEK_SET	0
文件当前位置	SEEK_CUR	1
文件末尾	SEEK_END	2

➤ 位移量指以起始点为基点，向前移动的字节数。位移量应是 `long` 型数据（在数字的末尾加一个字母 L）。

`fseek` 函数一般用于二进制文件。下面是 `fseek` 函数调用的几个例子：

```
fseek (fp,100L,0);
```



```
fseek (fp,50L,1);
```

```
fseek (fp,-10L,2);
```

（3）用 `ftell` 函数测定位置指针的当前位置

`ftell` 函数的作用是得到流式文件中位置指针的当前位置。

由于文件中的位置指针经常移动，人们往往不容易知道其当前位置，所以常用 `ftell` 函数得到当前位置。用相对于文件开头的位移量来表示。

如果 `ftell` 函数返回值为 `-1L`，表示出错。

```
i=ftell(fp);  
  
if(i==-1L) printf( "error\n" );
```

10.4.2 随机读写文件

例 10.6 在磁盘文件 `stu.dat` 上已存有 10 个学生的数据（`stu.dat` 是执行例 10.4 程序时建立的数据文件）。今要求将该文件中的第 1、3、5、7、9 个学生数据输入计算机，并在屏幕上显示出来。

➤ 解题思路：

（1）按“二进制只读”的方式打开文件。

（2）将文件位置指针指向文件的开头，然后从磁盘文件读入一个学生的信息，并把它显示在屏幕上。

（3）再将文件位置指针指向文件中第 3，5，7，9 个学生的数据区的开头，从磁盘文件读入相应学生的信息，并把它显示在屏幕上。

（4）关闭文件。

```
#include<stdio.h>  
  
#include <stdlib.h>  
  
struct student_type  
{ char name[10];  
  int num;  
  int age;  
  char addr[15];  
}stud[10];  
  
void main()  
{ int i;  FILE *fp;  
  
if((fp=fopen( "stu_dat" , "rb" ))==NULL)
```

```
{ printf("can not open file\n"); exit(0); }  
  
for(i=0;i<10;i+=2)  
{ fseek(fp,i*sizeof(struct student_type),0);  
fread(&stud[i], sizeof(struct student_type),1,fp);  
printf( "%-10s %4d %4d %-15s\n" ,  
        stud[i].name,stud[i].num,  
        stud[i].age,stud[i].addr;  
        }  
fclose(fp);}
```

讲解归纳与举例

1.函数调用语句 `fseek(fp,-20L,2)` 的含义是（ ）。

【参考答案：将文件位置指针从文件末尾向后退 20 个字节】

【答案解析】：Fseek 函数的理解。

2.函数 `rewind` 的作用是（ ）。

【参考答案：使位置指针重新返回文件的开头】

【答案解析】：rewind 函数使文件位置重新回到文件开头，其函数原型为：

`void rewind(FILE *stream);` 参数 `stream` 是已打开的文件指针。

10.5 提高部分

10.5.1 系统定义的文件类型指针

➤ 在标准输入输出库中（stdio.h），系统定义了三个 FILE 型的指针变量：

1.**stdin (标准输入文件指针)**。指向在内存中与键盘相应的文件信息区，因此用它进行输入就蕴含了从键盘输入。

2.**stdout (标准输出文件指针)**。指向在内存中与显示器屏幕相应的文件信息区，因此用它进行输出就蕴含了输出到显示器屏幕。

3.**stderr (标准错误文件指针)**，用来输出出错的信息，它也指向在内存中与显示器屏幕相应的文件信息区，因此在程序运行时的出错的信息就输出到显示器屏幕。

10.5.2 回车换行符的转换

在向计算机输入文本文件时，将回车换行符转换为换行符，在输出时把换行符转换成为回车和换行两个字符。

在用二进制文件时，不进行这种转换，在内存中的数据形式与输出到外部文件中的数据形式完全一致，一一对应。

10.5.3 fread 和 fwrite 函数用于二进制文件的输入输出

因为它们是按数据块的长度来处理输入输出的，按数据在存储空间存放的实际情况原封不动地在磁盘文件和内存之间传送，一般不会出错。如果在 ASCII 文件和二进制形式之间传送，在字符发生转换的情况下很可能出现与原设想的情况不同。

10.5.4 文件读写的出错检测

1.ferror 函数

➤ ferror 函数的一般调用形式为

ferror(fp);

如果返回值为 0，表示未出错，否则表示出错。

每次调用输入输出函数，都产生新的 ferror 函数值，因此调用输入输出函数后立即检查。

调用 fopen 时，ferror 的初始值自动置为 0。

2.clearerr 函数

作用是使文件错误标志和文件结束标志置为 0。

调用一个输入输出函数时出现错误（ferror 值为非零值），立即调用 clearerr(fp)，使 ferror(fp)值变 0，以便再进行下一次检测。

只要出现文件读写错误标志，它就一直保留，直到对同一文件调用 clearerr 函数或 rewind 函数，或任何其他一个输入输出函数。

讲解归纳与举例

1.在 C 程序中，可把整型数据以二进制形式存放到文件中的函数（ ）。

【正确答案：fwrite】

【答案解析】：fwrite 函数。

2.写出下面程序执行后的运行结果___。

```
#include <stdio.h>

int main() {
    int i, a[4] = { 1, 2, 3, 4 }, b;
    FILE *fp; fp = fopen( "data.dat", "wb" );
    for ( i = 0; i < 4; i++ )
        fwrite( &a[i], sizeof(int), 1, fp );      fclose( fp );
    fp = fopen( ""data.dat"", ""rb"" );
    fseek( fp, -2L * sizeof(int), SEEK_END );
    fread( &b, sizeof(int), 1, fp );
    fclose( fp );
    printf( "%d", b );
    return(0);
}
```

【正确答案：3】

【答案解析】：

fseek(fp,-2L*sizeof(int),SEEK_END);语句执行后将文件位置指针从文件末尾向文件头方向移动两个整数的位置，即文件位置指针被重置在整数 3 的前面。后续读取文件时将从整数 3 开始读取。

本章小结

- (1) 理解文件的基本概念、文件类型。
- (2) 掌握文件指针的概念及其应用。
- (3) 掌握文件的打开与关闭。
- (4) 了解文件的顺序读写和随机读写。
- (5) 熟悉文件操作相关的函数 P340 表 10.5。
- (6) 理解简单的案例以及文件的基本操作。

对应网课（每年按照最新考纲更新）

【优课必过】网校

<http://zikao.p1tao.com/mall/detail?id=26359621>

开课加叶学长微信prince-zip咨询

课程: [44-K0033] 专升本计算机基础与程序设计

课程精讲班

共99讲 总时长2695分钟

普通高等教育招生考试
专升本 计算机基础与程序设计
(数据结构)

米秋香老师
计算机基础与程序设计-数据结构

优课必过

第一至三节 数据结构的研究内容、... 试学

课后练习

第四至五节 算法和算法分... 试学

扫一扫看课



00:02 / 49:24

线路1 超清 倍速



正版网课内部价开通
加叶学长微信